

Python Machine Learning

Algoritmos Machine Learning

Guilherme Silveira
gfsilveira@gmail.com

08/08/2026

5 – Algoritmos



4 – Algoritmos

- k-Means;
- DBSCAN;
- k-NN;
- Logistic Regression;
- SVM;
- Decision Trees;
- Random Forests;
- Naive Bayes;
- Simple/Multiple Linear Regression;
- Neural Networks



Quem é o chef que descobre a receita?

Dados



Algoritmos



Modelos



Predições



Ingredientes

Chefs

Receitas

Pratos

Variável Dependente

Variável		Tipo	Característica	Exemplo	Valores	ML
Categórica	Qualitativa	Nominal	Nomes	Nome proteínas	IL-10, IL-12	Clusterização Classificação
Categórica	Qualitativa	Ordinal	Ordens	Cascata de reação	Citrato -> Isocitrato	Clusterização Classificação
Numérica	Quantitativa	Contínua	Mensurável	Concentração química	1,34 mg, 5,0 mg	Regressão
Numérica	Quantitativa	Discreta	Contagem	Nº proteínas	5, 10, 1.042	Regressão

Quais as abordagens existem em ML?



Não
supervisionada

Clusterização

Regressão



Semi-
supervisionada

Classificação

Regressão

Supervisionada

Classificação

Regressão

Reforço

Clusterização

Regressão

$$\begin{aligned}f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\&= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\&= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\&= \lim_{h \rightarrow 0} h(2x + h)\end{aligned}$$

Quais as abordagens existem em ML?

Clusterização

k-means;
DBSCAN;
Agrupamento Hierárquico
Neural Networks;

Classificação

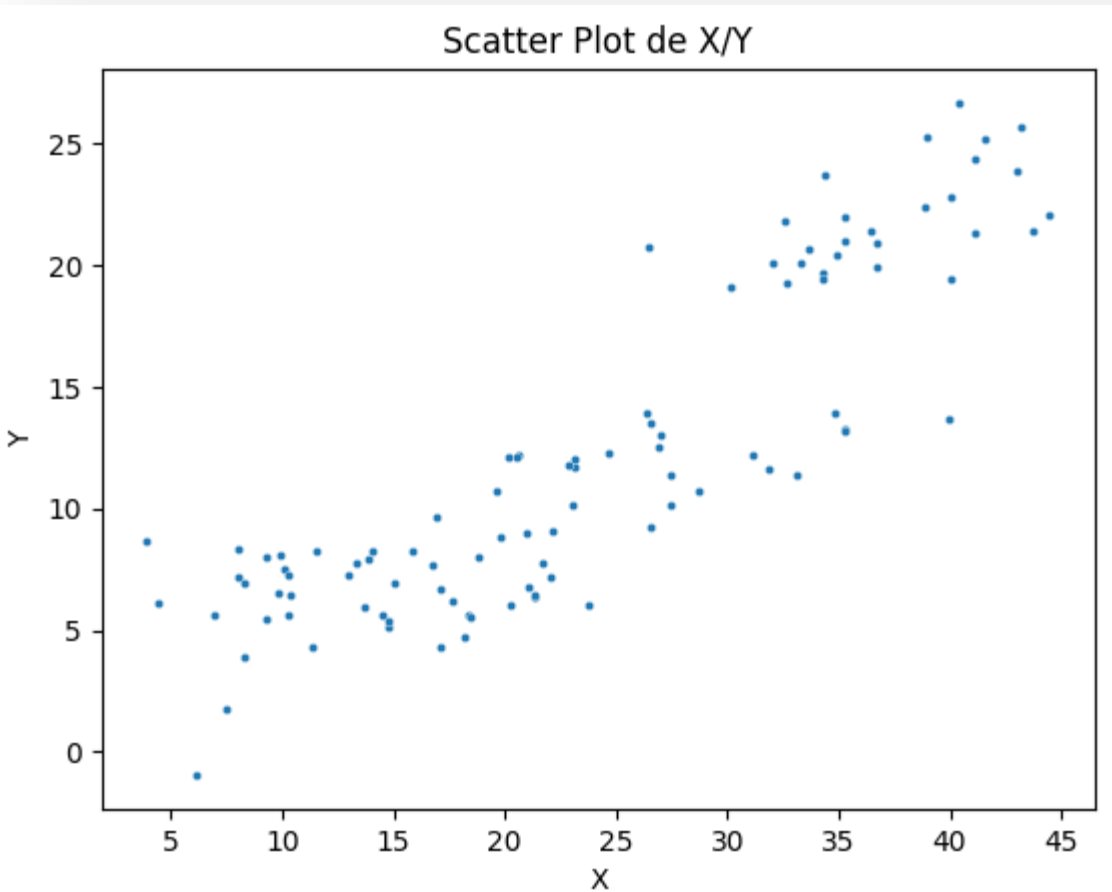
K-Nearest Neighbors;
Logistic Regression;
SVM;
Decision Trees;
Random Forests;
XGBoost
Naive Bayes;
Neural Networks;

Regressão

Linear Simple;
Linear Multipla;
SVM;
Naive Bayes;
Neural Networks;

The image shows a chalkboard with handwritten mathematical derivations. The first line shows the definition of the derivative: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. The second line shows the application of this definition to $f(x) = x^2$: $f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$. The third line shows the expansion of the numerator: $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$. The fourth line shows the simplification: $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$. The fifth line shows the final result: $= \lim_{h \rightarrow 0} h(2x + h)$.

Dados



Dados



Ingredientes

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

StandardScaler

Padronização dos dados

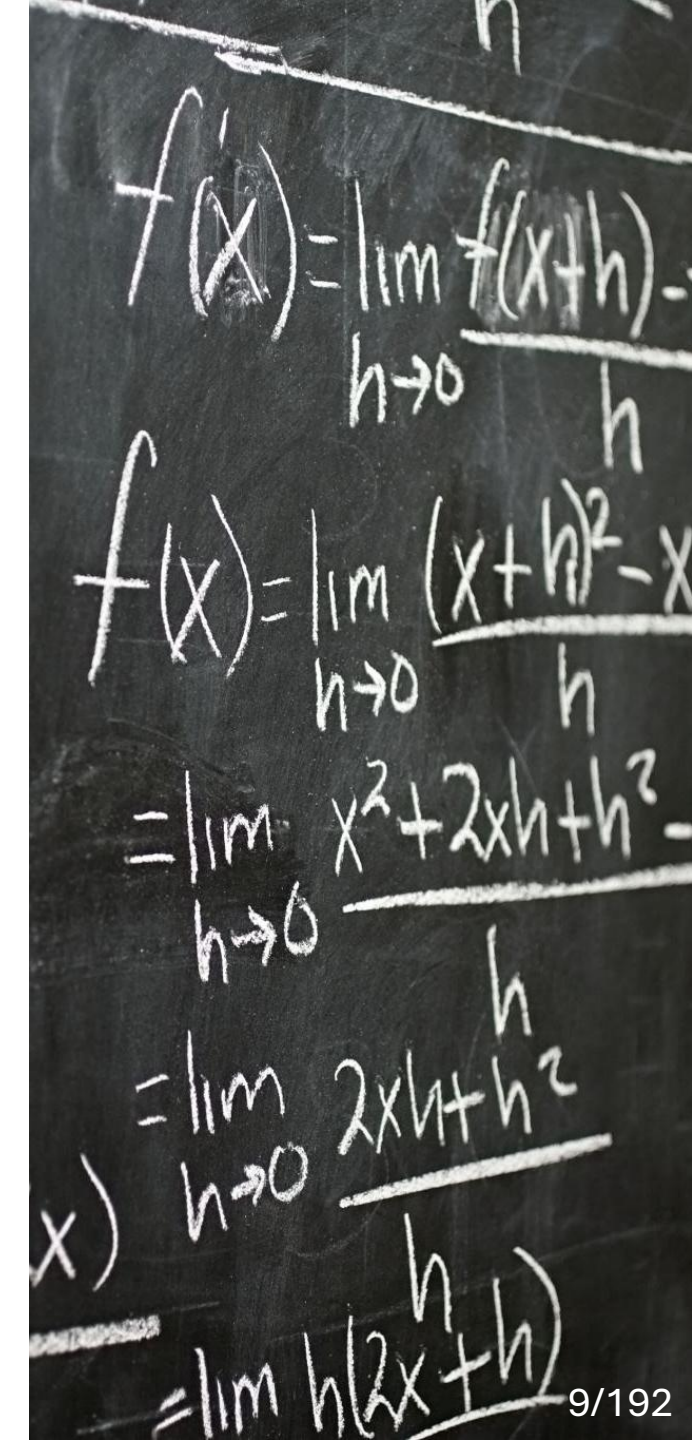
$$z = (x - u) / s$$

x -> variável

u -> média

s -> desvio padrão

Transformação dos dados para que tenham uma média de 0 e um desvio padrão de 1, garantindo assim uma distribuição normal.



Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Padronização – StandardScaler

Padronização dos dados

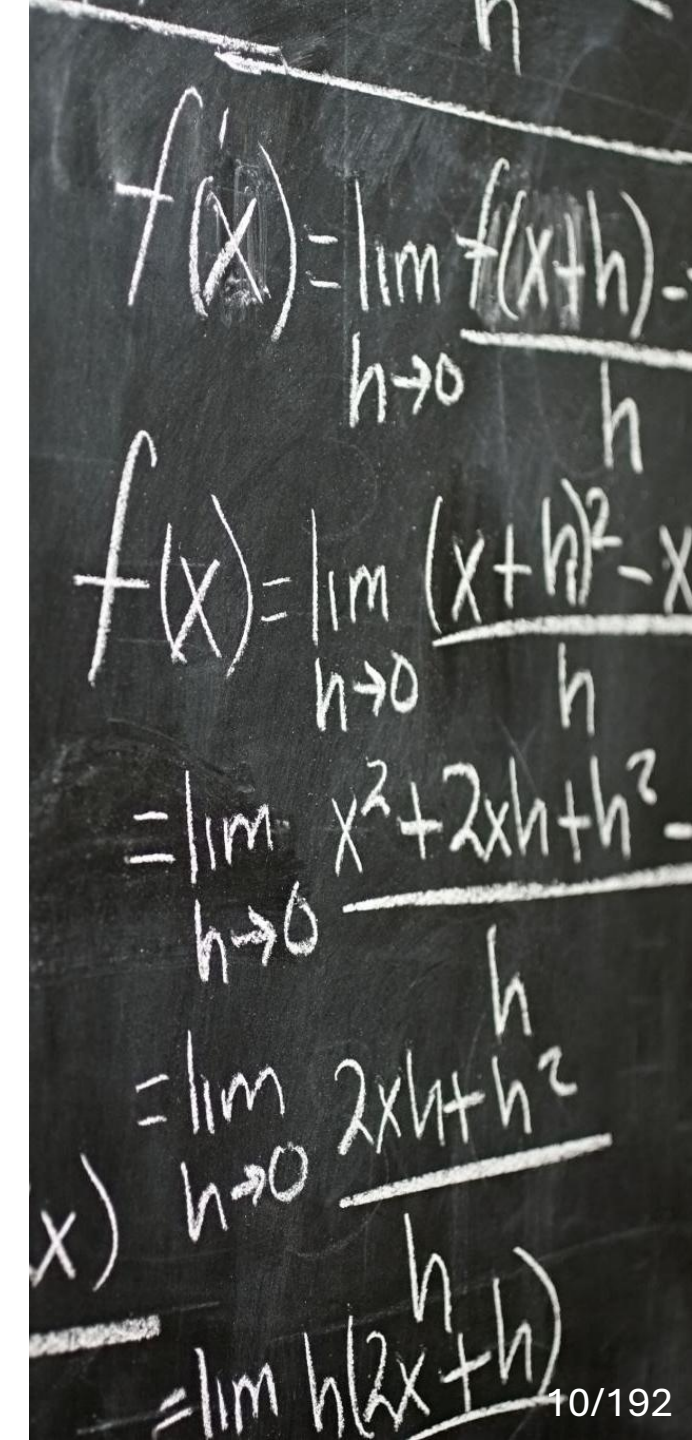
$$x_p = (x - u) / s$$

x -> variável

u -> média

s -> desvio padrão

Transformação dos dados para que tenham uma média de 0 e um desvio padrão de 1, garantindo assim uma distribuição normal.



Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Normalização – MinMaxScaler

Padronização dos dados

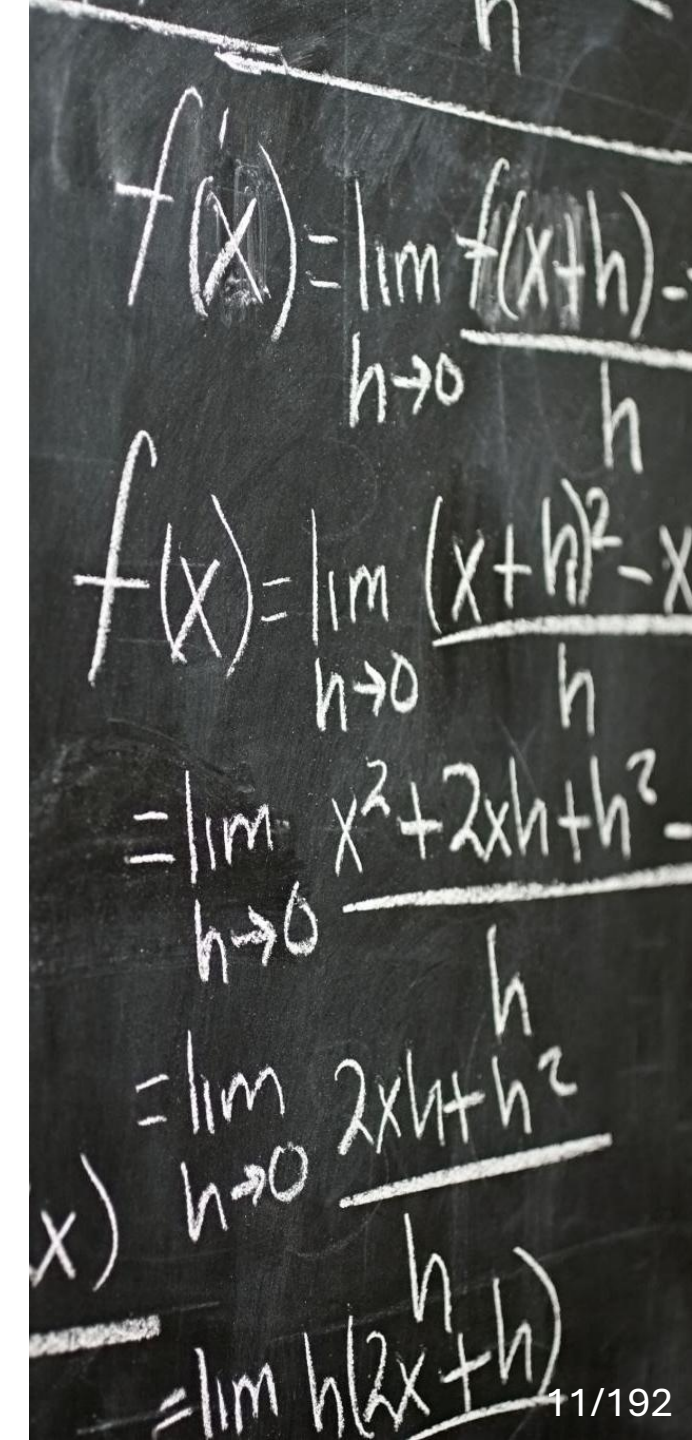
$$x_n = (x - x_{\min}) / (x_{\max} - x_{\min})$$

x -> variável

$x_{\max}$ -> Máximo de X

$x_{\min}$ -> Mínimo de X

A normalização de dados numéricos coloca variáveis de diferentes escalas em um intervalo comum. A técnica mais comum é o dimensionamento Min-Max



Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Clusterização



Clusterização

k-means;

DBSCAN;

Agrupamento Hierárquico

Neural Networks;

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Não Supervisionada

k-means;

DBSCAN;

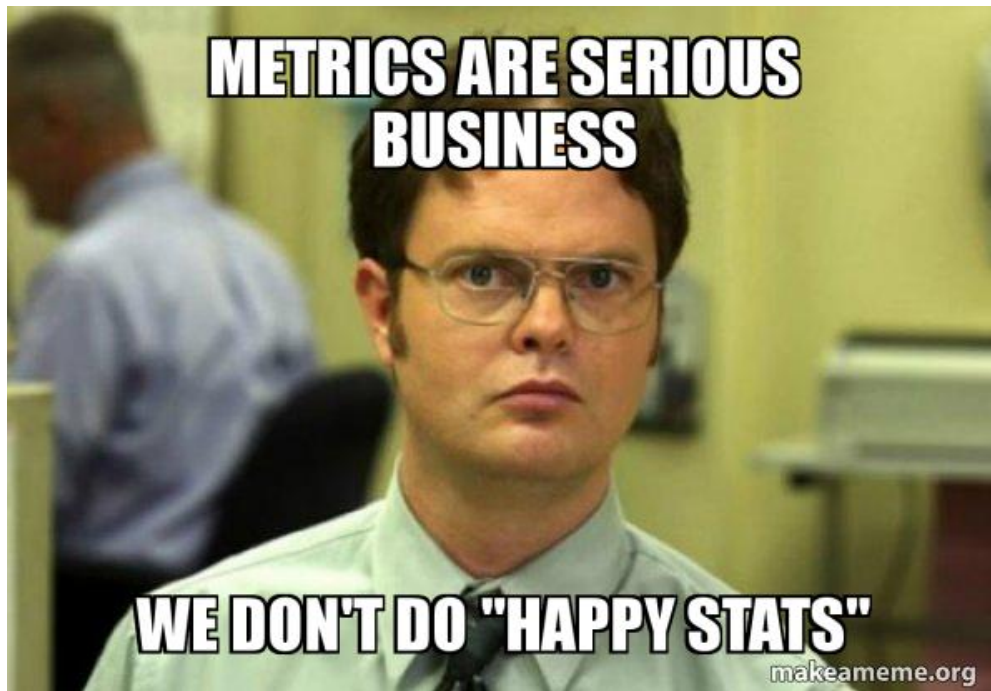
Neural Networks;



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Métricas

Qual é o melhor modelo?



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

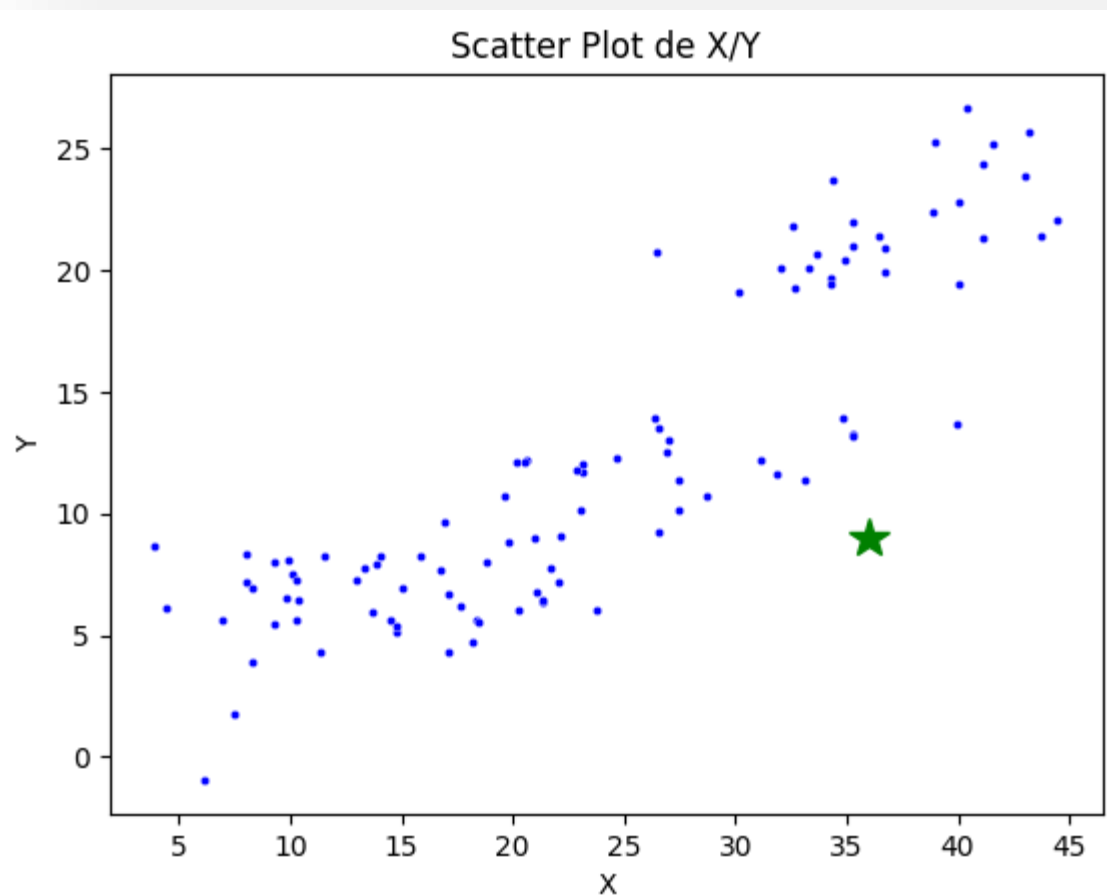
k-means



K-means é um algoritmo de aprendizado de máquina **não supervisionado** usado para particionar dados em **k clusters** distintos e não sobrepostos. Ele funciona atribuindo iterativamente pontos de dados ao centroide do cluster mais próximo (média) e, em seguida, recalculando os centroides para que sejam a média dos pontos atribuídos a eles. O objetivo é minimizar a distância entre os pontos de dados e seus centroides atribuídos.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means – Hiperparâmetro

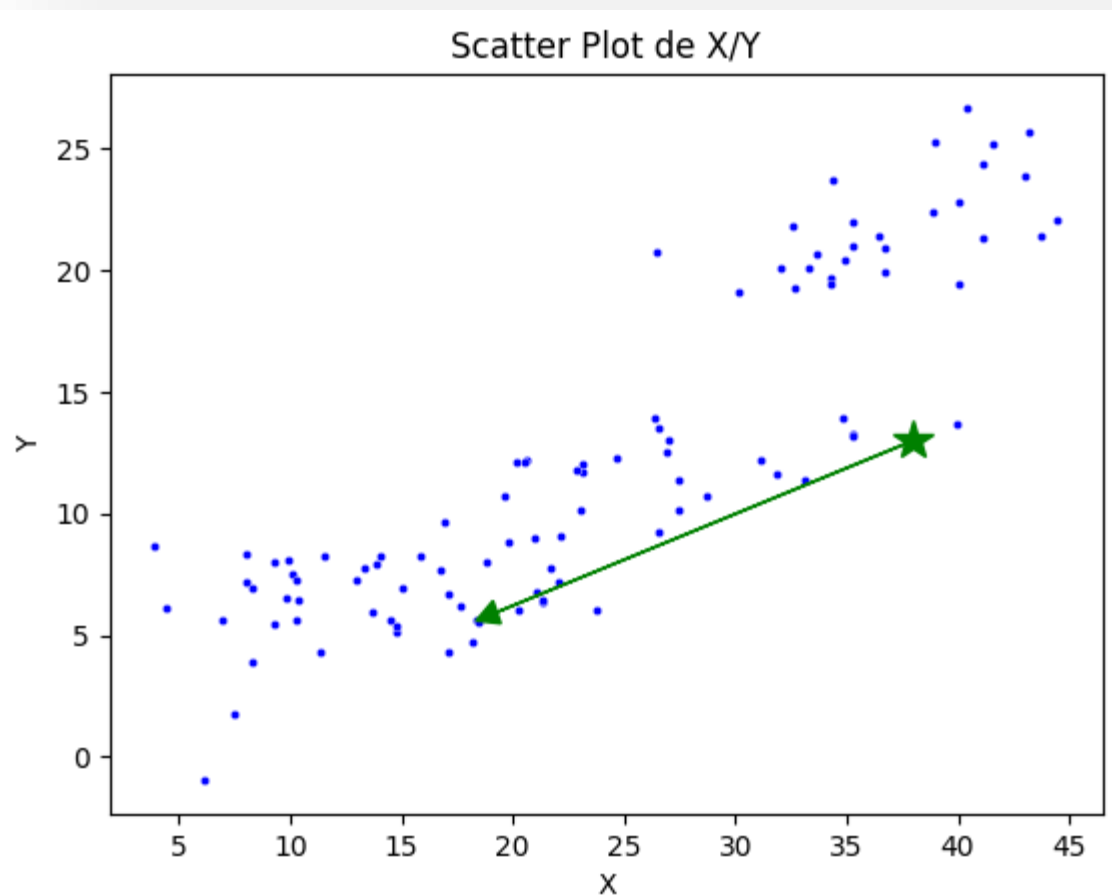


n_clusters
Número de clusters

k = 1

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

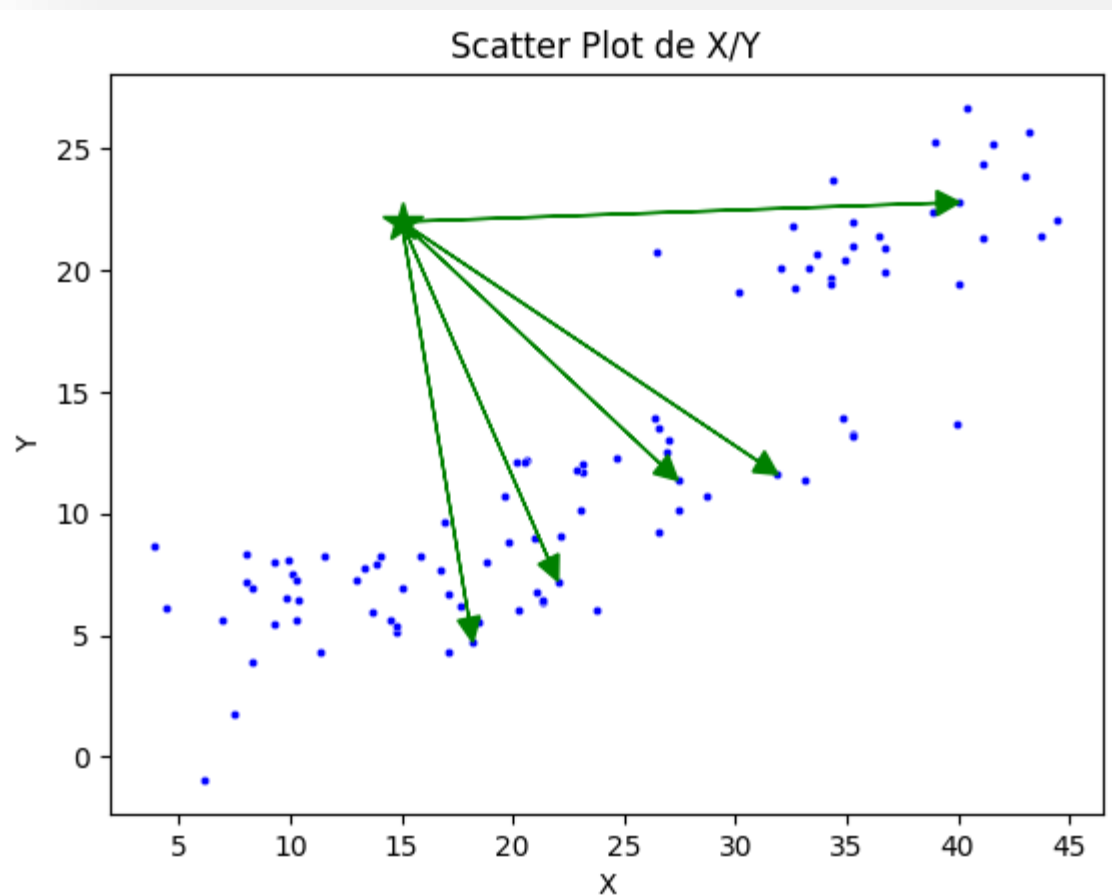
k-means



$k = 1$

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means



k = 1

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means – Método do Cotovelo

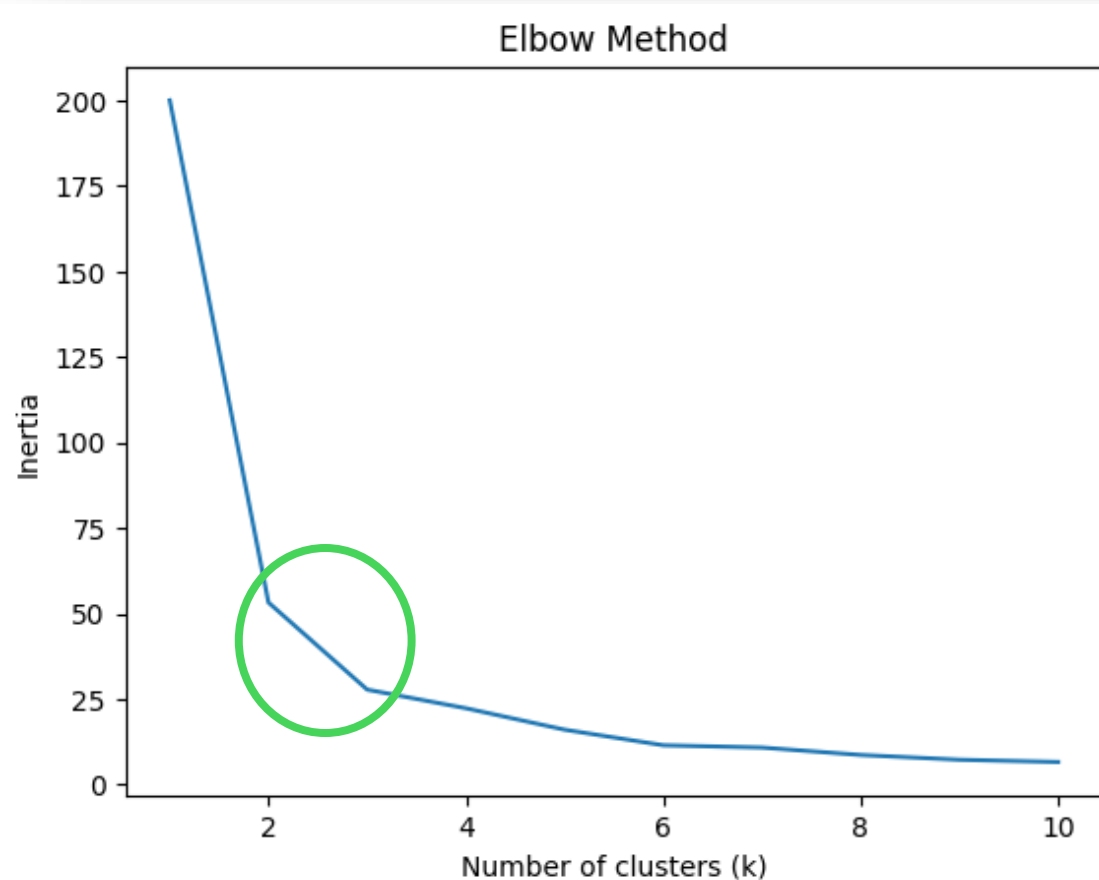
Inercia

Uma métrica que mede a compactação de um cluster calculando a soma das distâncias quadradas entre cada ponto de dados e seu centróide atribuído

The image shows a handwritten derivation of the derivative of the inertia function $f(x) = (x+h)^2 - x^2$. The steps are as follows:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means – Método do Cotovelo



Inercia

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

k-means – Método da Silhueta média

Para o ponto de dados $i \in C_I$ (ponto de dados i no cluster C_I), seja

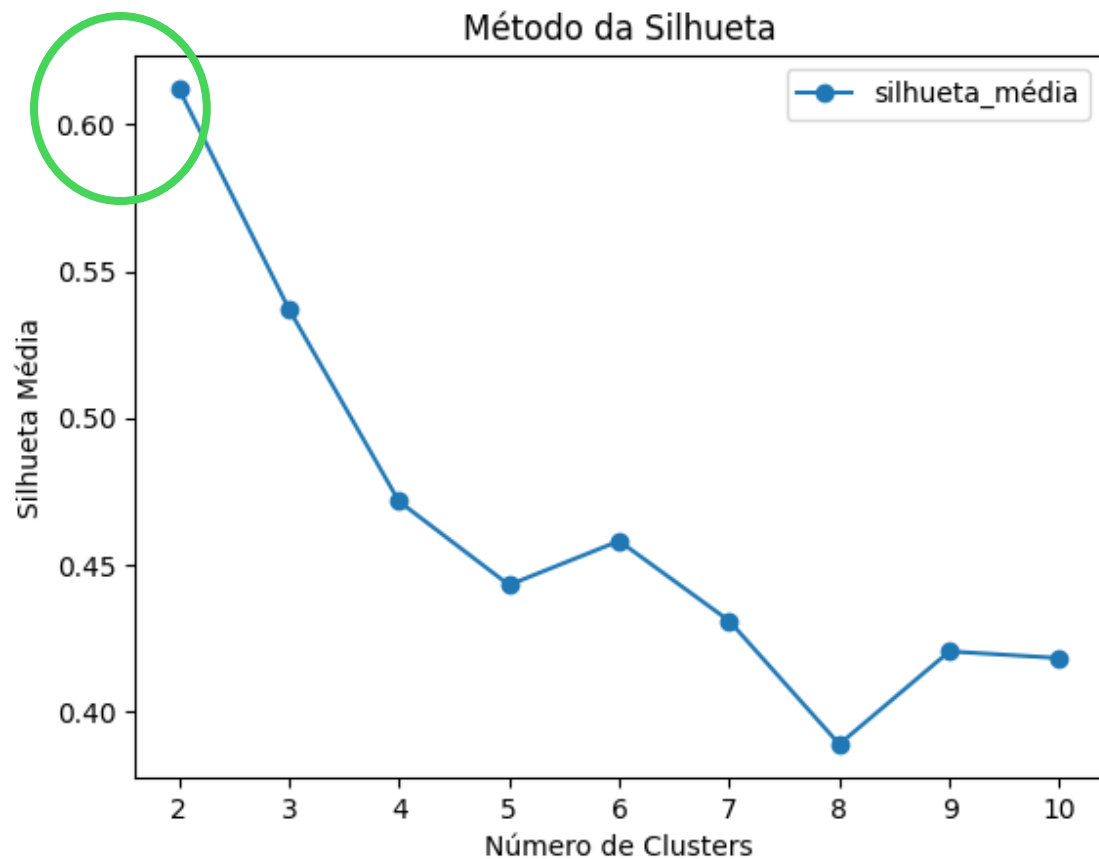
$$a(i) = \frac{1}{|C_I| - 1} \sum_{j \in C_I, i \neq j} d(i, j)$$

a distância média entre i e todos os outros pontos de dados no mesmo cluster, onde $|C_I|$ é o número de pontos pertencentes ao cluster C_I , e $d(i, j)$ é a distância entre os pontos de dados i e j no cluster C_I (dividimos por $|C_I| - 1$ porque não incluímos a distância $d(i, i)$ na soma). Podemos interpretar $a(i)$ como uma medida de quão bem i está atribuído ao seu cluster (quanto menor o valor, melhor a atribuição).

Handwritten mathematical derivation on a chalkboard:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

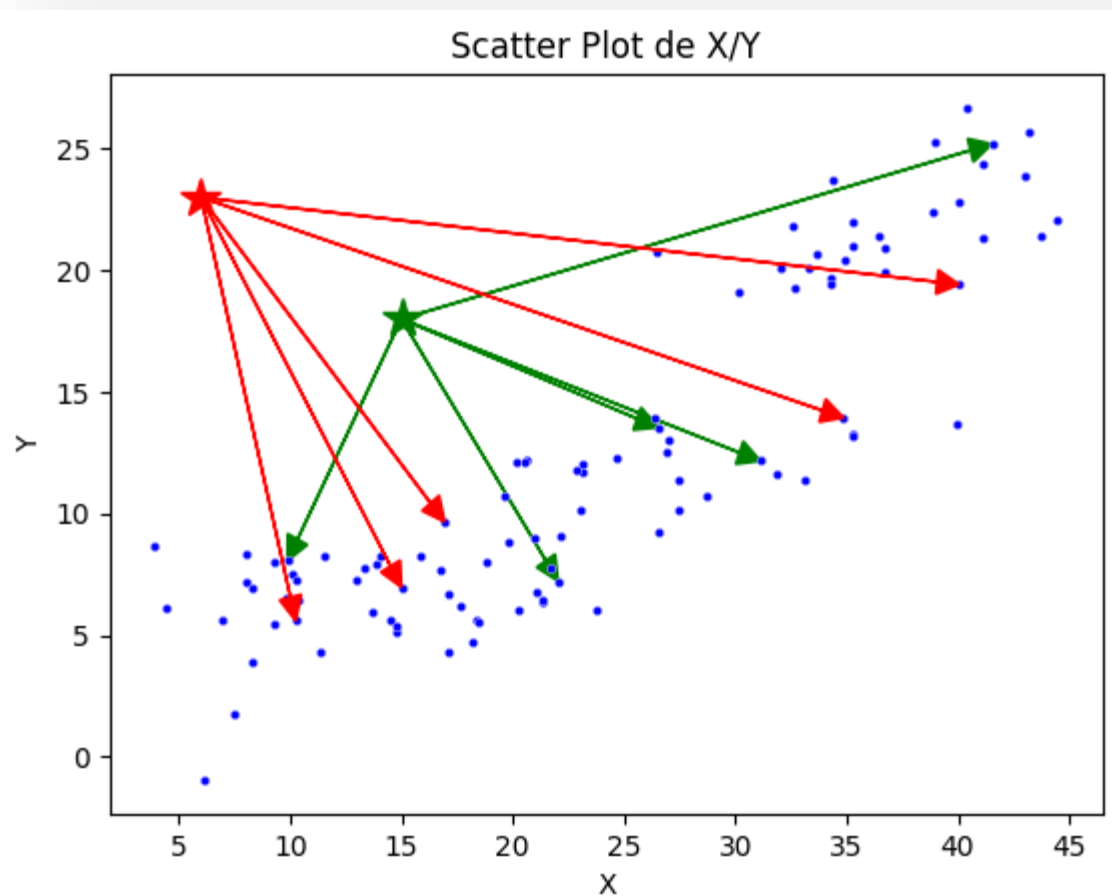
k-means – Método da Silhueta média



Cluster com o maior ponto de Silhueta Média

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

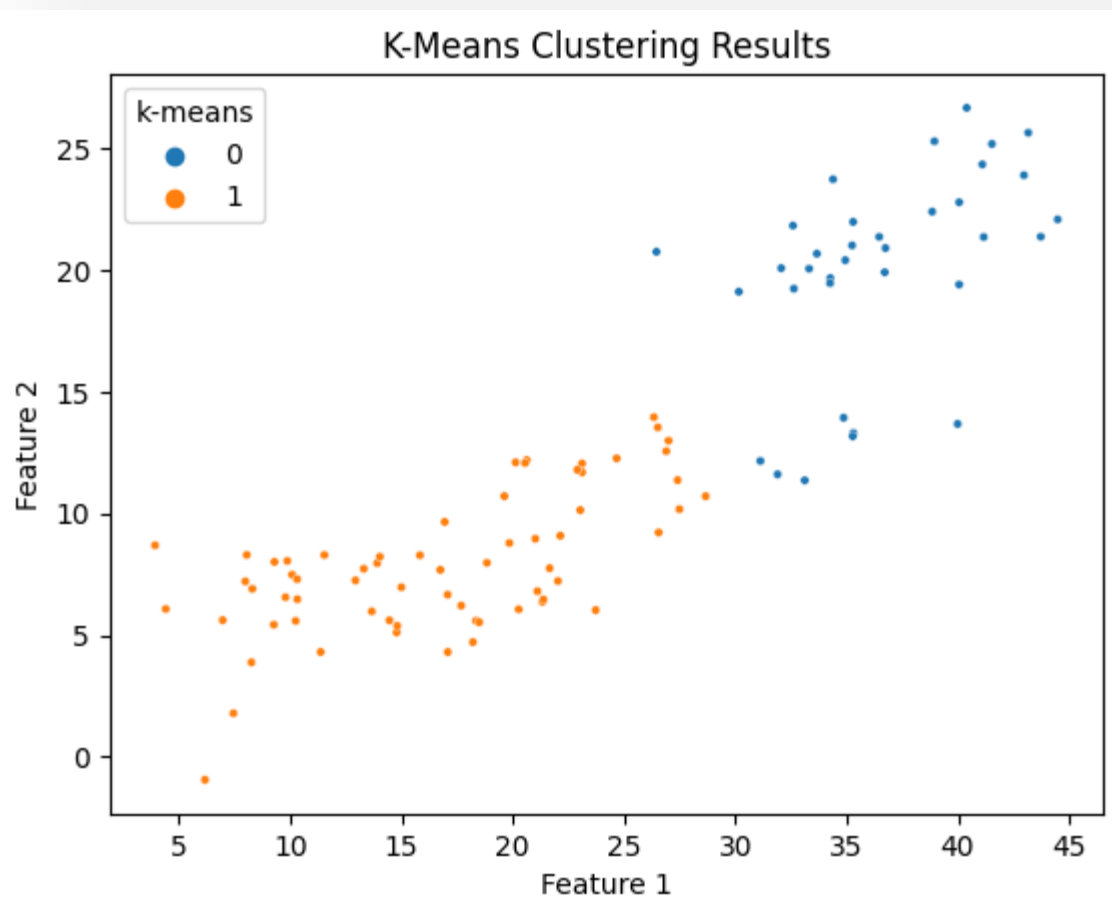
k-means



k = 2

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means



k = 2

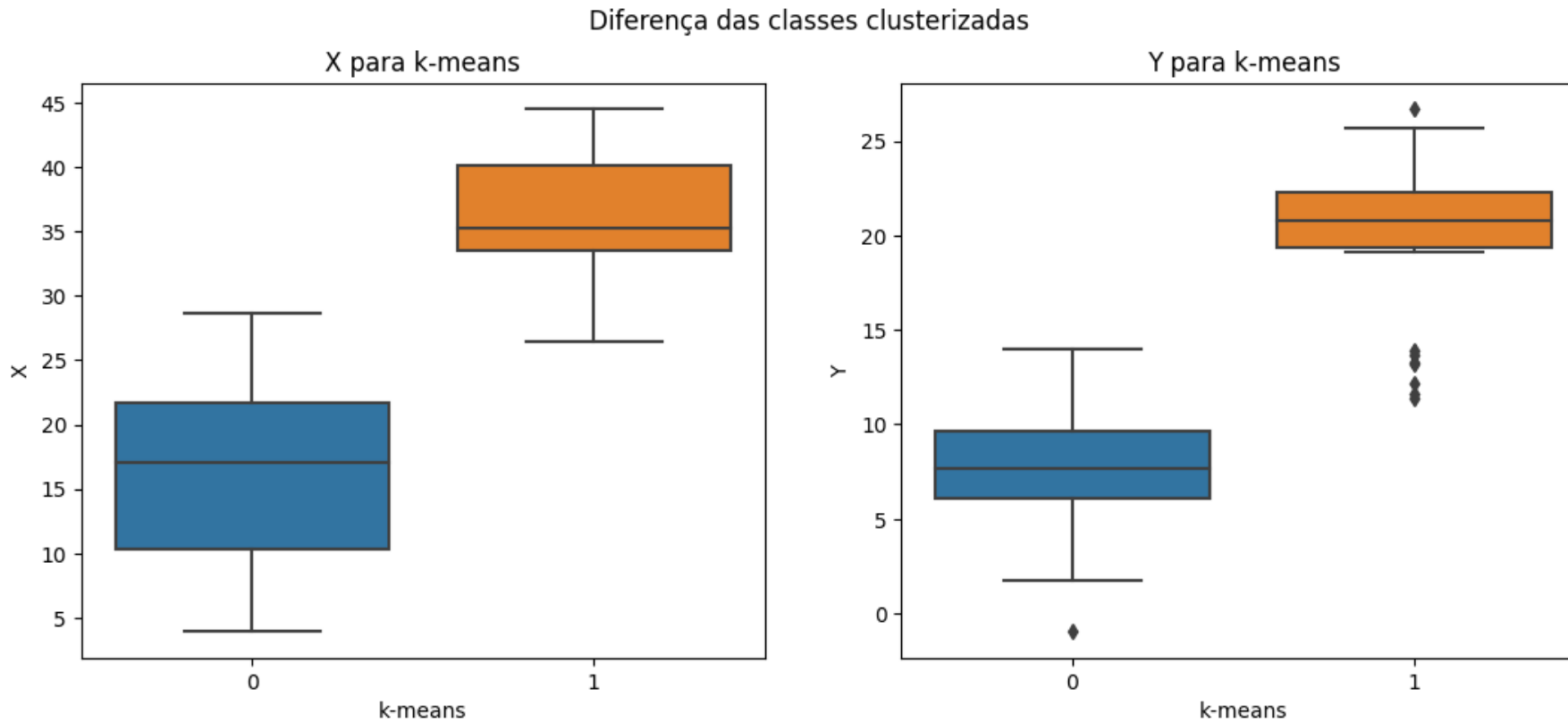
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

k-means

Silhouette Score -> 0.62

Calinski Harabasz Score -> 270.218

Davies Bouldin Score -> 0.503



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

DBSCAN



O DBSCAN é um algoritmo de **agrupamento baseado em densidade** que agrupa pontos compactados, marcando como ruído aqueles em regiões de **baixa densidade**. Ele funciona identificando "**pontos centrais**" (pontos com pelo menos um número mínimo de **vizinhos dentro de um raio especificado**, épsilon) e expandindo clusters a partir deles, encontrando clusters de formato arbitrário e sendo robusto a outliers. Ao contrário de algoritmos como o k-Means, ele não requer um **número predefinido** de clusters.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

DBSCAN

1. O algoritmo começa com um ponto aleatório.
2. Se o ponto for um ponto central, pela medida de distância utilizada, ele é atribuído a um novo cluster.
3. Todos os pontos alcançáveis por densidade a partir deste ponto central são adicionados ao mesmo cluster.
4. Este processo continua até que todos os pontos no cluster sejam encontrados. Se um ponto não for um ponto central, ele é considerado ruído.
5. O algoritmo repete esse processo para quaisquer pontos centrais não visitados até que todos os pontos sejam classificados.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

DBSCAN – Hiperparâmetros

eps

A distância máxima entre duas amostras para que uma seja considerada como estando na vizinhança da outra.

min_samples

O número de amostras (ou peso total) em uma vizinhança para que um ponto seja considerado um ponto central.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

DBSCAN – hiperparâmetros

```
1 # Create a DBSCAN instance with specified parameters
2 min_samples = [1, 2, 3, 4, 5, 6]
3 eps = [0.1, 0.2, 0.25, 0.3, 0.4, 0.5]
4
5 for sample in min_samples:
6     for ep in eps:
7         db = DBSCAN(eps=ep, min_samples=sample)
8         # Fit the model to the data and predict cluster labels
9         clusters = db.fit_predict(df_1[[x_scaled, y_scaled]])
10        cluster, num = np.unique(clusters, return_counts=True)
11        if len(cluster) == 2 or len(cluster) == 3:
12            print("-----"*20)
13            print(f"DBSCAN with eps={ep} and min_samples={sample}")
14            print(cluster, num)
```

✓ 0.1s

DBSCAN with eps=0.5 and min_samples=1
[0 1] [28 72]

DBSCAN with eps=0.4 and min_samples=2
[-1 0 1] [3 27 70]

DBSCAN with eps=0.5 and min_samples=2
[0 1] [28 72]

DBSCAN with eps=0.4 and min_samples=3
[-1 0 1] [3 27 70]

DBSCAN with eps=0.5 and min_samples=3
[0 1] [28 72]

DBSCAN with eps=0.1 and min_samples=4
[-1 0 1] [92 4 4]

DBSCAN with eps=0.3 and min_samples=4
[-1 0 1] [13 25 62]

DBSCAN with eps=0.4 and min_samples=4
[-1 0 1] [3 27 70]

DBSCAN with eps=0.5 and min_samples=4
[-1 0 1] [1 28 71]

DBSCAN with eps=0.2 and min_samples=5
[-1 0 1] [51 36 13]

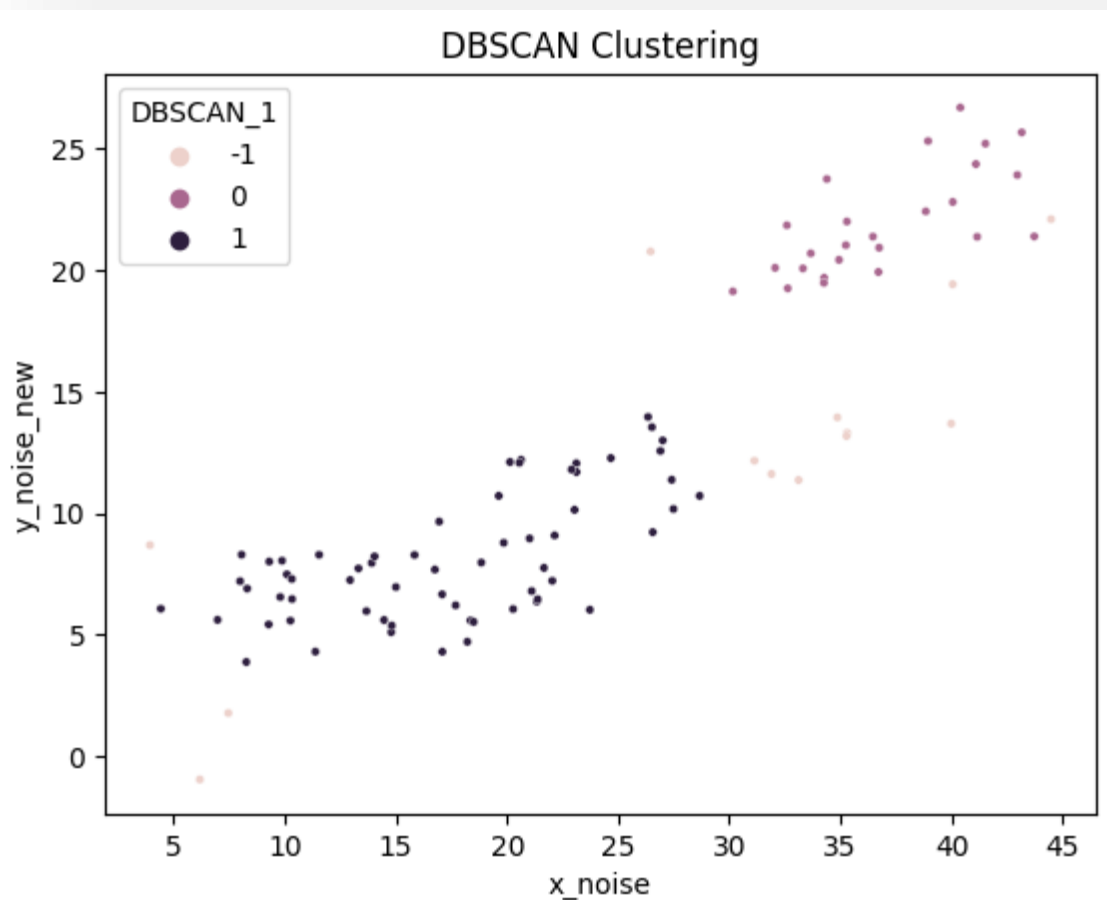
DBSCAN with eps=0.4 and min_samples=5
[-1 0 1] [3 27 70]

DBSCAN with eps=0.5 and min_samples=5
[-1 0 1] [1 28 71]

DBSCAN with eps=0.4 and min_samples=6
[-1 0 1] [4 27 69]

DBSCAN with eps=0.5 and min_samples=6
[-1 0 1] [1 28 71]

DBSCAN



eps=0.3

min_samples=4

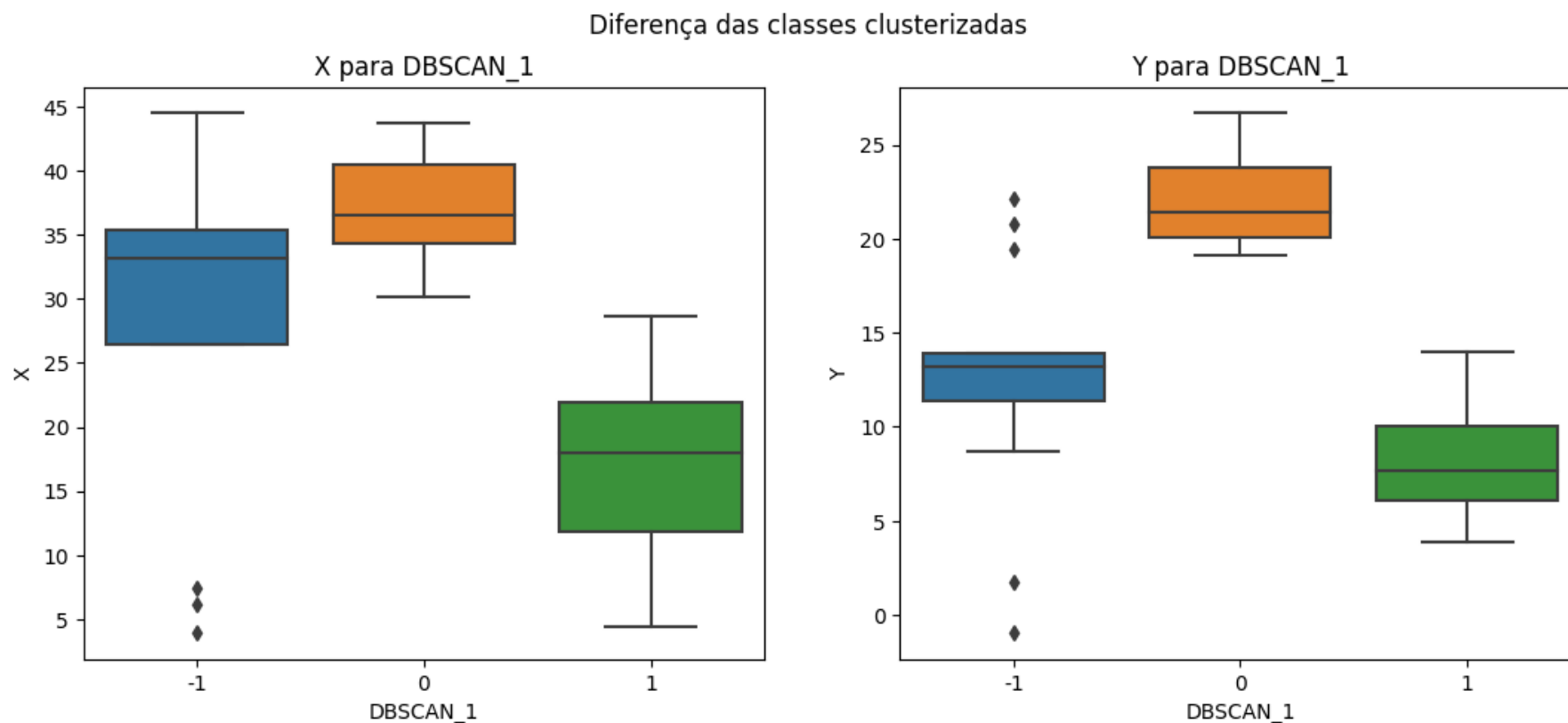
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

DBSCAN

Silhouette Score -> 0.47

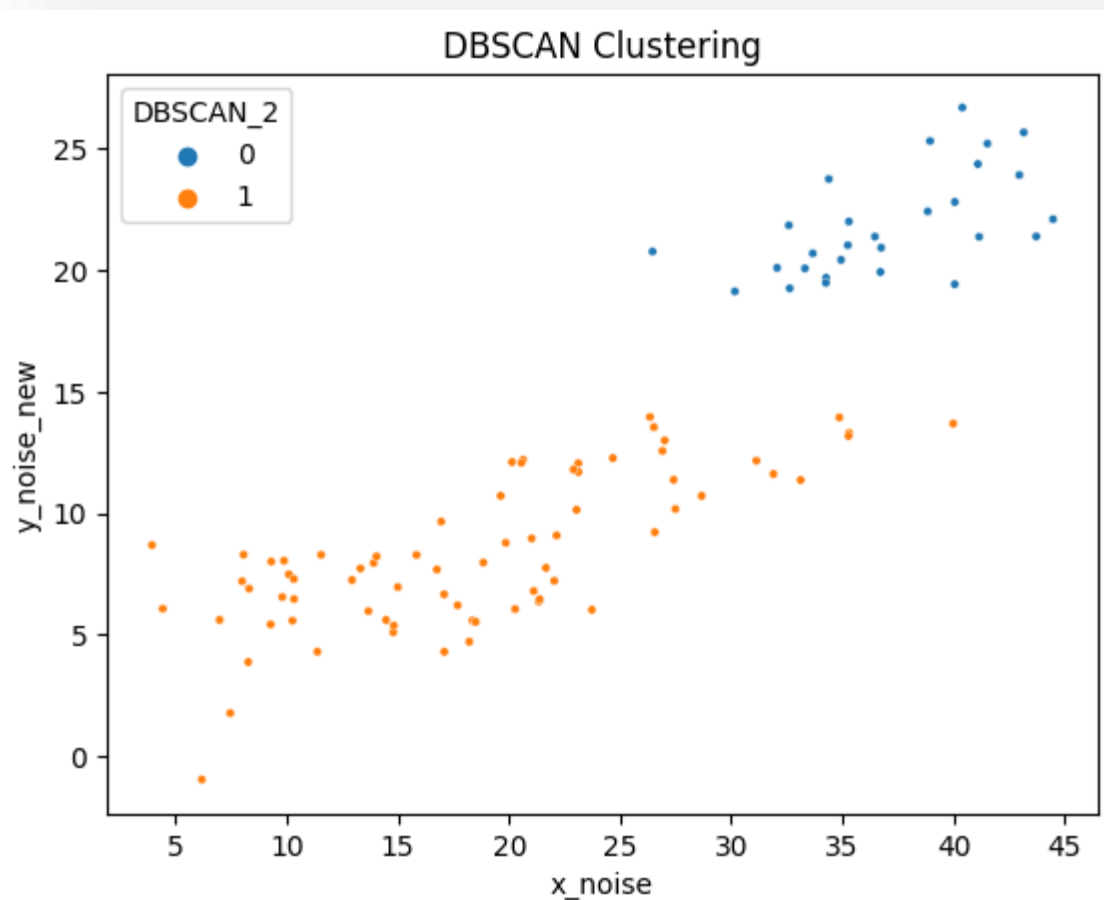
Calinski Harabasz Score -> 101.352

Davies Bouldin Score -> 1.402



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

DBSCAN



eps=0.5

min_samples=3

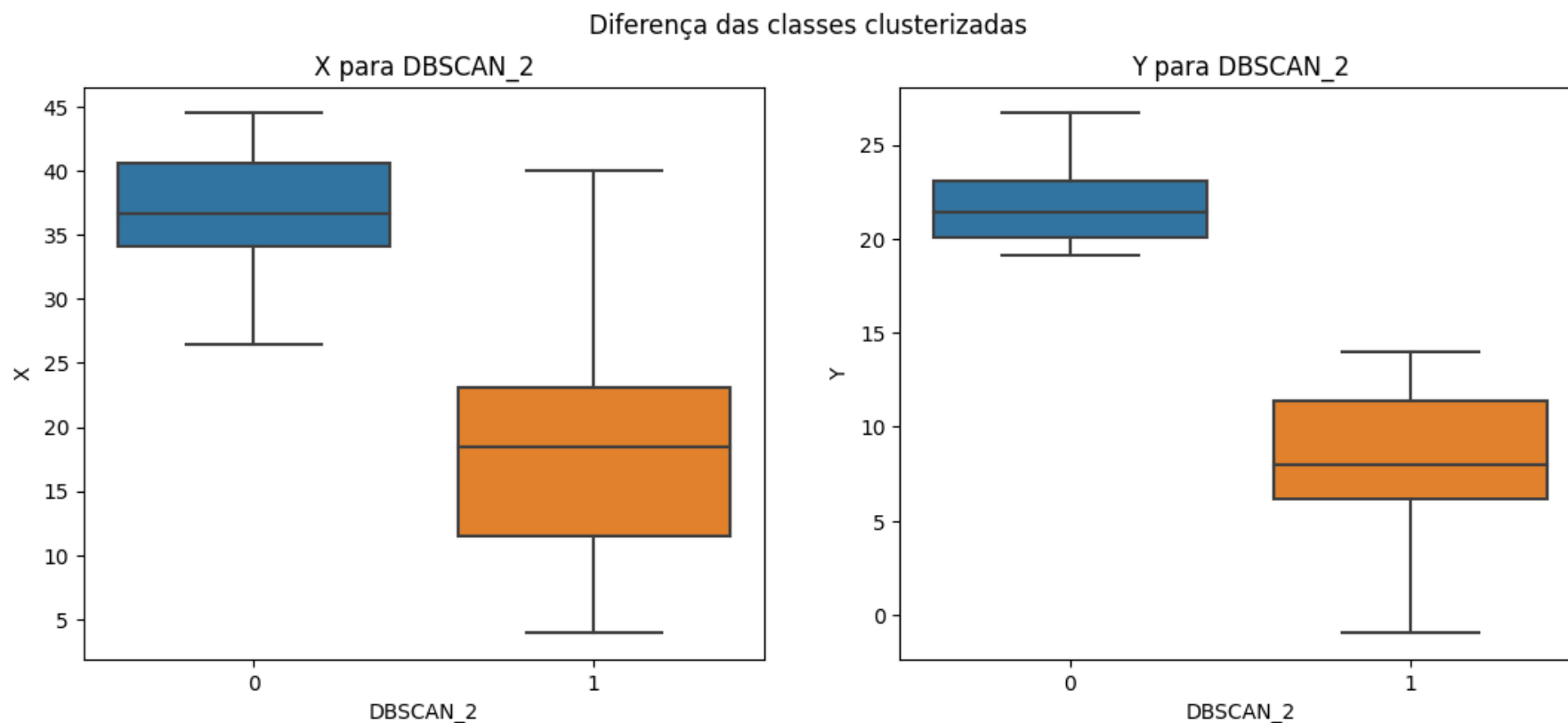
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

DBSCAN

Silhouette Score -> 0.61

Calinski Harabasz Score -> 217.816

Davies Bouldin Score -> 0.466



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Agrupamento Hierárquico



O Agrupamento Hierárquico (ou Hierarchical Grouping) é um algoritmo de aprendizado não supervisionado que organiza dados em uma estrutura em forma de árvore, chamada Dendrograma.

Ao contrário do K-Means, você não precisa definir o número exato de grupos (clusters) de antemão. O algoritmo cria toda a hierarquia de agrupamentos, e você pode "cortar" a árvore na altura que achar melhor.

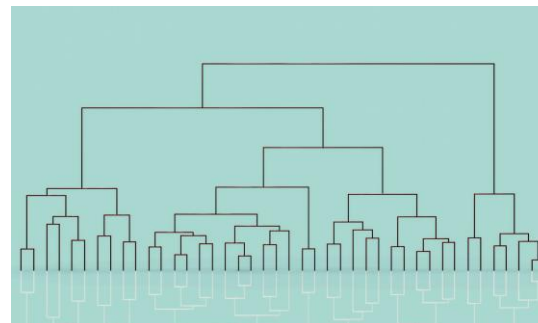
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Agrupamento Hierárquico



Como funciona (Abordagem Aglomerativa) Existem duas abordagens principais, mas a Aglomerativa (de baixo para cima) é a mais usada:

1. **Início:** Cada ponto começa sendo seu próprio cluster separado.
2. **Cálculo de Distância:** O algoritmo calcula a proximidade entre todos os pontos/clusters (usando métricas como Distância Euclidiana).
3. **Fusão:** Os dois clusters mais próximos são unidos em um único cluster.
4. **Repetição:** O processo se repete até que todos os pontos estejam agrupados em um único grande cluster.



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Agrupamento Hierárquico

Hiperparâmetros

n_clusters e **distance_threshold** (Critério de Parada)

Formas opostas de definir onde o algoritmo deve parar de fundir os pontos.

n_clusters (padrão=2): Define exatamente quantos grupos você quer no final.

distance_threshold (float, padrão=None): Em vez de fixar a quantidade de grupos, você define a distância máxima na qual dois clusters podem ser unidos (equivale a cortar o dendrograma em uma altura Y fixa)

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Agrupamento Hierárquico

Hiperparâmetros

linkage (Método de Ligação)

Define a regra matemática para calcular a distância entre dois grupos (e não apenas entre dois pontos). É o parâmetro que mais altera o formato dos grupos resultantes:

- 'ward' (padrão): minimiza a variância total;
- 'complete': usa a distância máxima entre um ponto do Grupo A e um do Grupo B;
- 'average': Usa a distância média entre todos os pontos do Grupo A e do Grupo B;
- 'single': Usa a distância mínima entre dois pontos dos dois grupos;

$$\begin{aligned}f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\&= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\&= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\&= \lim_{h \rightarrow 0} h(2x + h)\end{aligned}$$

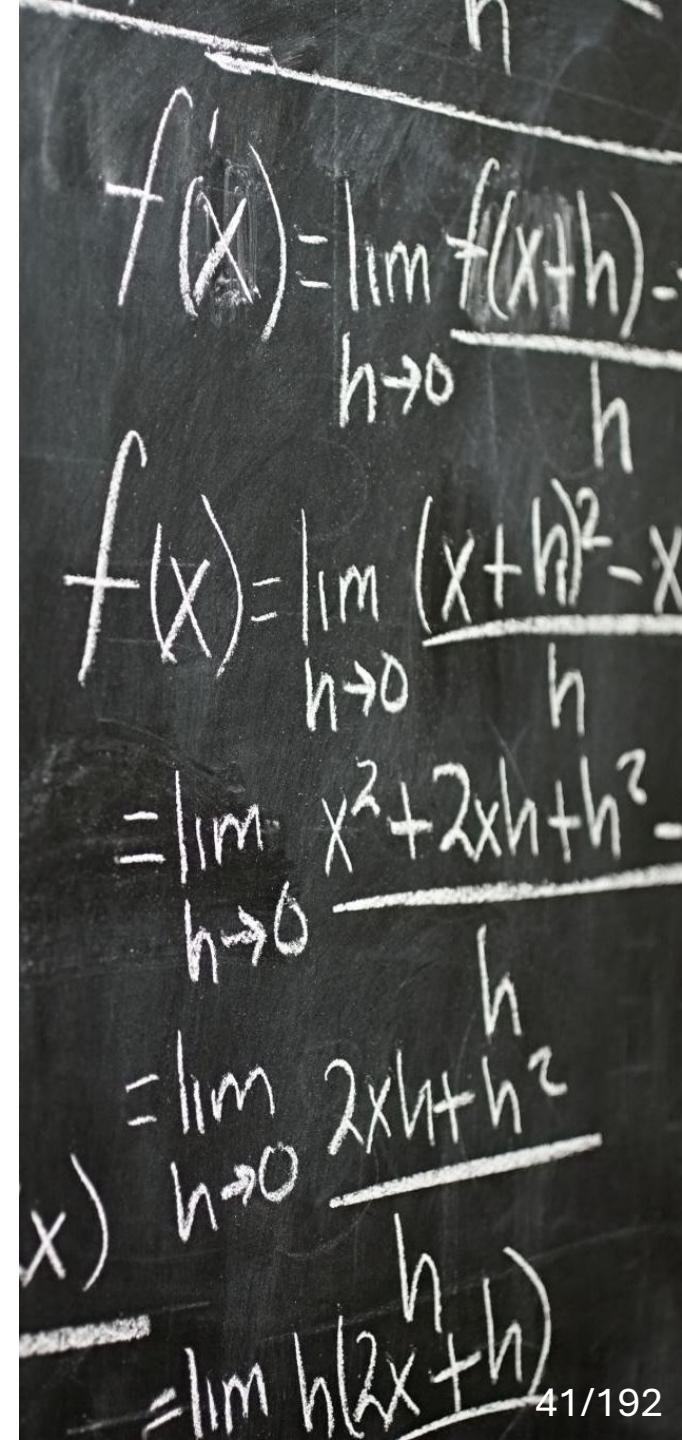
Agrupamento Hierárquico

Hiperparâmetros

metric (Métrica de Distância)

Define como o algoritmo mede o espaço geométrico entre os pontos individuais:

- 'euclidean' (padrão): Distância em linha reta. É a mais comum para dados numéricos contínuos.
- 'manhattan' / 'l1': Soma das diferenças absolutas (útil quando as variáveis têm unidades muito diferentes ou em alta dimensão).
- 'cosine': Mede o ângulo entre os vetores de dados. Muito útil para análise de texto (NLP) ou dados de contagem/frequência, pois ignora a escala e foca no padrão do comportamento.



Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Agrupamento Hierárquico

Hiperparâmetros

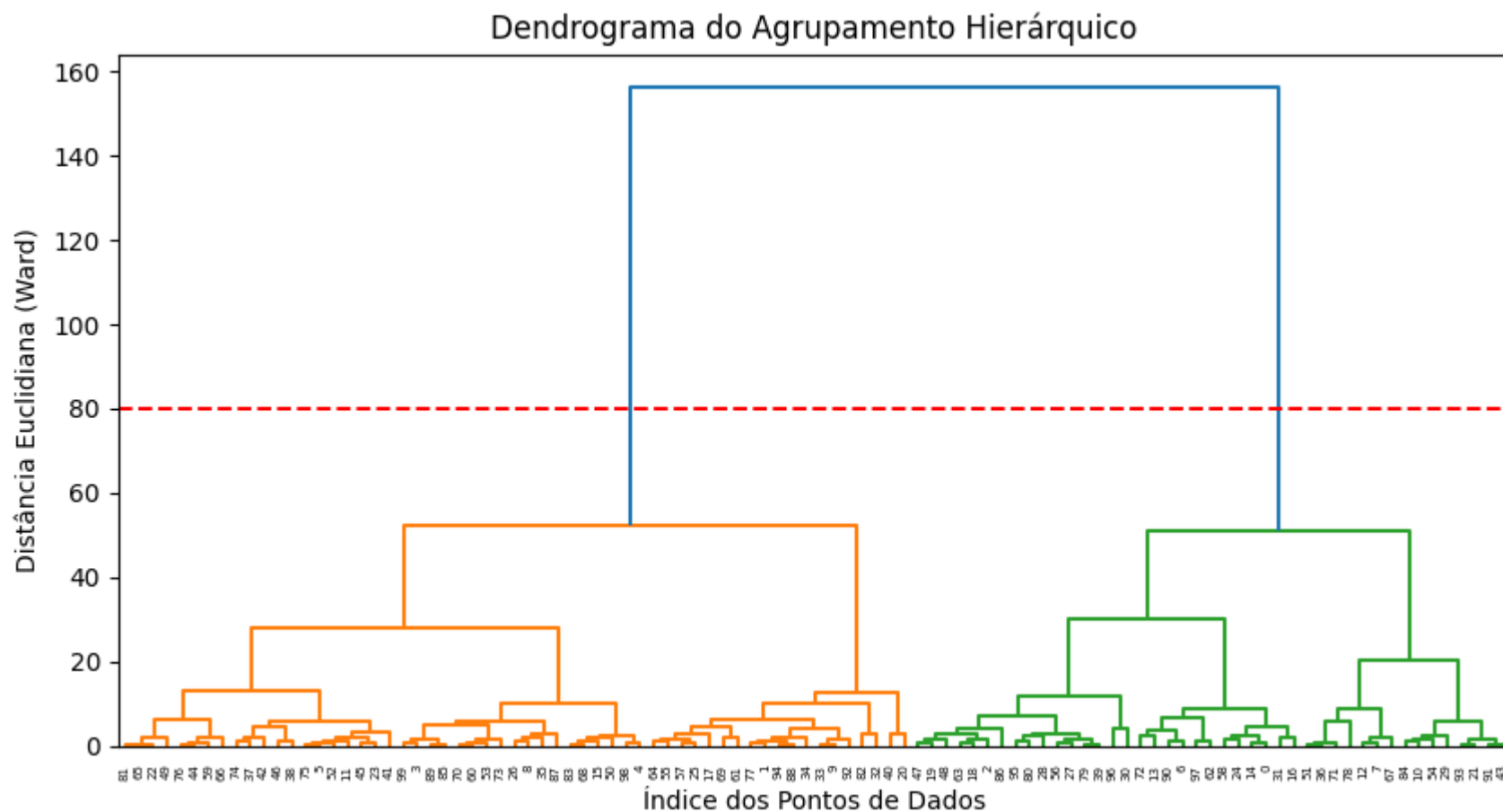
```
1 # 4. Aplicar o algoritmo com o número de clusters desejado
2 # Com base no dendrograma, definimos n_clusters=3
3
4 matriz_conectividade = kneighbors_graph(df_1[[x,y]], n_neighbors=5, include_self=False)
5 modelo = AgglomerativeClustering(
6     n_clusters=None,           # Desativado para usar distance_threshold
7     distance_threshold=distancia, # Corte do dendrograma na altura 70
8     metric="euclidean",        # Métrica de distância em linha reta
9     linkage="ward",            # Minimiza a variância total dentro de cada cluster
10    connectivity=matriz_conectividade, # Apensar funde pontos se forem vizinhos
11 )
12 rotulos = modelo.fit_predict(df_1[[x,y]])
13 rotulos[:5]
```

✓ 0.0s

array([1, 0, 1, 0, 0])

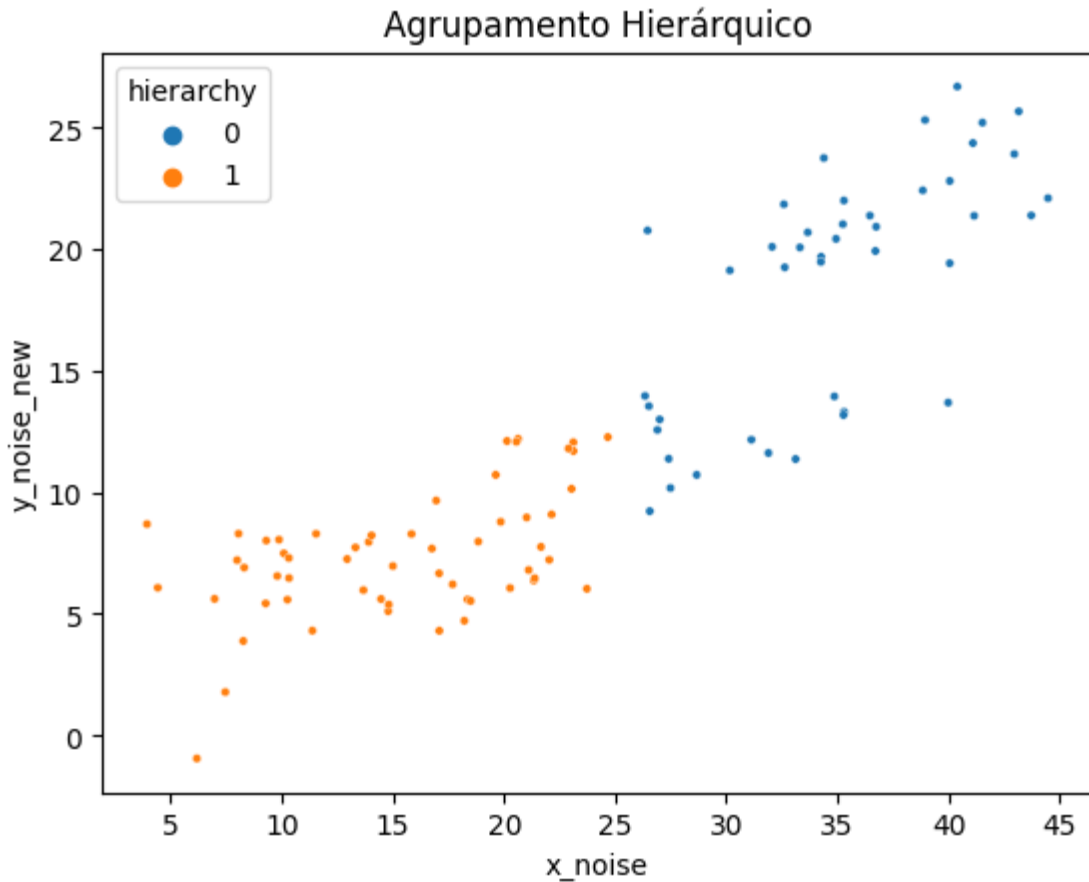
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Agrupamento Hierárquico



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

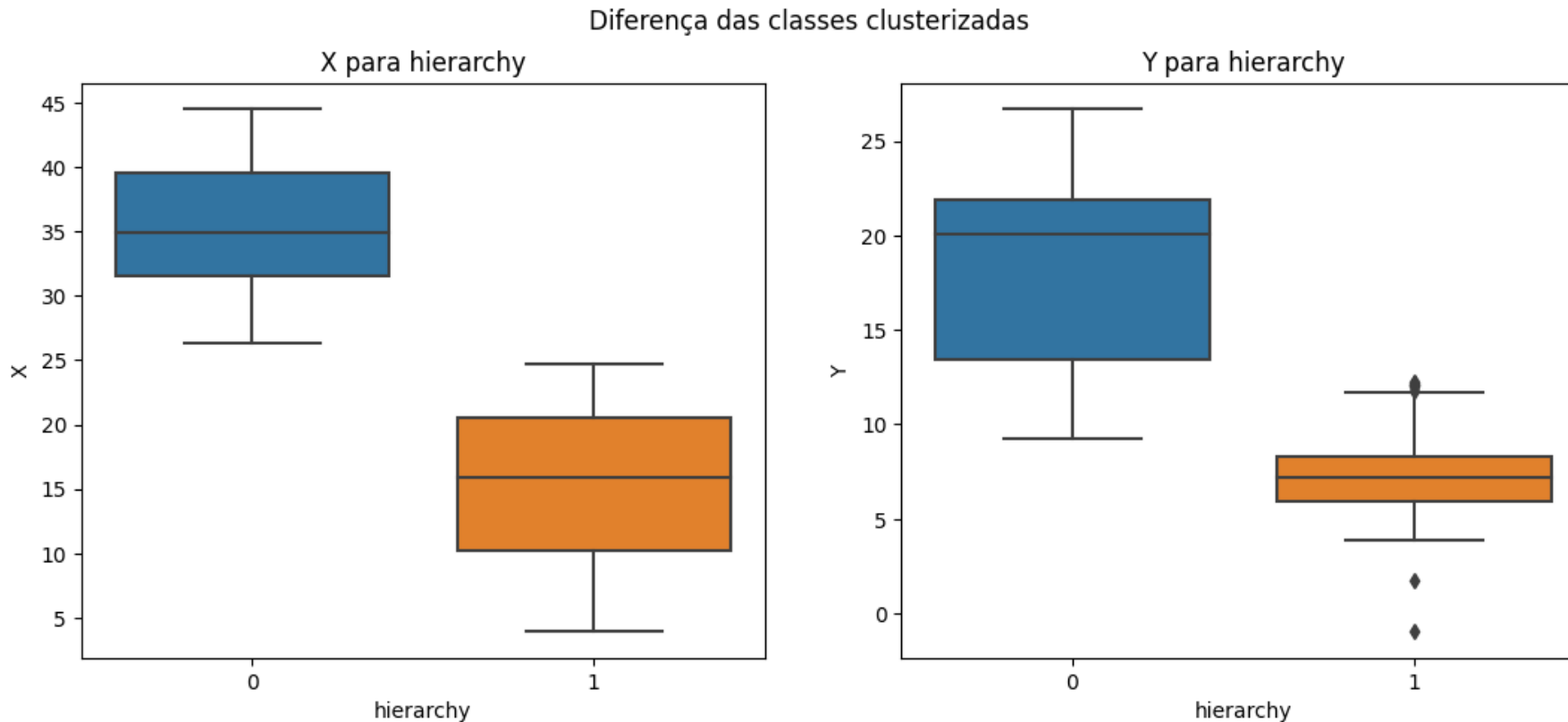
Agrupamento Hierárquico



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Agrupamento Hierárquico

Silhouette Score -> 0.586
Calinski Harabasz Score -> 247.346
Davies Bouldin Score -> 0.565



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Redes Neurais - Autoencoder



Autocodificadores (Autoencoders) são redes neurais treinadas para reconstruir suas entradas. A camada oculta de um autocodificador treinado pode aprender uma representação compactada e de baixa dimensão dos dados que captura características essenciais. Essa representação é então usada para algoritmos de clusterização tradicionais, como o K-Means.

Seria próximo a uma redução de dimensionalidade

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Redes Neurais - Autoencoder

Model: "model_6"

Layer (type)	Output Shape	Param #
input_4 (InputLayer)	[None, 9]	0
dense_6 (Dense)	(None, 2)	20
dense_7 (Dense)	(None, 9)	27

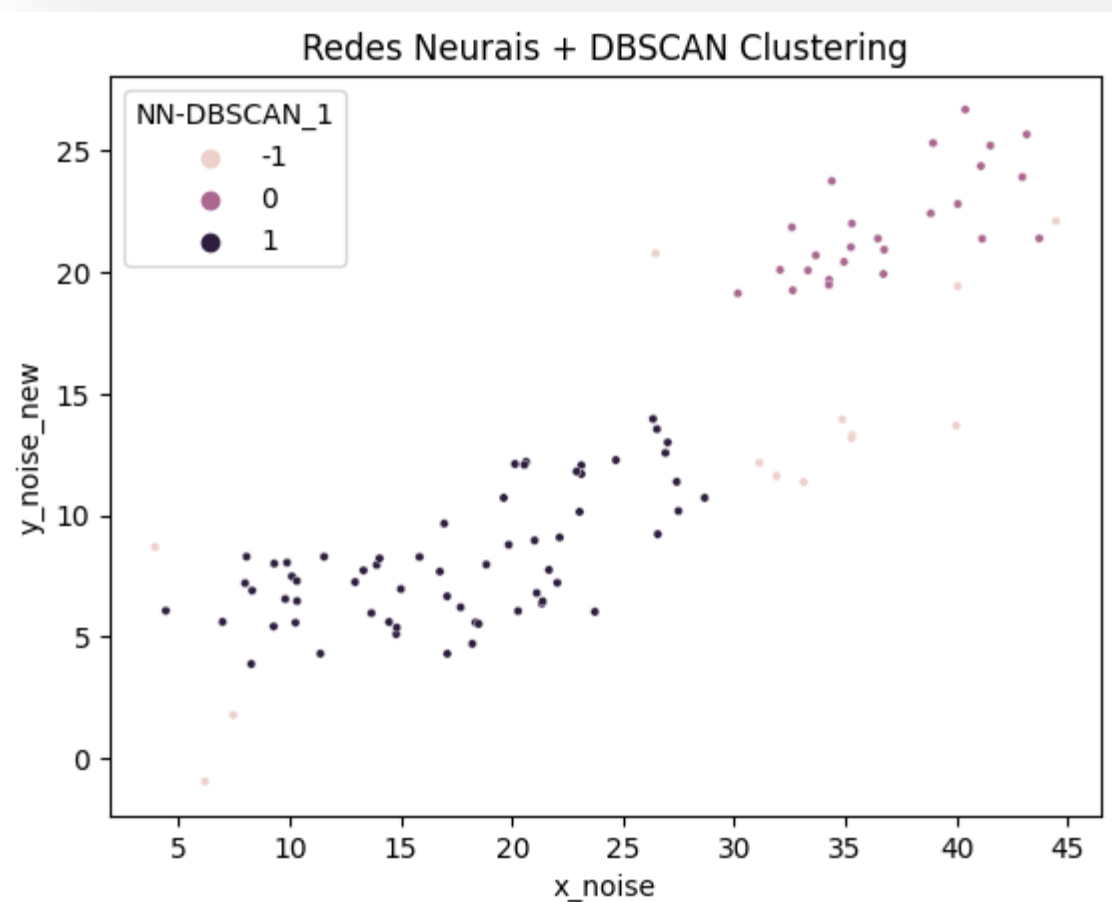
Total params: 47

Trainable params: 47

Non-trainable params: 0

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Redes Neurais

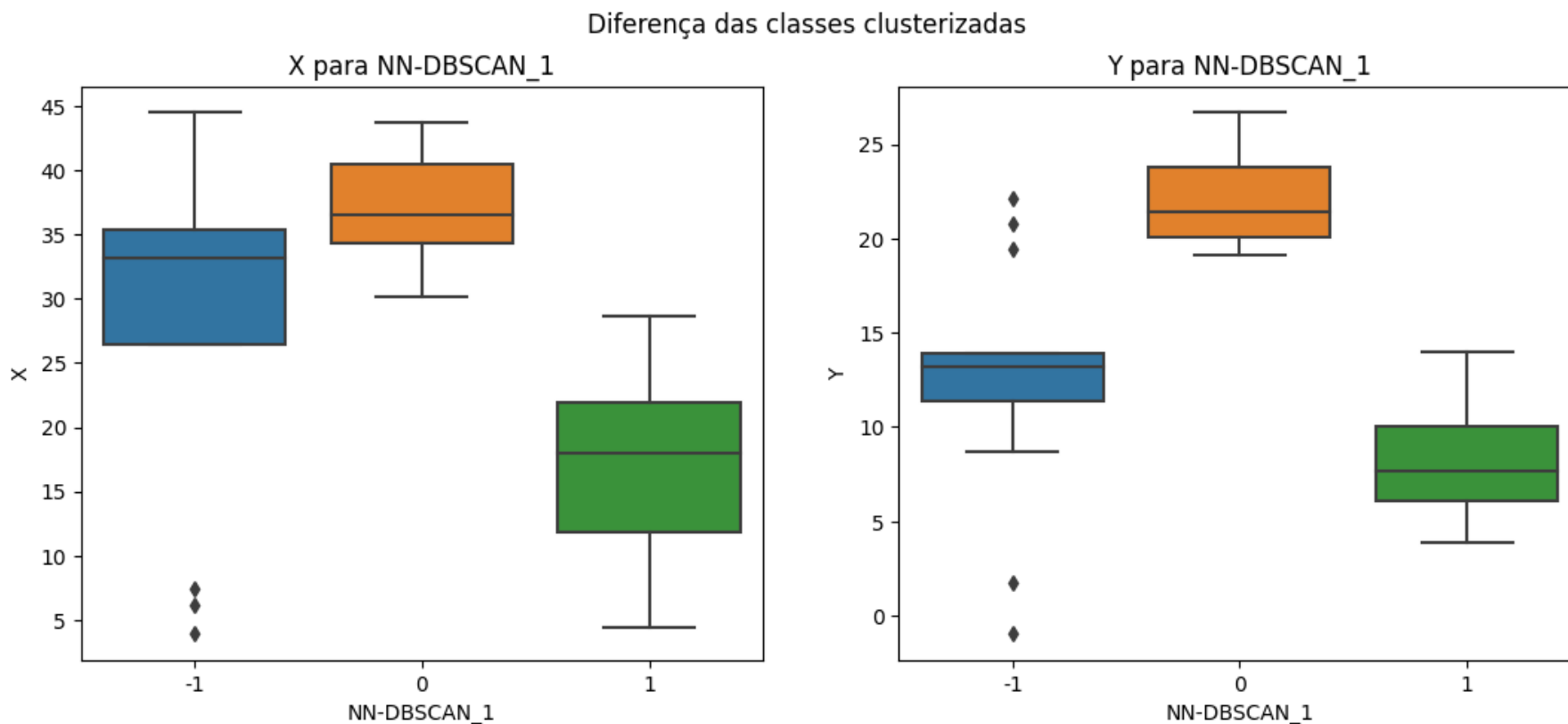


$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Redes Neurais

DBSCAN

Silhouette Score -> 0.47
Calinski Harabasz Score -> 101.352
Davies Bouldin Score -> 1.402



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Métricas - Clusterização

Métrica	Descrição	Interpretação
Calinski Harabasz	Extrínseco	Quanto menor melhor
Davies Bouldin	Extrínseco	Quanto maior melhor
Silhouette	Intrínseco	1 = Agrupamento perfeito 0 = Aleatório -1 = Pior que aleatório
ARI	Extrínseco	1 = Agrupamento perfeito 0 = Aleatório -1 = Pior que aleatório
AMI	Extrínseco	1 = Agrupamento perfeito 0 = Aleatório -1 = Pior que aleatório

Métricas



Modelo	Silhouette	Calinski Harabasz	Davies Bouldin	ARI ($k=3$)	AMI ($k=3$)
K-means	0,62	270,218	0,503	0,813	0,739
DBSCAN ($k=3$)	0,47	101,352	1,402	1,0	1,0
DBSCAN ($k=2$)	0,61	217,816	0,466	0,712	0,692
NN-DBSCAN	0,47	101,352	1,402	1,0	1,0
Hierarchy	0,58	247,346	0,565	0,534	0,464

Métricas



Modelo	Silhouette	Calinski Harabasz	Davies Bouldin	ARI (k=3)	AMI (k=3)
K-means	0,62	270,218	0,503	0,813	0,739
DBSCAN (k=3)	0,47	101,352	1,402	1,0	1,0
DBSCAN (k=2)	0,61	217,816	0,466	0,712	0,692
NN-DBSCAN	0,47	101,352	1,402	1,0	1,0
Hierarchy	0,58	247,346	0,565	0,534	0,464

Métricas



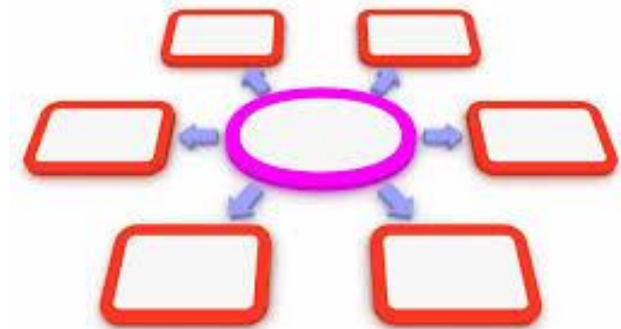
Modelo	Silhouette	Calinski Harabasz	Davies Bouldin	ARI (k=2)	AMI (k=2)
K-means	0,62	270,218	0,503	1,0	1,0
DBSCAN (k=3)	0,47	101,352	1,402	0,813	0,739
DBSCAN (k=2)	0,61	217,816	0,466	0,733	0,671
NN-DBSCAN	0,47	101,352	1,402	0,813	0,739
Hierarchy	0,58	247,346	0,565	0,702	0,66



Vamos
praticar?

LIMC-IA

Classificação



Classificação

K-Nearest Neighbors;

Logistic Regression;

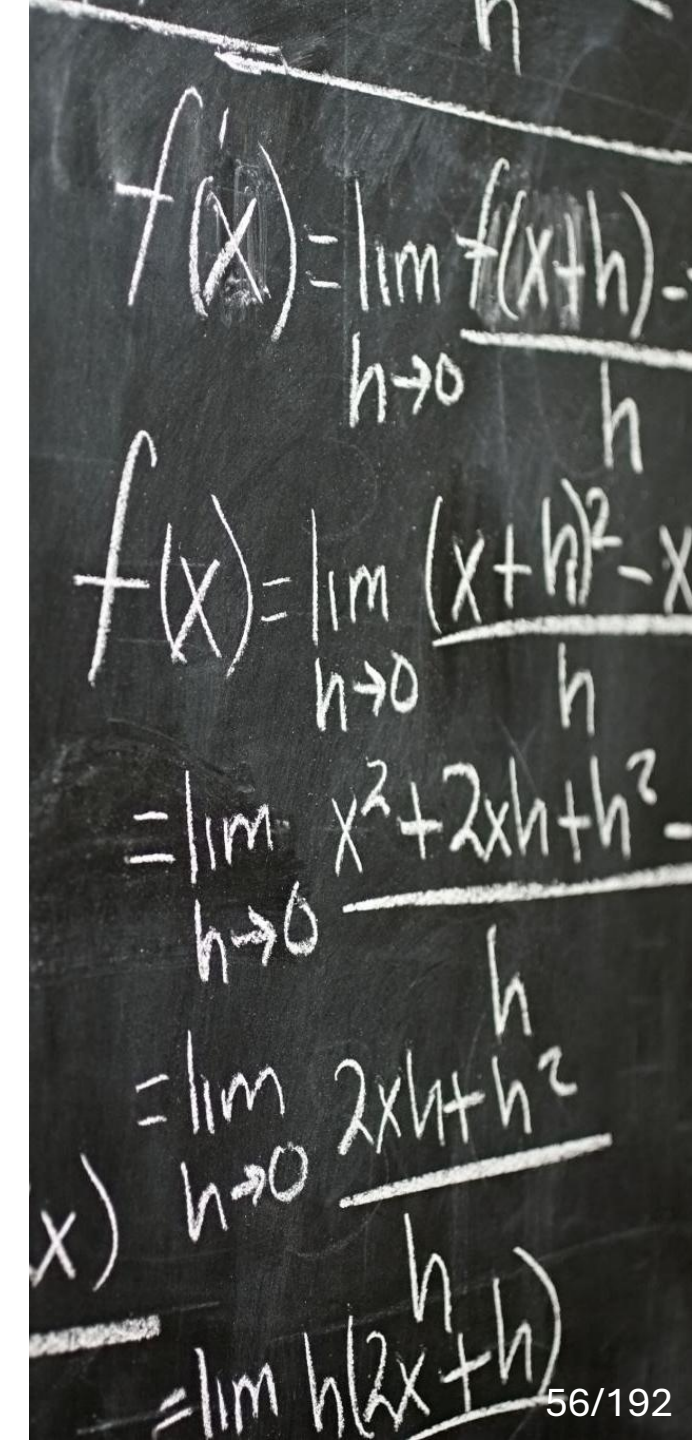
SVM;

Decision Trees;

Random Forests;

Naive Bayes;

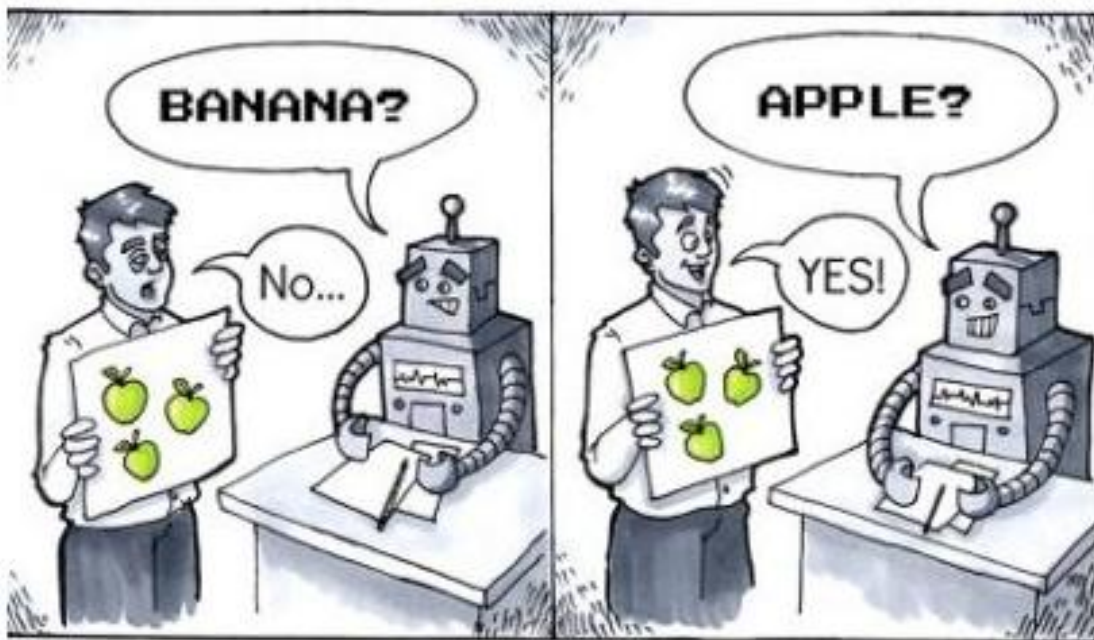
Neural Networks;



Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Supervisionada

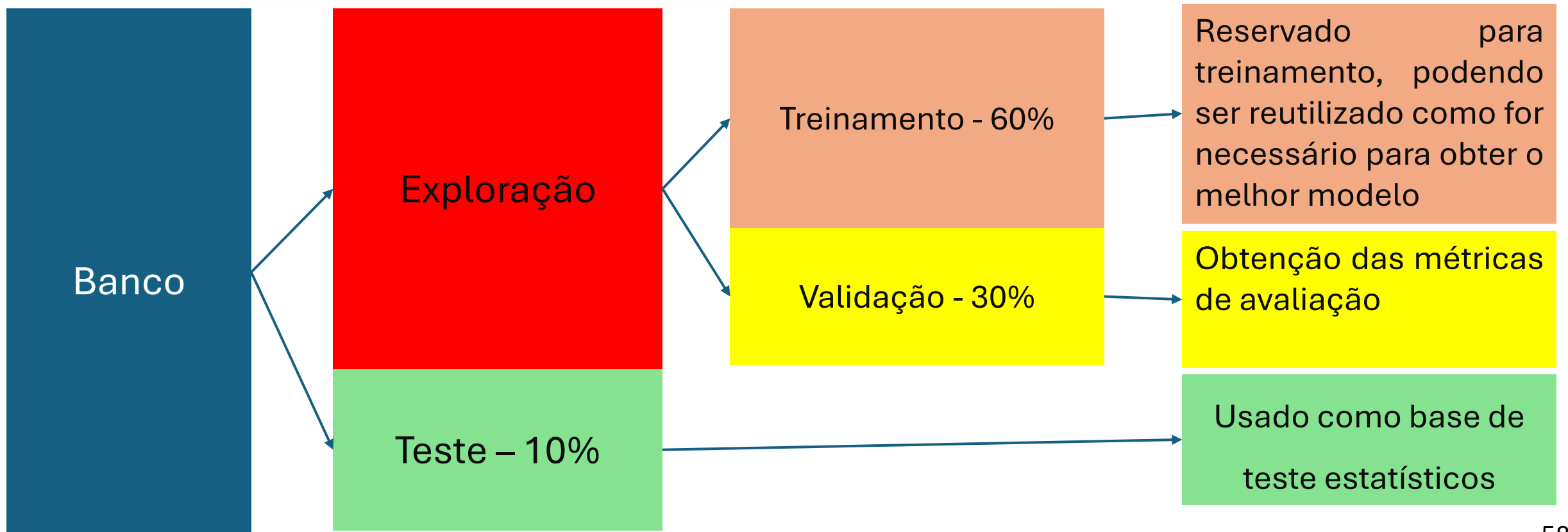


Supervised Learning

Temos um alvo definido que é usado para treinamento

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Divisão do banco de dados



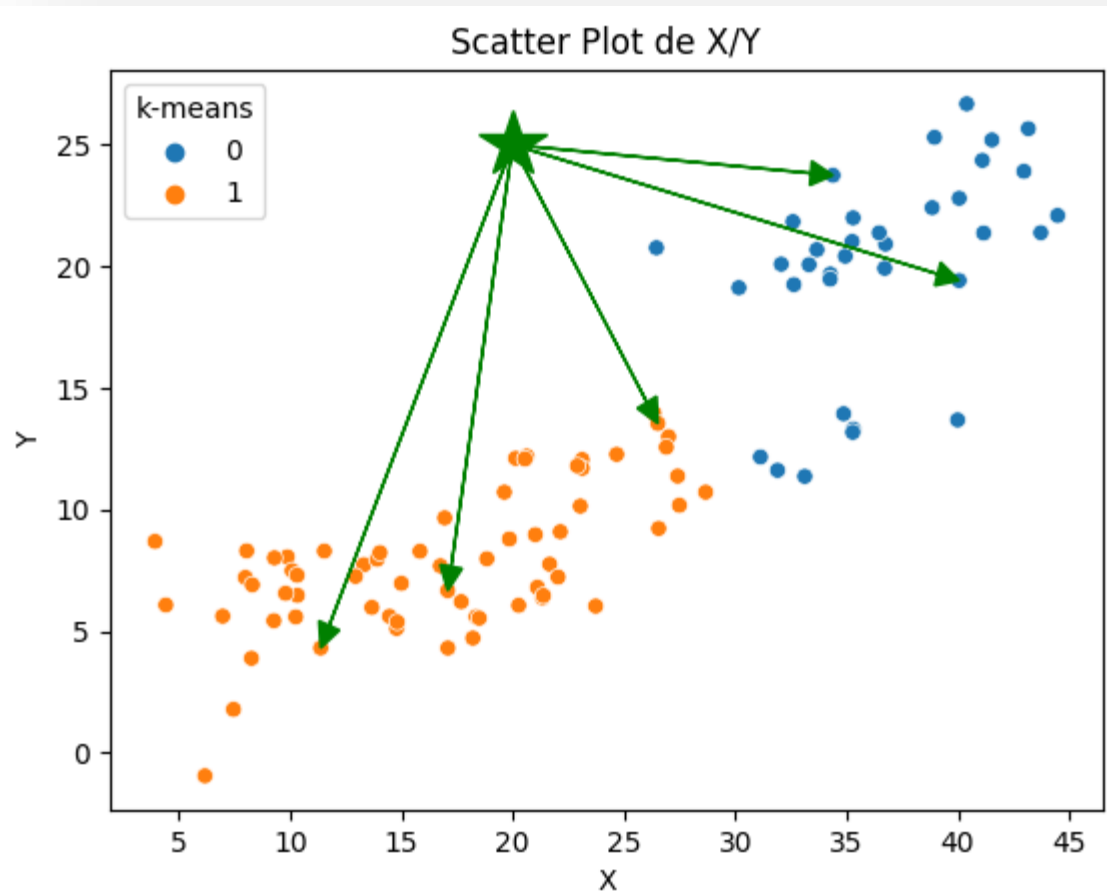
K-Nearest Neighbors



Um classificador K-NN (K-Nearest Neighbors) é um algoritmo de aprendizado de máquina simples e não paramétrico que classifica um novo ponto de dados com base na classe **majoritária de seus "k" vizinhos** mais próximos nos dados de treinamento. Ele funciona calculando a distância entre **o novo ponto e todos os pontos existentes**, identificando os "k" vizinhos mais próximos e, em seguida, atribuindo a classe que aparece com mais frequência entre eles. É um algoritmo versátil e amplamente utilizado para tarefas como detecção de caligrafia e diagnóstico médico.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

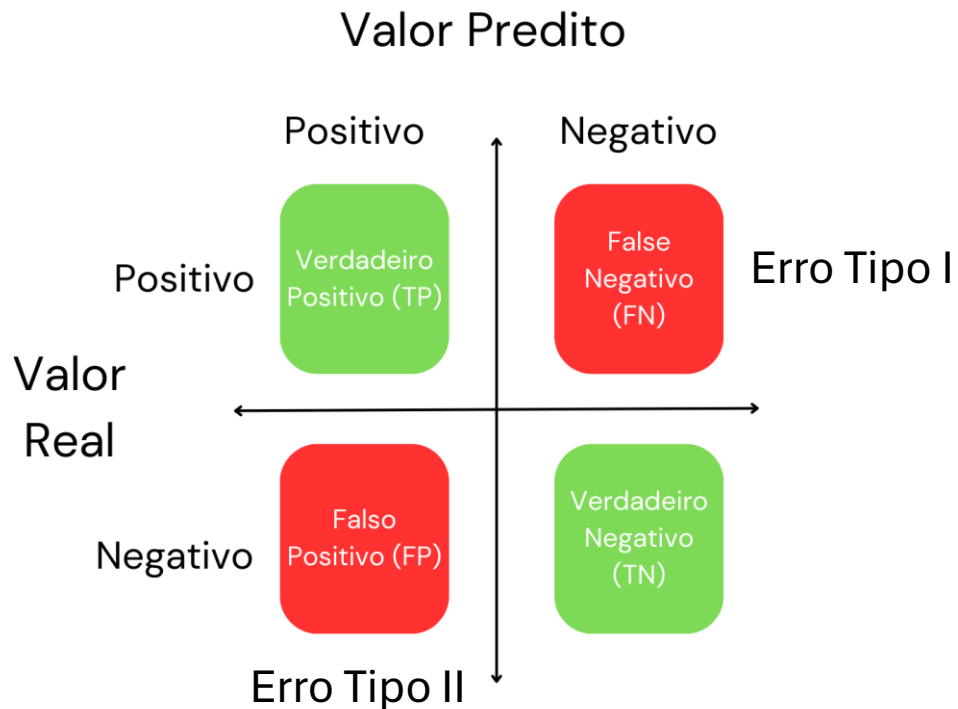
K-Nearest Neighbors



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Métricas – Matriz de Confusão

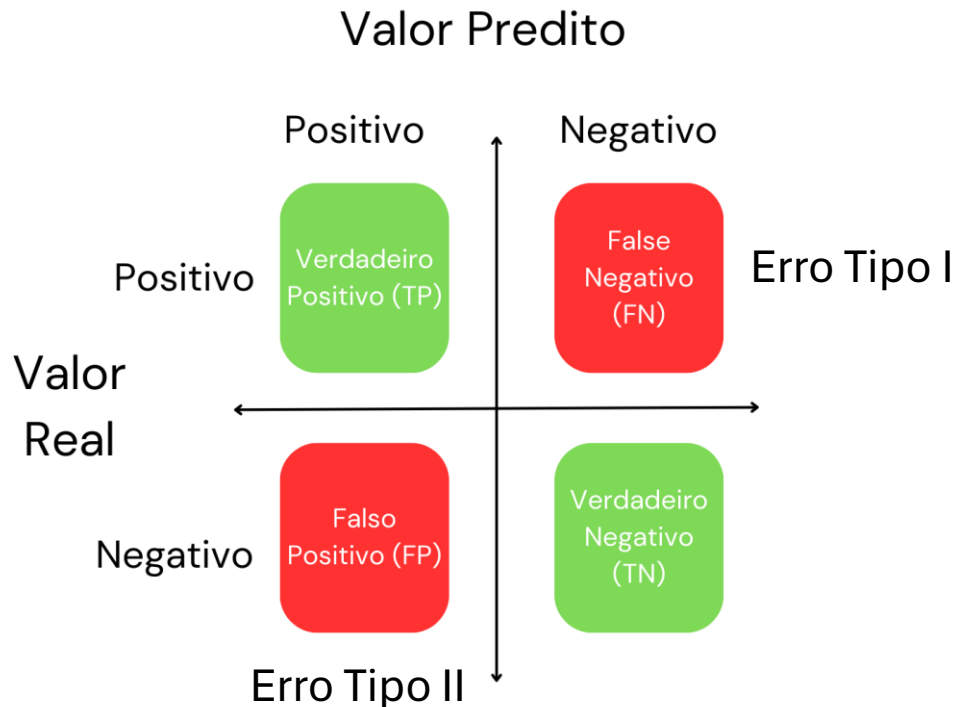
Classificação



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Métricas – Matriz de Confusão

Classificação



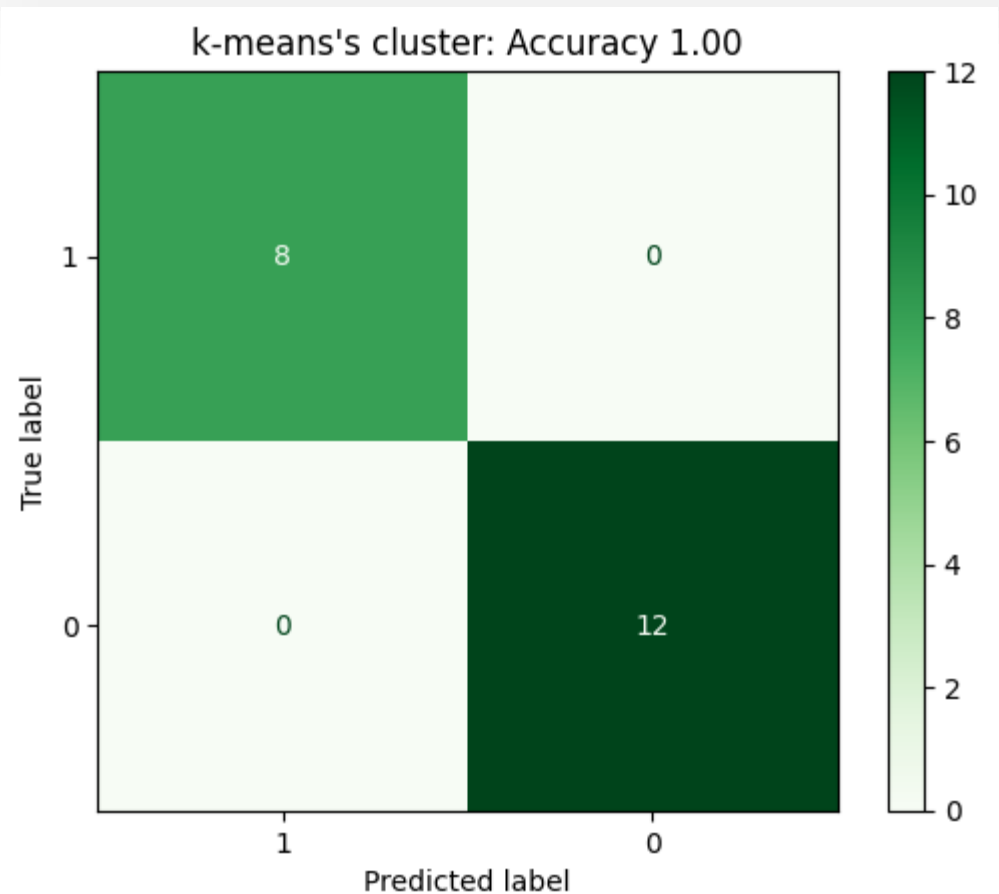
- **Acurácia:** $\frac{VP+VN}{VP+VN+FP+FN}$
- **Precisão:** $\frac{VP}{VP+FP}$
- **Recall (Sensibilidade):** $\frac{VP}{VP+FN}$
- **F1-Score:** $2 \times \frac{\text{Precisão} \times \text{Recall}}{\text{Precisão} + \text{Recall}}$

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Métricas - Classificação

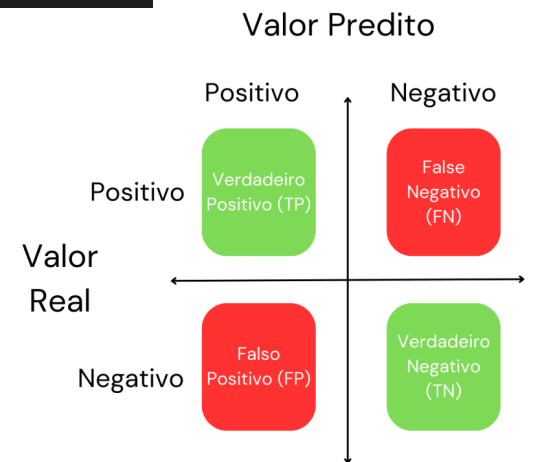
Métrica	Descrição	Interpretação
Acurácia	É a quantidade de acertos do nosso modelo dividido pelo total da amostra.	Qual a proporção de gatos e não gatos que foram corretamente classificados.
Precisão	De todos os dados classificados como positivos, quantos são realmente positivos.	Qual a proporção dos dados classificados como gatos eram realmente gatos. (aqui buscamos os falsos positivos)
Recall	Qual a porcentagem de dados classificados como positivos comparado com a quantidade real de positivos que existem em nossa amostra.	Entre todas as amostras que realmente eram de gatos, qual a proporção classificada como gatos. (aqui buscamos os falsos negativos)
F1-Score	Essa métrica une precisão e <i>recall</i> afim de trazer um número único que determine a qualidade geral do nosso modelo.	Uma maneira de observar em um único número a precisão e o recall.

K-Nearest Neighbors *VS* k-means

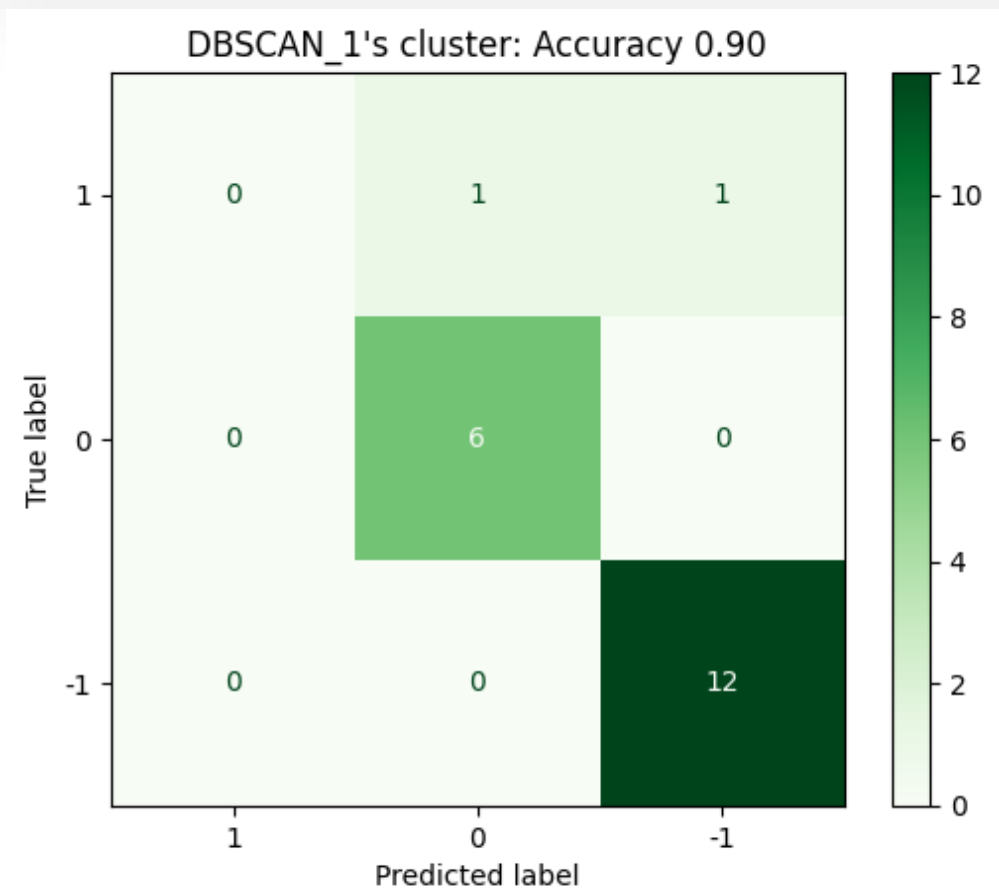


Classification Report:

	precision	recall	f1-score	support
0	0.92	1.00	0.96	12
1	1.00	0.88	0.93	8
accuracy			0.95	20
macro avg	0.96	0.94	0.95	20
weighted avg	0.95	0.95	0.95	20

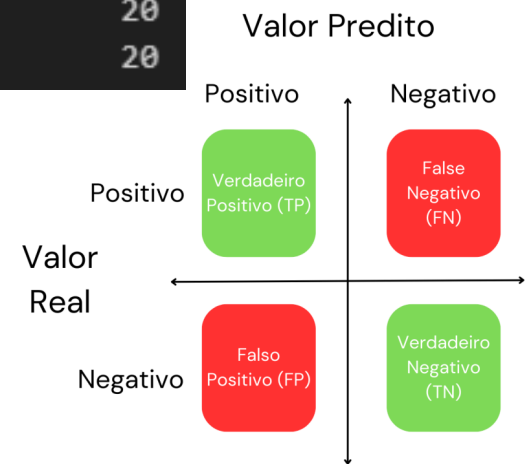


K-Nearest Neighbors *VS* DBSCAN

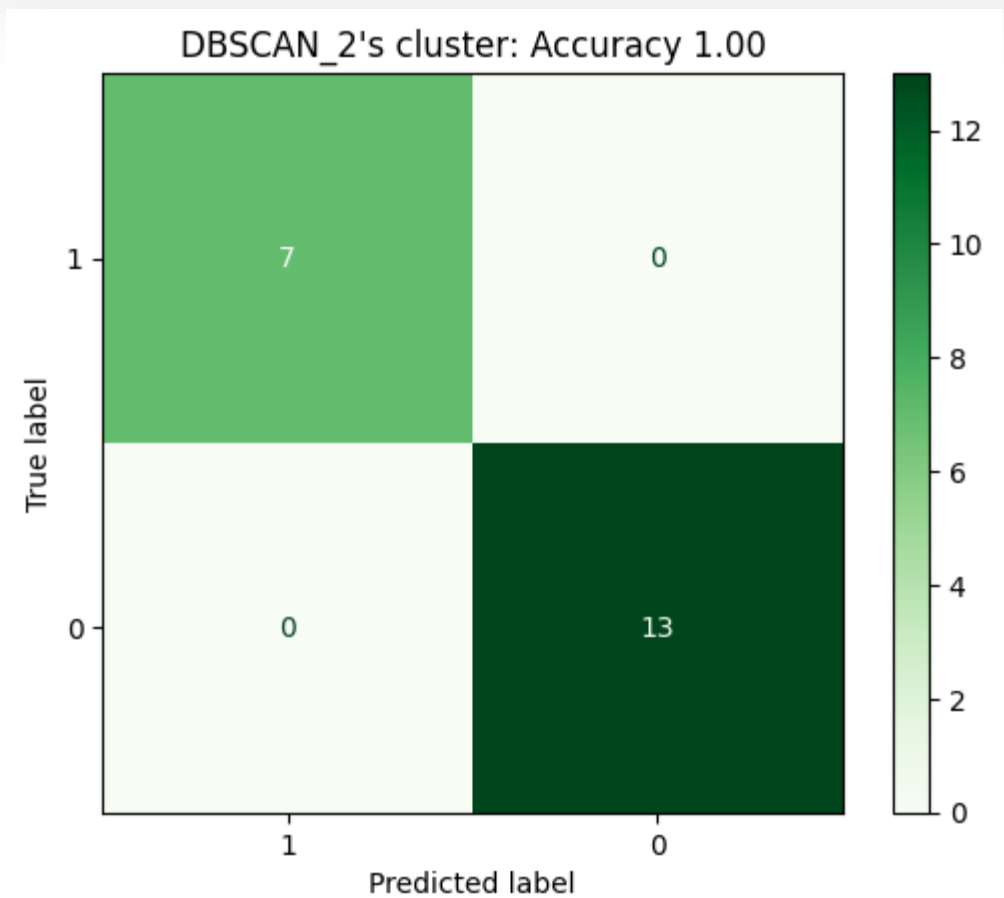


Classification Report:

	precision	recall	f1-score	support
-1	0.00	0.00	0.00	2
0	0.86	1.00	0.92	6
1	0.92	1.00	0.96	12
accuracy			0.90	20
macro avg	0.59	0.67	0.63	20
weighted avg	0.81	0.90	0.85	20

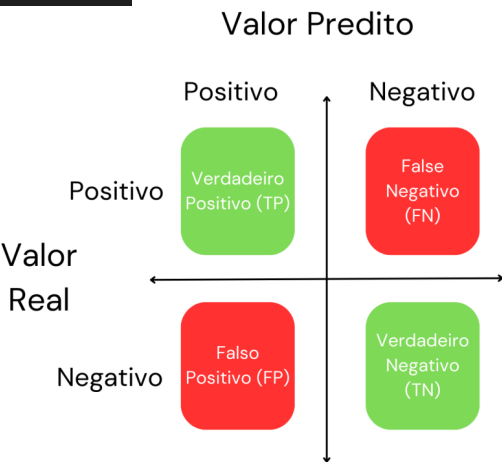


K-Nearest Neighbors *VS* DBSCAN

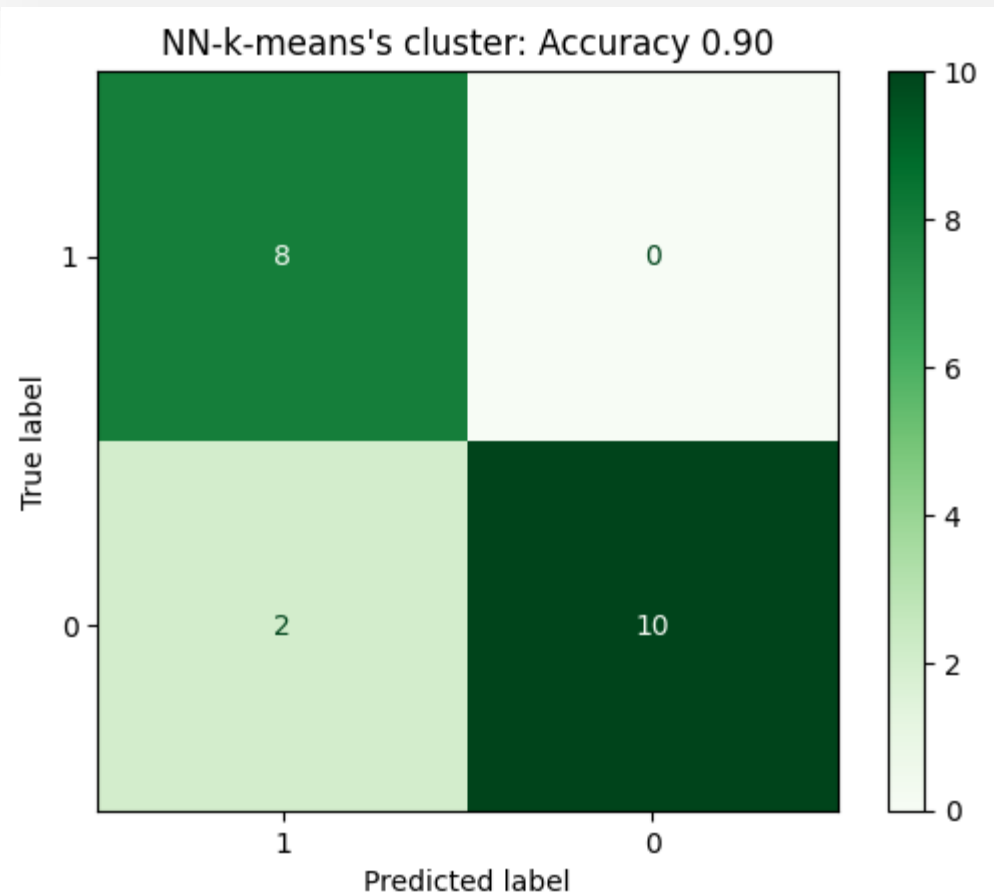


Classification Report:

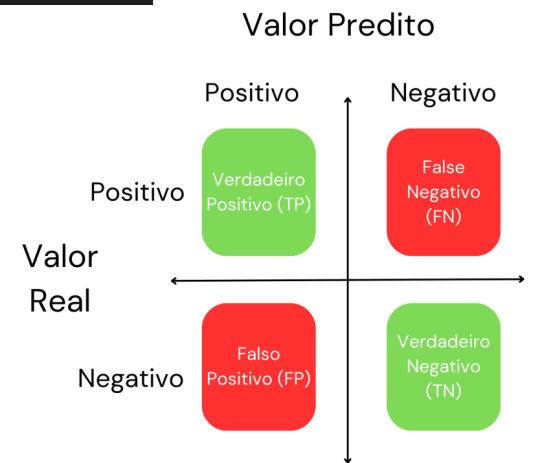
	precision	recall	f1-score	support
0	1.00	1.00	1.00	7
1	1.00	1.00	1.00	13
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20



K-Nearest Neighbors *VS* NN-k-means



Classification Report:				
	precision	recall	f1-score	support
0	0.89	0.89	0.89	9
1	0.91	0.91	0.91	11
accuracy			0.90	20
macro avg	0.90	0.90	0.90	20
weighted avg	0.90	0.90	0.90	20



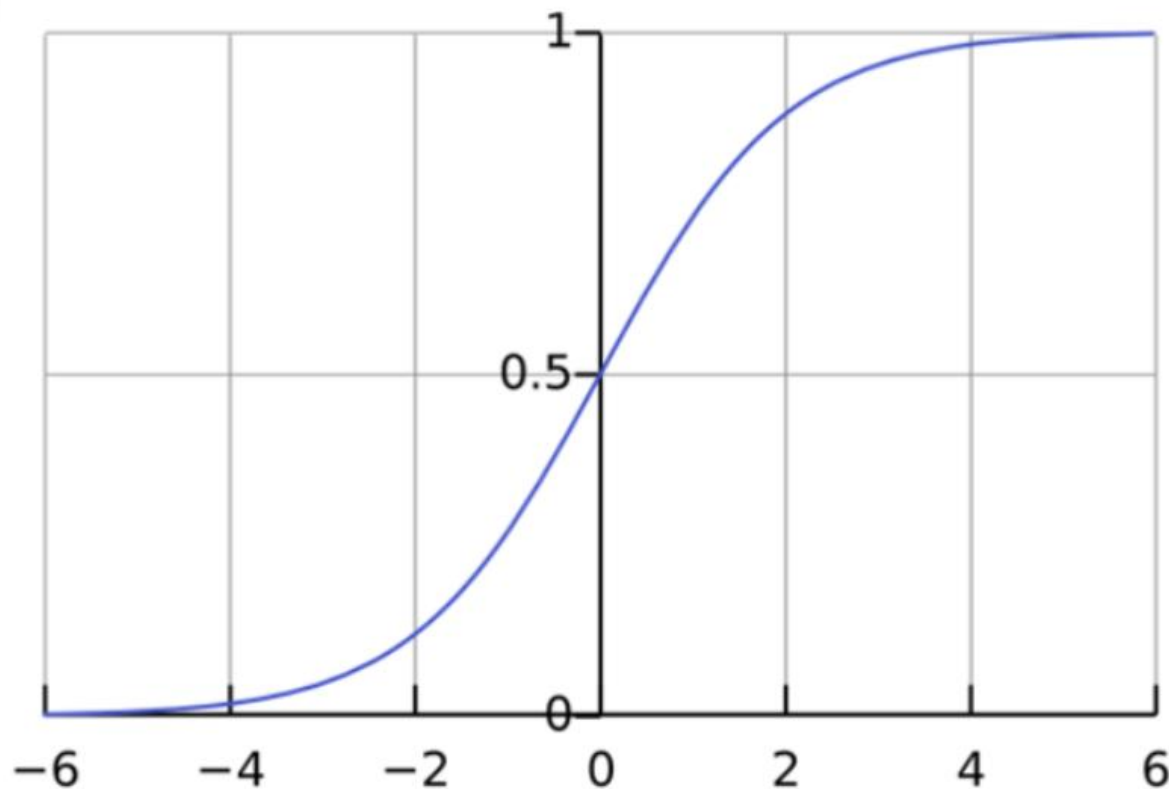
Logistic Regression



A **regressão logística** é um modelo estatístico que prevê a probabilidade de um resultado **categórico** (como "sim" ou "não") com base em uma ou mais variáveis independentes. Ela é usada para classificação e utiliza uma **função logit** (ou sigmoide) para mapear os resultados **entre 0 e 1**, formando uma curva em "S".

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Logistic Regression - Sigmoide



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

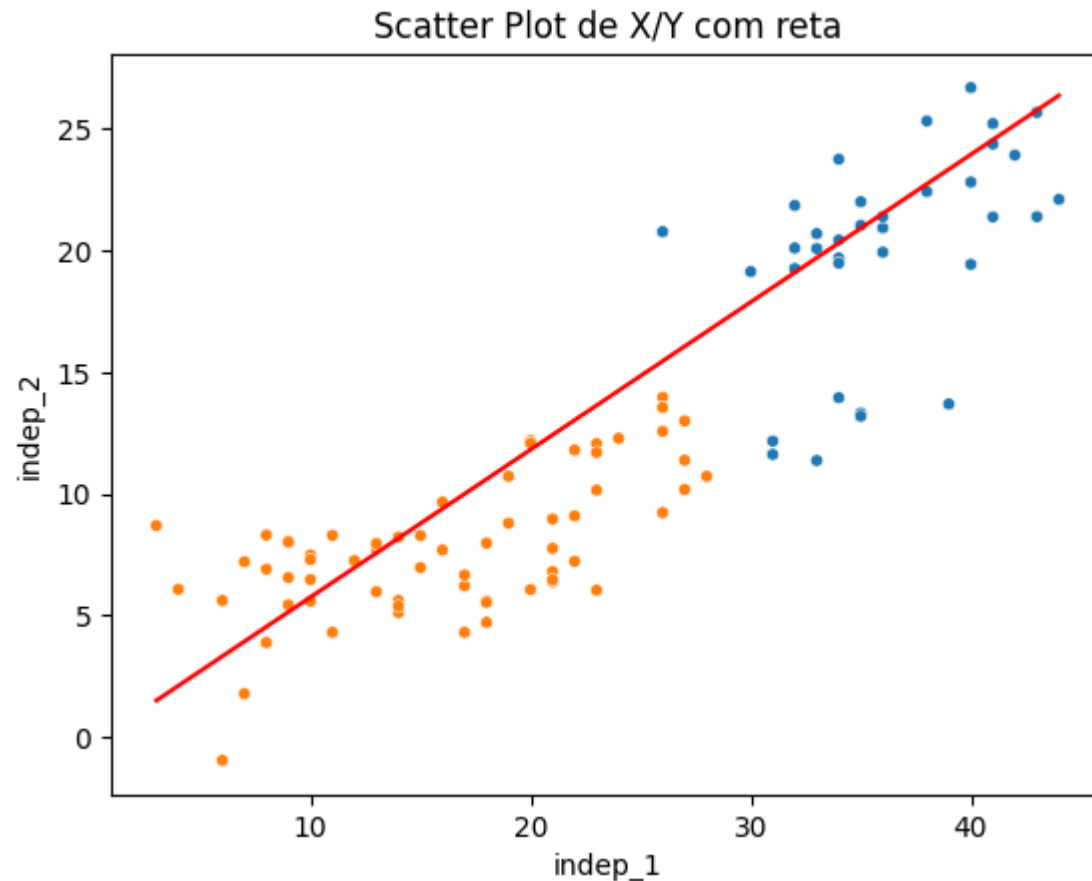
Dados – Categorização (50%)



```
df_4['class_'+y] = pd.qcut(x=df_4[y], q=2, labels=["min", "max"])
```

	indep_1	indep_2	indep_3	indep_4	k_means	DBSCAN_1	DBSCAN_2	NN_k_means	k_NN
53	41	24.36	max	max	0	0	0	0	0
93	9	8.05	min	med	1	1	1	1	1
20	36	19.92	max	max	0	0	0	0	0
177	13	7.73	max	min	1	1	1	1	1
26	17	6.21	max	min	1	1	1	1	1

Dados – Categorização (50%)



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Dados



Calculo de probabilidade

Probabilidade (**prob**):

Ser max ou min no conjunto total de dados

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Dados



Calculo de probabilidade

Risco do Evento (**risck_event**):

De um max ou de um min estar na classe (i.e. k-means, k-NN, etc)

Chance do Evento (**odds**)

De um max ou de um min de estar em uma classe (i.e. k-means, k-NN, etc)

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Dados



Razão da Chance do Risco (**odds_ratio**)

A razão entre as chances de estar na classe para max e min

Logaritmo da Chance do Chance (**log_ratio**)

Logaritmo neperiano (Log na base 2,71826) da chance

Peso da Evidência (**woe**)

Logaritmo neperiano (Log na base 2,71826) da razão da chance

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Probabilidades



Flag negativo: 35
Flag positivo: 65
Total: 100

k_means	0	1	All	prob	risk_event	odds	odds_ratio	log_odds	woe
indep_3									
max	18	26	44	0.44	0.590909	1.444444	0.777778	0.367725	-0.251314
min	17	39	56	0.56	0.696429	2.294118	1.235294	0.830348	0.211309
All	35	65	100	1.00	0.650000	1.857143	1.000000	0.619039	0.000000

Probabilidades

```
Flag negativo: 35
Flag positivo: 65
Total: 100
```

k_means	0	1	All	prob	risk_event	odds	odds_ratio	log_odds	woe
indep_3									
max	18	26	44	0.44	0.590909	1.444444	0.777778	0.367725	-0.251314
min	17	39	56	0.56	0.696429	2.294118	1.235294	0.830348	0.211309
All	35	65	100	1.00	0.650000	1.857143	1.000000	0.619039	0.000000

Conclusões:

- A probabilidade (**prob**) de um dado em nosso banco ser de min é de 56%;
- O risco (**risk_event**) de min estar na classe 1 é de 69% e de max 59%;
- A chance (**odds**) de um min estar na classe 1 é de 2,29 e de um max, 1,44 vezes;
- Dado qualquer observação (**odds_ratio**), min tem 1,23 vezes de entrar na classe 1 e max 0,77 vezes;
- Dado qualquer observação (**log_odds**), min tem 0,83/log vezes de entrar na classe 1 e max 0,36/log vezes;
- A força explicativa (**woe**) do min em relação a classe 1 é menos provável para max (-0,25) do que para min (0,21);

Logistic Regression



Fórmula principal

A forma mais comum da fórmula é: $P(y = 1) = \frac{1}{1 + e^{-z}}$

Onde:

- $P(y = 1)$ é a probabilidade de o resultado ser "sucesso" (categoria 1).
- e é a base do logaritmo natural, o número de Euler.
- z é a combinação linear das variáveis independentes e seus coeficientes:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Handwritten derivation of the derivative of the logistic function on a chalkboard:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Logistic Regression



Logit Regression Results						
Dep. Variable:	k_means		No. Observations:	70		
Model:	Logit		Df Residuals:	68		
Method:	MLE		Df Model:	1		
Date:	Wed, 05 Nov 2025		Pseudo R-squ.:	1.000		
Time:	15:33:23		Log-Likelihood:	-1.2800e-06		
converged:	False		LL-Null:	-45.623		
Covariance Type:	nonrobust		LLR p-value:	1.269e-21		
	coef	std err	z	P> z	[0.025	0.975]
Intercept	280.4598	1.74e+04	0.016	0.987	-3.38e+04	3.43e+04
indep_1	-9.5088	589.941	-0.016	0.987	-1165.772	1146.755

Handwritten mathematical derivation of the derivative of a function $f(x)$ using the limit definition:

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Logistic Regression



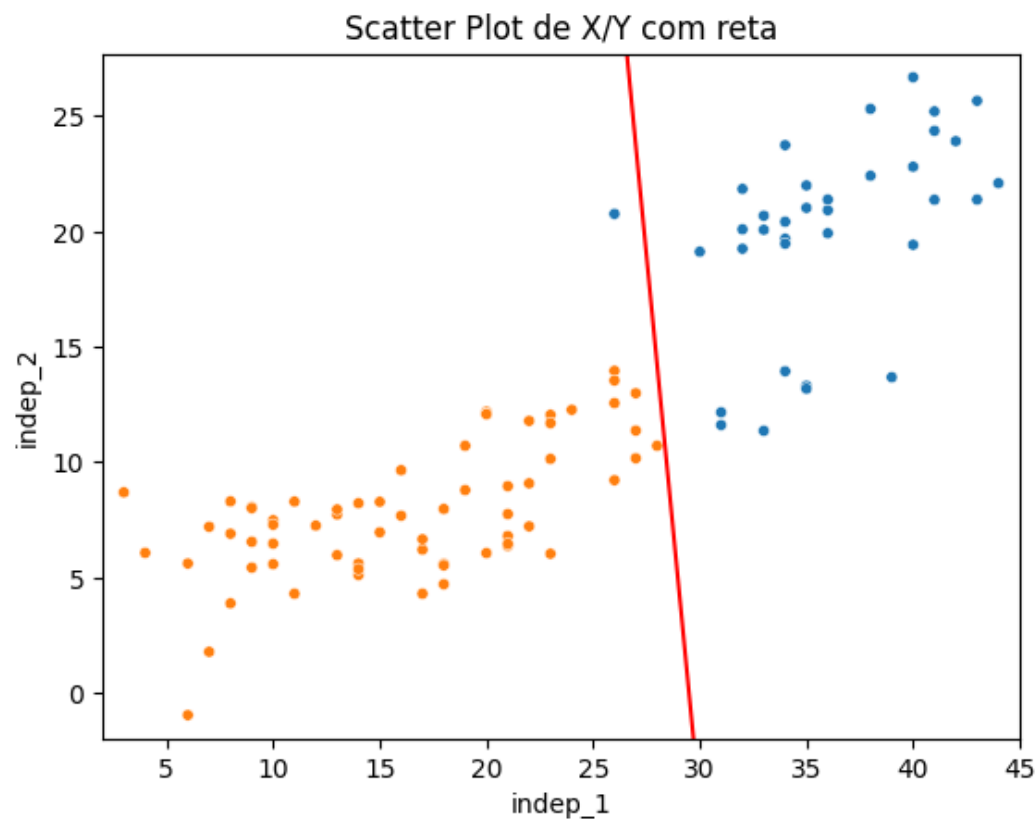
Descrição dos índices obtidos no modelo

coef		Estimativa pontual do coeficiente beta para a variável
std err		Erro padrão dado o coeficiente
z		Estatística Z que é o coef/std error
P> z		Probabilidade de o coef ser zero
[0.025 0.975]		Intervalo de confiança para 95%

A chalkboard with handwritten mathematical derivations. The first line shows the definition of the derivative: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. The second line shows the function $f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$. The third line shows the expansion of the numerator: $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$. The fourth line shows the simplification: $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$. The fifth line shows the final result: $= \lim_{h \rightarrow 0} h(2x + h)$.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Logistic Regression



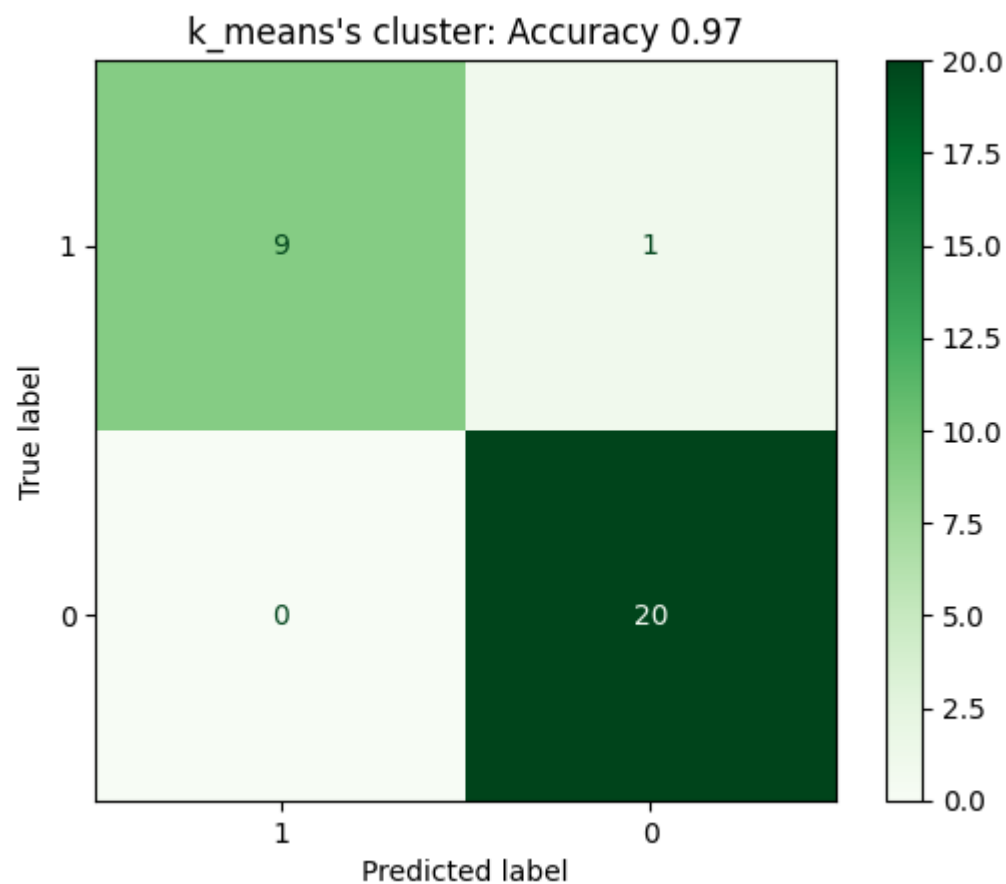
$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Intercept	280.459827
indep_1	-9.508832

$$Y = 280.45 + -9.5 * x$$

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Logistic Regression

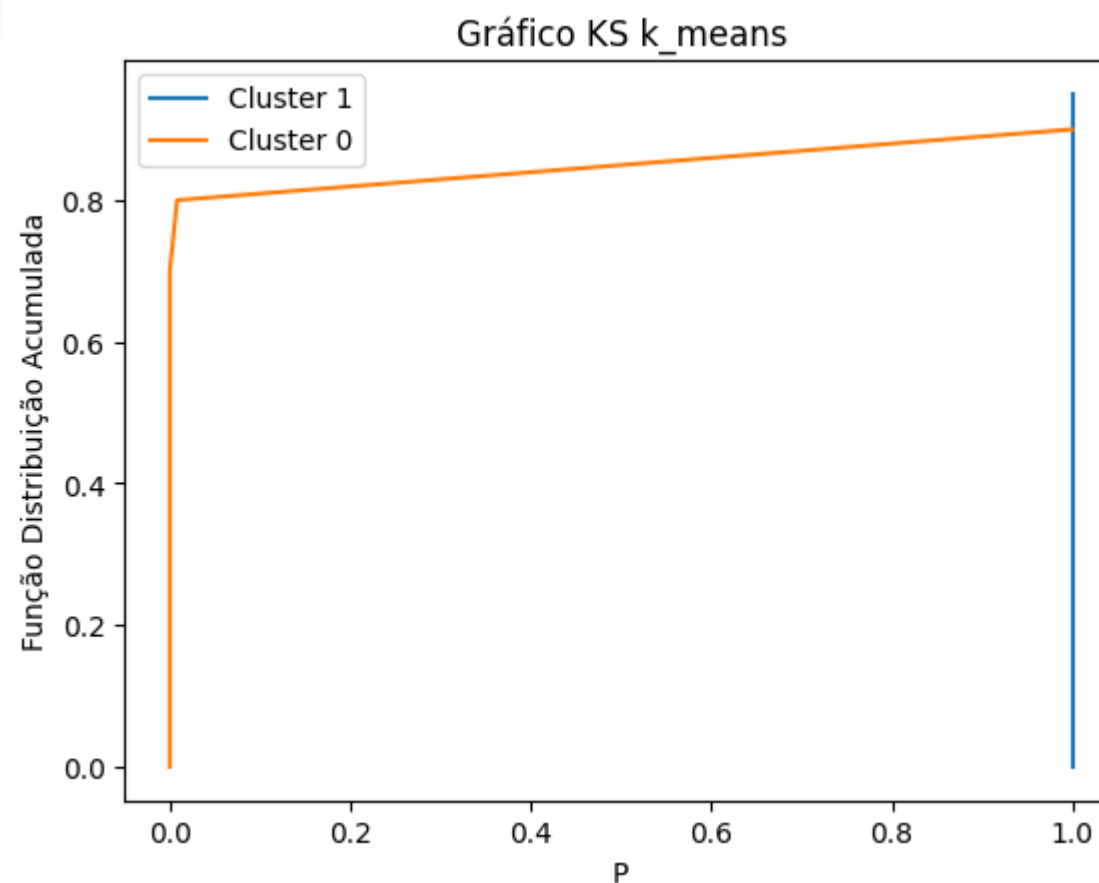
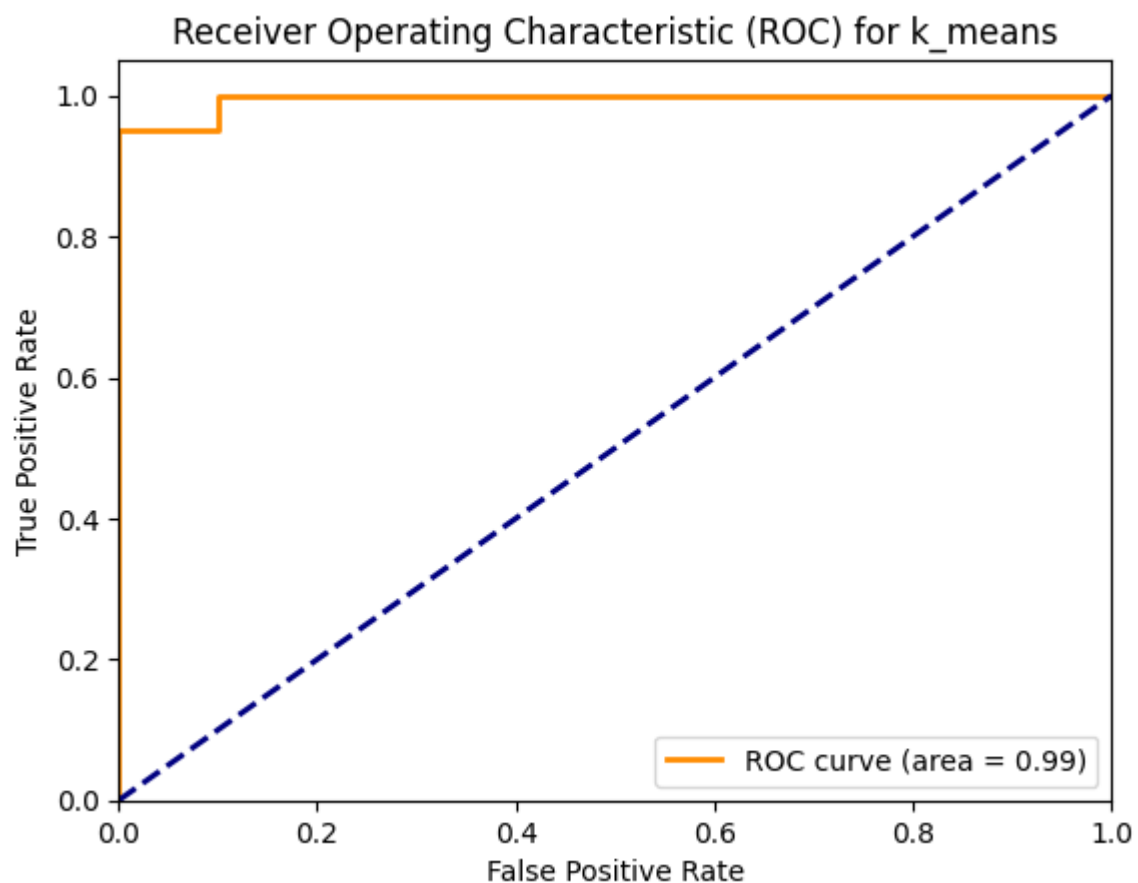


Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.90	0.95	10
1	0.95	1.00	0.98	20
accuracy			0.97	30
macro avg	0.98	0.95	0.96	30
weighted avg	0.97	0.97	0.97	30

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Logistic Regression

k_means:
Acurácia: 96.67%
Precision: 95.24%
Recall: 100.00%
F1-Score: 97.56%
KS: 95.00%
AUC: 99.50%
GINI: 99.00%



Logistic Regression

```
k_means:  
Acurácia: 99.00%  
KS: 97.14%  
AUC: 99.74%  
GINI: 99.47%
```

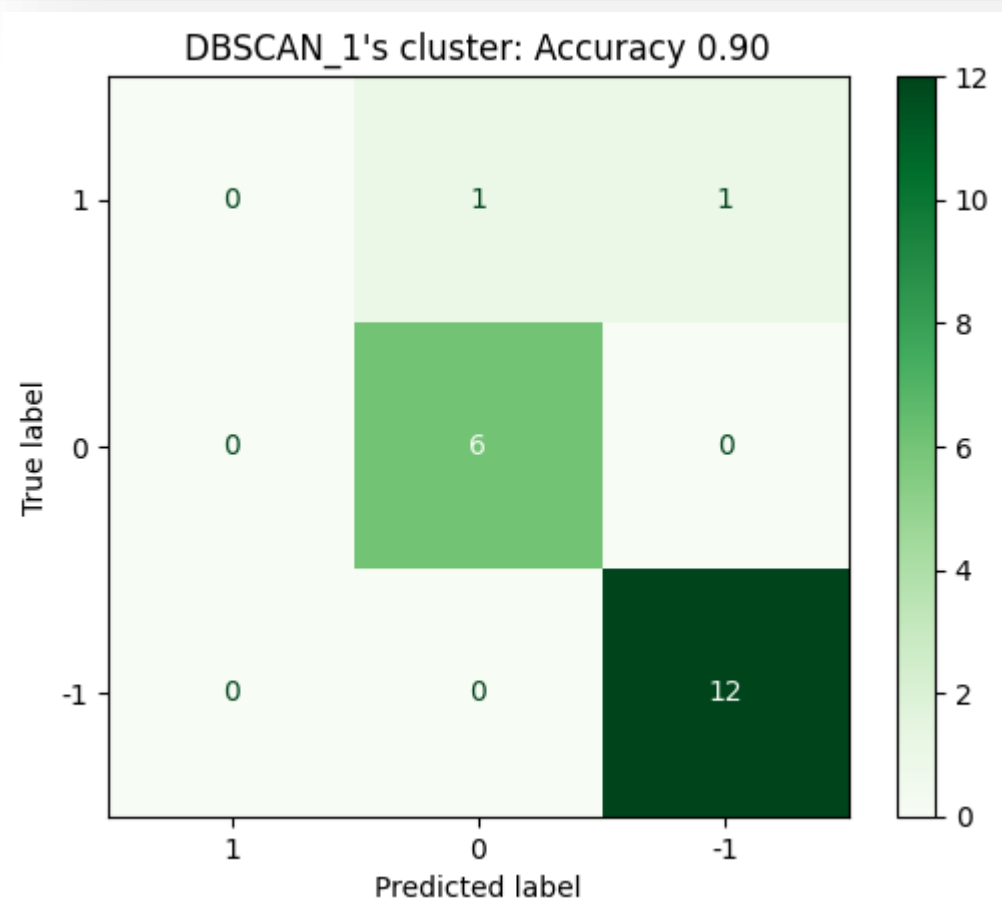


A métrica **Gini**, ou **Coeficiente de Gini**, é um índice que mede o grau de desigualdade na distribuição em uma variável para determinar a divisão ideal em cada nó de uma árvore. Varia de 0 a 1, onde 0 representa a igualdade perfeita e 1 representa a desigualdade máxima. Geralmente usado em distribuição não normal.

A métrica **AUC** (Area Under the Curve) é uma medida de desempenho em aprendizado de máquina para modelos de classificação, que avalia a capacidade do modelo de distinguir entre classes positiva e negativa.

Teste **KS** - Teste **de Kolmogorov-Smirnov**: este é um teste não paramétrico usado para comparar as distribuições contínuas subjacentes de duas amostras independentes para determinar se elas são estatisticamente diferentes.

Logistic Regression



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Logistic Regression



53	0.001343
93	0.997014
20	0.009283
177	0.986048
26	0.937342

	indep_1	indep_2	indep_3	indep_4	k_means	DBSCAN_1	DBSCAN_2	NN_k_means	k_NN	log_reg
53	41	24.36	max	max	0	0	0	0	0	0
93	9	8.05	min	med	1	1	1	1	1	1
20	36	19.92	max	max	0	0	0	0	0	0
177	13	7.73	max	min	1	1	1	1	1	1
26	17	6.21	max	min	1	1	1	1	1	1

Definindo um alvo



```
df_5['target'] = df_5[['k_means', 'DBSCAN_1', 'DBSCAN_2', 'NN_k_means', 'k_NN', 'log_reg']].mean(axis=1)
```



	indep_1	indep_2	indep_4	target	k_means	DBSCAN_1	DBSCAN_2	NN_k_means	k_NN	log_reg
53	41	24.36	max	0	0	0	0	0	0	0
93	9	8.05	med	1	1	1	1	1	1	1
20	36	19.92	max	0	0	0	0	0	0	0
177	13	7.73	min	1	1	1	1	1	1	1
26	17	6.21	min	1	1	1	1	1	1	1

Métricas



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676

Métricas



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676

SVM - Support Vector Machine



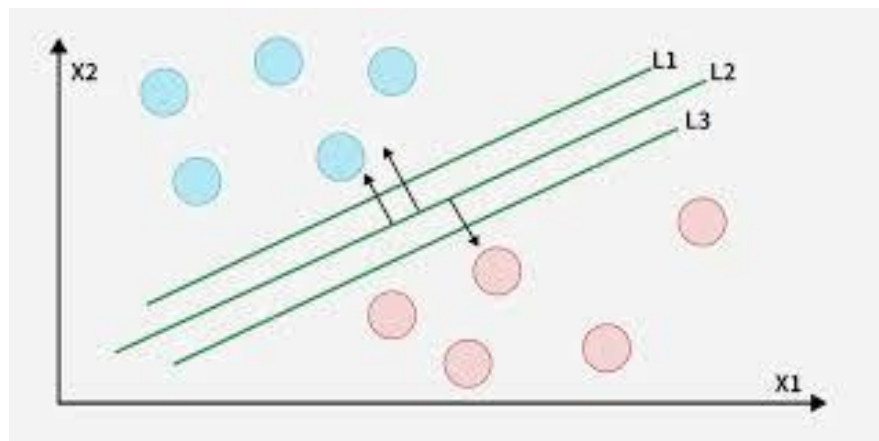
SVM significa Máquina de Vetores de Suporte (Support Vector Machine), um algoritmo de aprendizado de máquina supervisionado usado para tarefas de **classificação e regressão**. Ele funciona encontrando o "**hiperplano**" (uma linha em 2D, um plano em 3D, etc.) que melhor separa os dados em diferentes classes, maximizando a distância entre as classes. Essa margem máxima é definida pelos **pontos de dados mais próximos de cada classe**, chamados de vetores de suporte.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

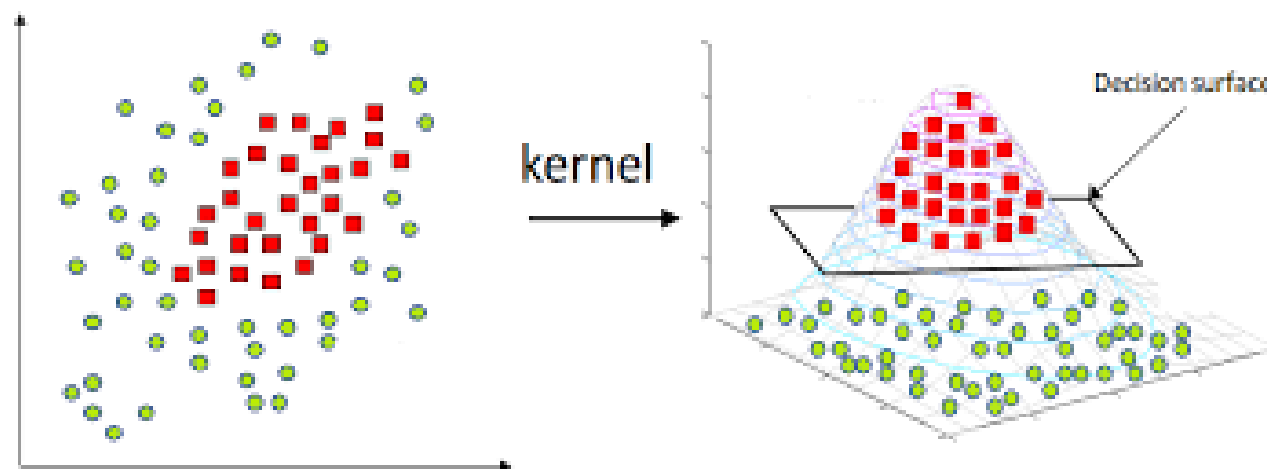
SVM



Support Vector



Support Vector Machine



SVM – Hiperparâmetro



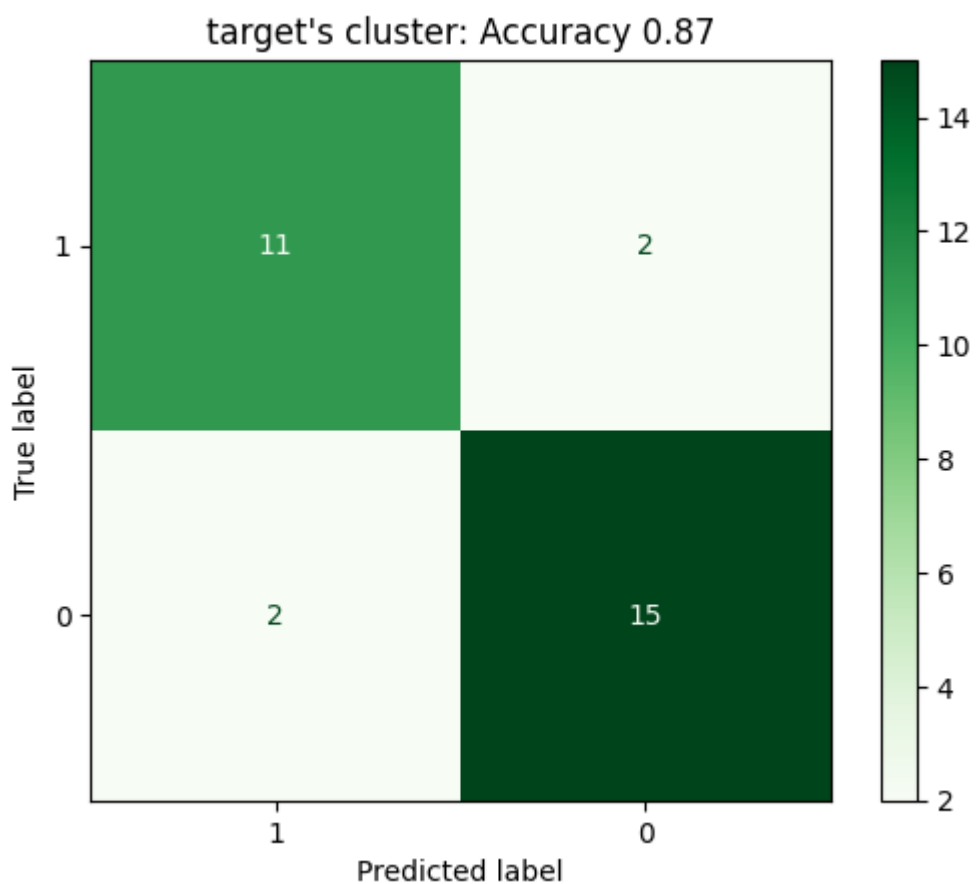
'linear': O kernel mais simples, adequado para dados linearmente separáveis. Ele calcula o produto escalar entre os vetores de características e resulta em um limite de decisão (ou hiperplano) linear.

'poly': O kernel polinomial é usado para dados não lineares e pode capturar relações curvas mais complexas. É definido pelo grau do polinômio (parâmetro `degree`) e pelo termo independente (parâmetro `coef0`).

'rbf': O RBF (Radial Basis Function), ou kernel Gaussiano, é o kernel padrão para SVC e SVR. É altamente eficaz para dados não lineares e mapeia os dados de entrada para um espaço de dimensão superior onde podem se tornar linearmente separáveis.

'sigmoid': O kernel sigmoide usa a função tangente hiperbólica e pode ser uma alternativa ao RBF, às vezes se comportando de forma semelhante à função de ativação de uma rede neural de duas camadas.

SVM



Classification Report:				
	precision	recall	f1-score	support
0	0.85	0.85	0.85	13
1	0.88	0.88	0.88	17
accuracy			0.87	30
macro avg	0.86	0.86	0.86	30
weighted avg	0.87	0.87	0.87	30

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

SVM



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8667	0.8824	0.8824	0.8824	0.7285	0.8643	0.7285

Receiver Operating Characteristic (ROC) for target

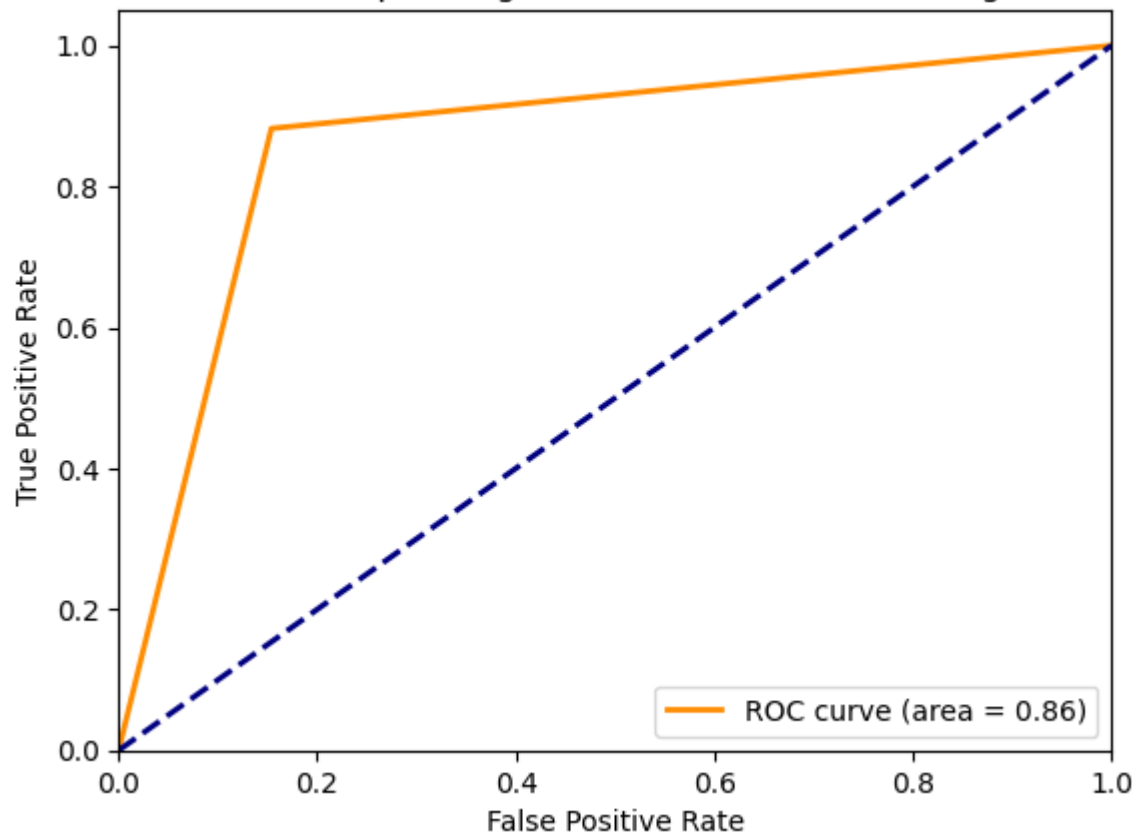
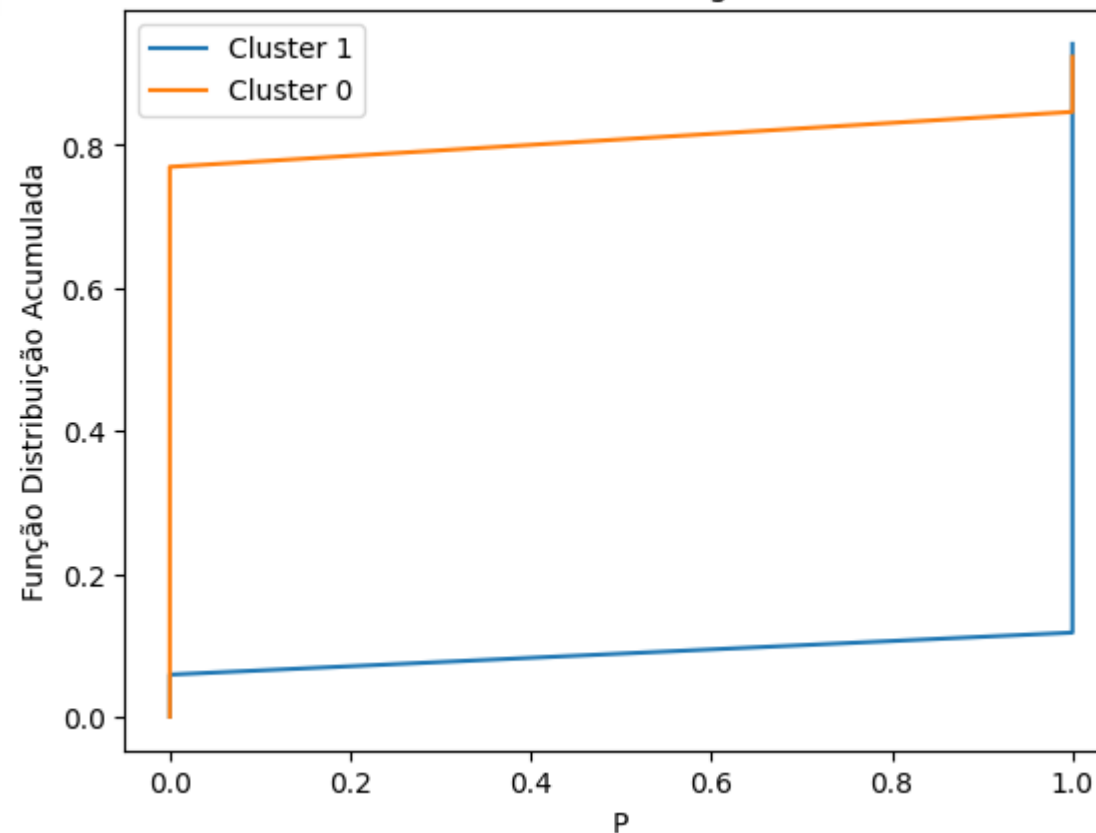


Gráfico KS target



SVM



	indep_1	indep_2	indep_3	indep_4
53	41	24.36	max	max
93	9	8.05	min	med
20	36	19.92	max	max
177	13	7.73	max	min
26	17	6.21	max	min

get_dummies

	indep_1	indep_2	indep_3_max	indep_3_min	indep_4_min	indep_4_med	indep_4_max	target
132	18	5.53	0	1	1	0	0	1
143	14	5.61	0	1	1	0	0	1
121	14	5.37	0	1	1	0	0	1
91	22	7.22	0	1	1	0	0	1
74	20	12.19	0	1	0	1	0	0

SVM – kernel



linear

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8667	0.8824	0.8824	0.8824	0.7285	0.8643	0.7285

poly

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.9	0.8889	0.9412	0.9143	0.7873	0.8937	0.7873

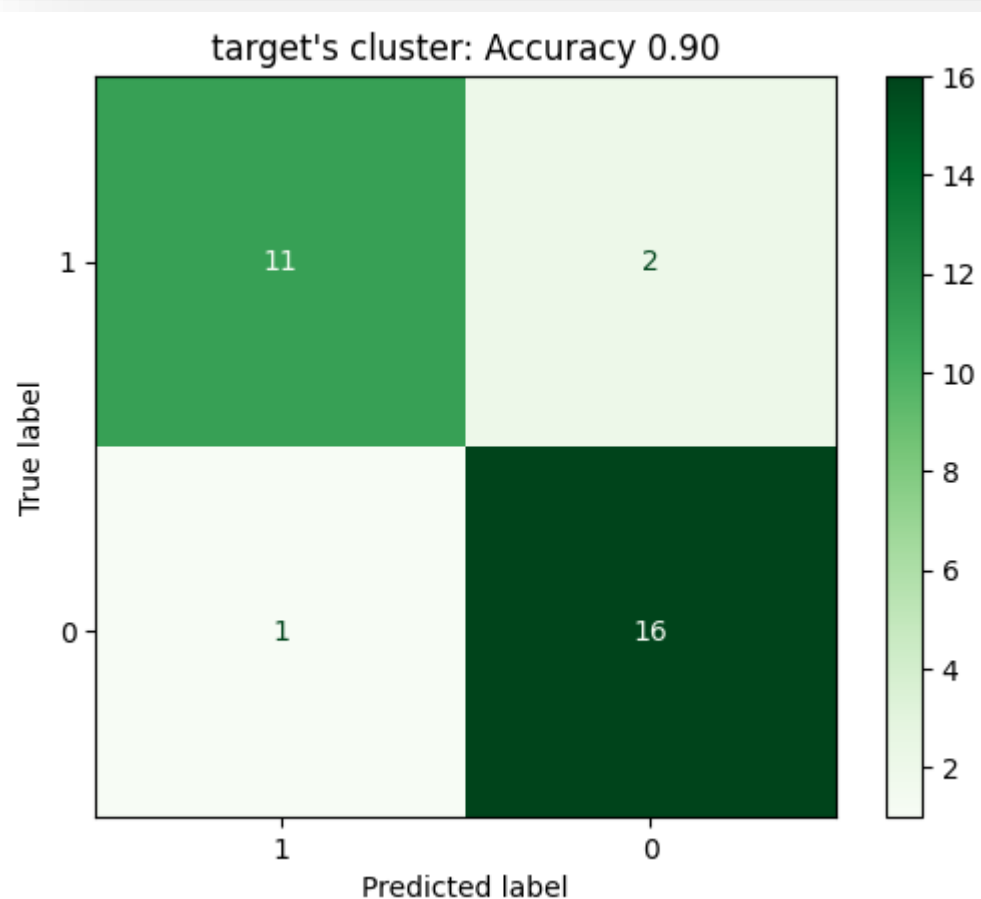
rbf

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.9	0.8889	0.9412	0.9143	0.7873	0.8937	0.7873

sigmoid

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.1667	0.1	0.0588	0.0741	0.6335	0.1833	-0.6335

SVM – múltipla, polinomial



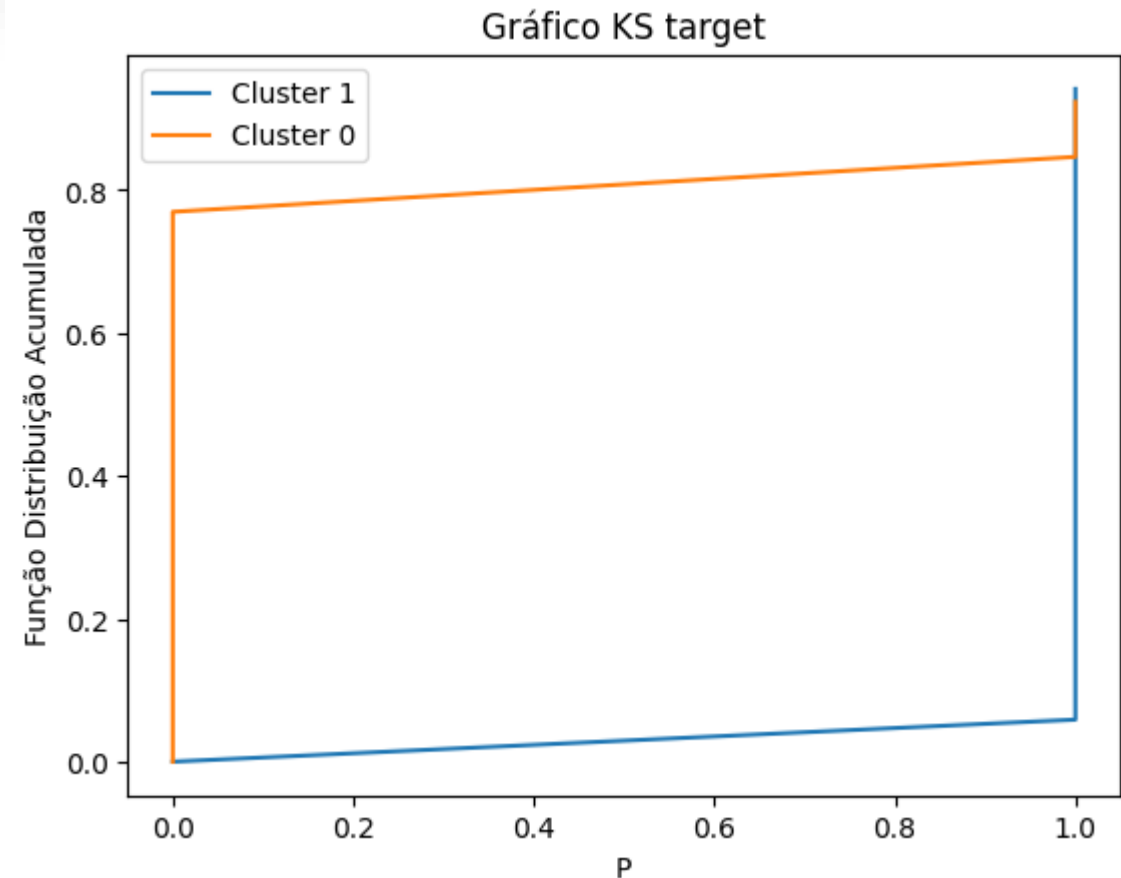
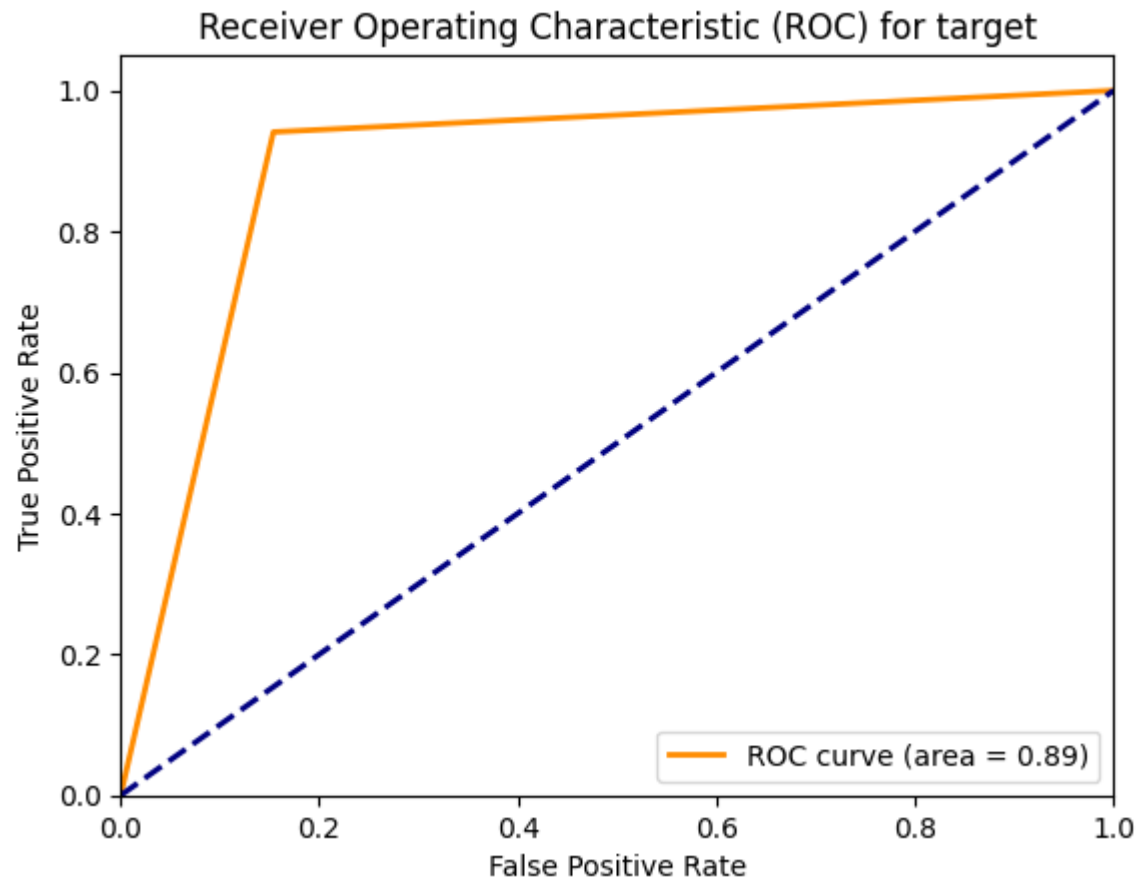
Classification Report:				
	precision	recall	f1-score	support
0	0.92	0.85	0.88	13
1	0.89	0.94	0.91	17
accuracy			0.90	30
macro avg	0.90	0.89	0.90	30
weighted avg	0.90	0.90	0.90	30

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

SVM – múltipla, polinomial



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.9	0.8889	0.9412	0.9143	0.7873	0.8937	0.7873



Métricas



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873

SVM



	indep_1	indep_2	indep_3	indep_4	target	k_means	DBSCAN_1	DBSCAN_2	NN_k_means	k_NN	log_reg	SVM
53	41	24.36	max	max	0	0	0	0	0	0	0	0
93	9	8.05	min	med	1	1	1	1	1	1	1	1
20	36	19.92	max	max	0	0	0	0	0	0	0	0
177	13	7.73	max	min	1	1	1	1	1	1	1	1
26	17	6.21	max	min	1	1	1	1	1	1	1	1

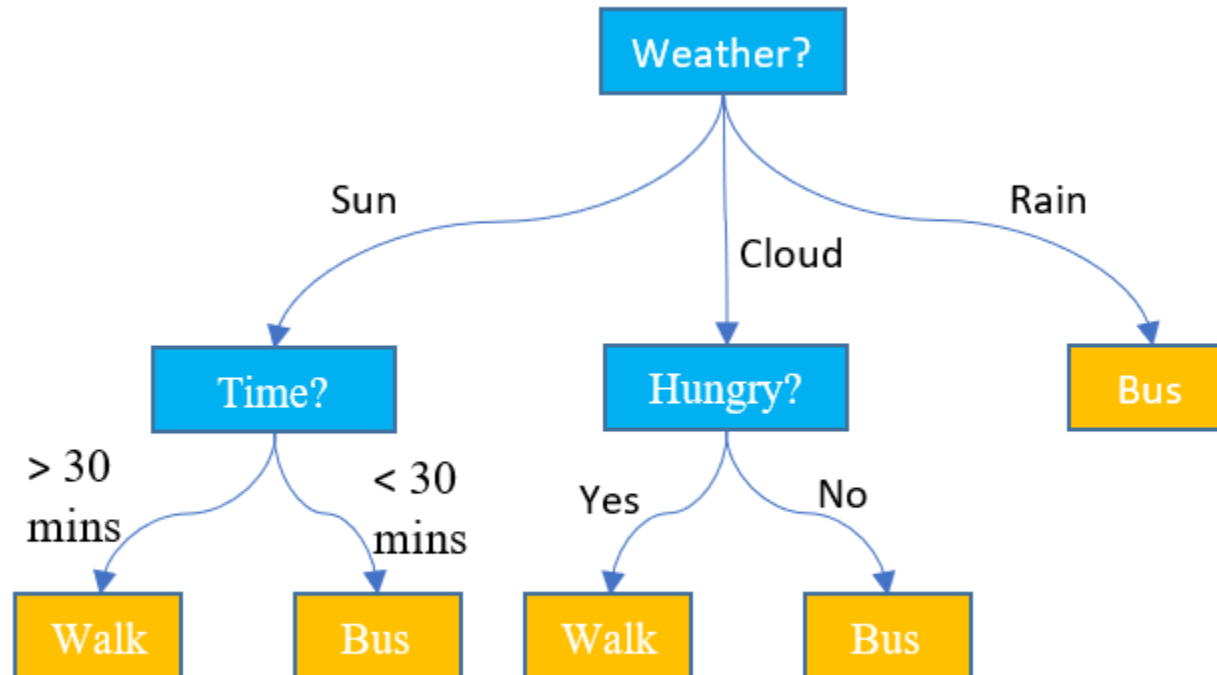
Decision Trees



Uma **árvore de decisão** é um algoritmo de aprendizado de máquina supervisionado que usa uma estrutura hierárquica, semelhante a um **fluxograma**, de perguntas ou condições para tomar decisões ou fazer previsões com base em um conjunto de dados. São ferramentas poderosas e interpretáveis usadas tanto para tarefas de **classificação** (previsão de categorias) quanto de **regressão** (previsão de valores numéricos).

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Decision Trees



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Decision Trees - hiperparâmetros

max_depth

Profundidade máxima da árvore

min_samples_leaf

Número mínimo de amostras ligadas por variável

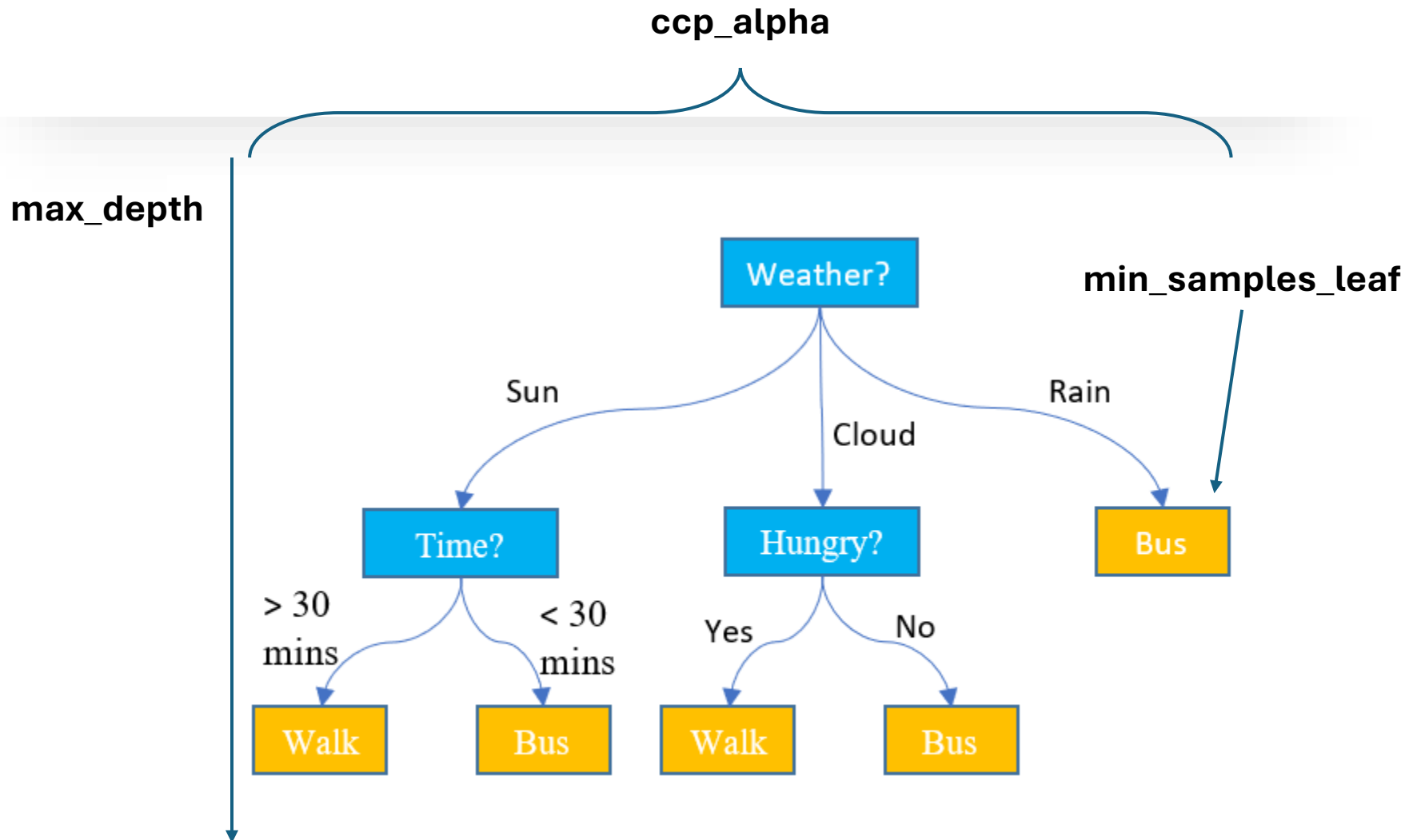
ccp_alpha

caminho de poda de complexidade de custo. Custo de complexidade da árvore

Handwritten mathematical derivation on a chalkboard:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Decision Trees - hiperparâmetros



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Decision Trees – multiparamétrica



	indep_1	indep_2	indep_3_max	indep_3_min	indep_4_min	indep_4_med	indep_4_max	target
53	41	24.36	1	0	0	0	1	0
93	9	8.05	0	1	0	1	0	1
20	36	19.92	1	0	0	0	1	0
177	13	7.73	1	0	1	0	0	1
26	17	6.21	1	0	1	0	0	1

Decision Trees – multiparamétrica

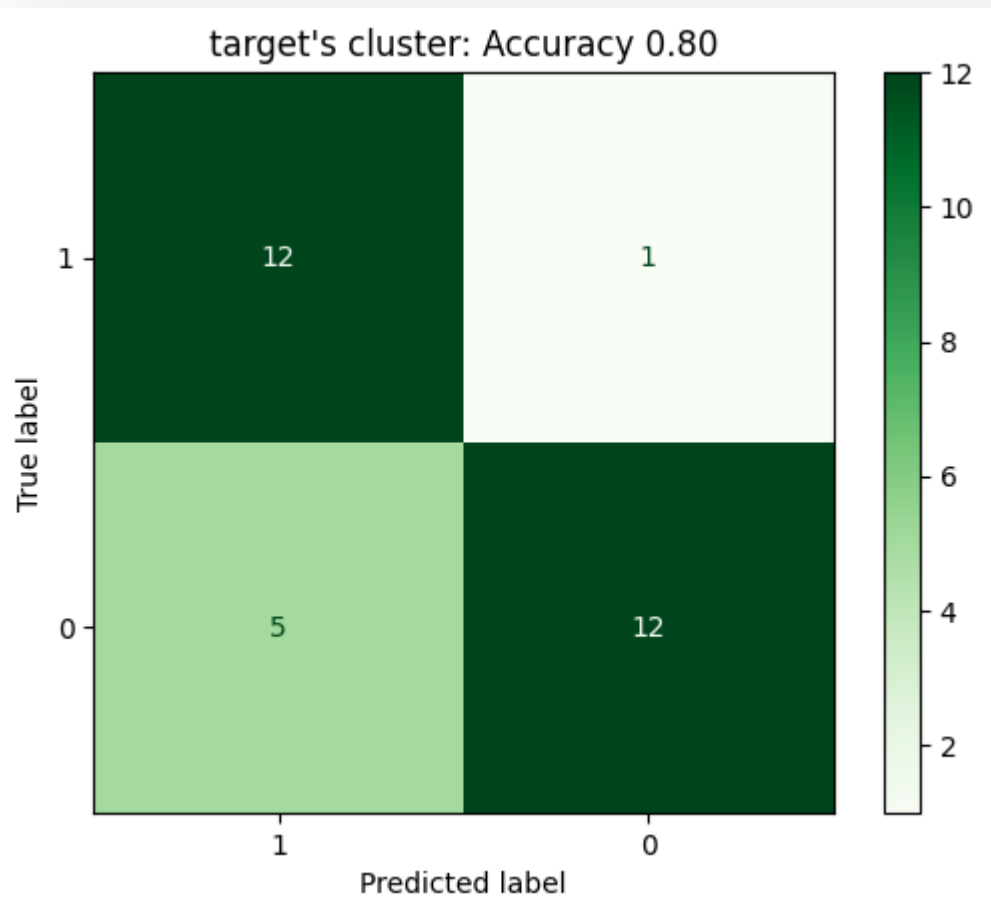


```
clf = DecisionTreeClassifier(max_depth=10, min_samples_leaf=1, random_state=42)
```

	importance	columns
1	1.0	indep_2
0	0.0	indep_1
2	0.0	indep_3_max
3	0.0	indep_3_min
4	0.0	indep_4_min
5	0.0	indep_4_med
6	0.0	indep_4_max

```
n: 1 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 2 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 3 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 4 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 5 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 6 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00  
n: 7 -----  
Treinamento  
0 R² da árvore com profundidade 1 é: 1,00
```

Decision Trees – multiparamétrica



Classification Report:				
	precision	recall	f1-score	support
0	0.71	0.92	0.80	13
1	0.92	0.71	0.80	17
accuracy			0.80	30
macro avg	0.81	0.81	0.80	30
weighted avg	0.83	0.80	0.80	30

Decision Trees – multiparamétrica



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8	0.9231	0.7059	0.8	0.629	0.8145	0.629

Receiver Operating Characteristic (ROC) for target

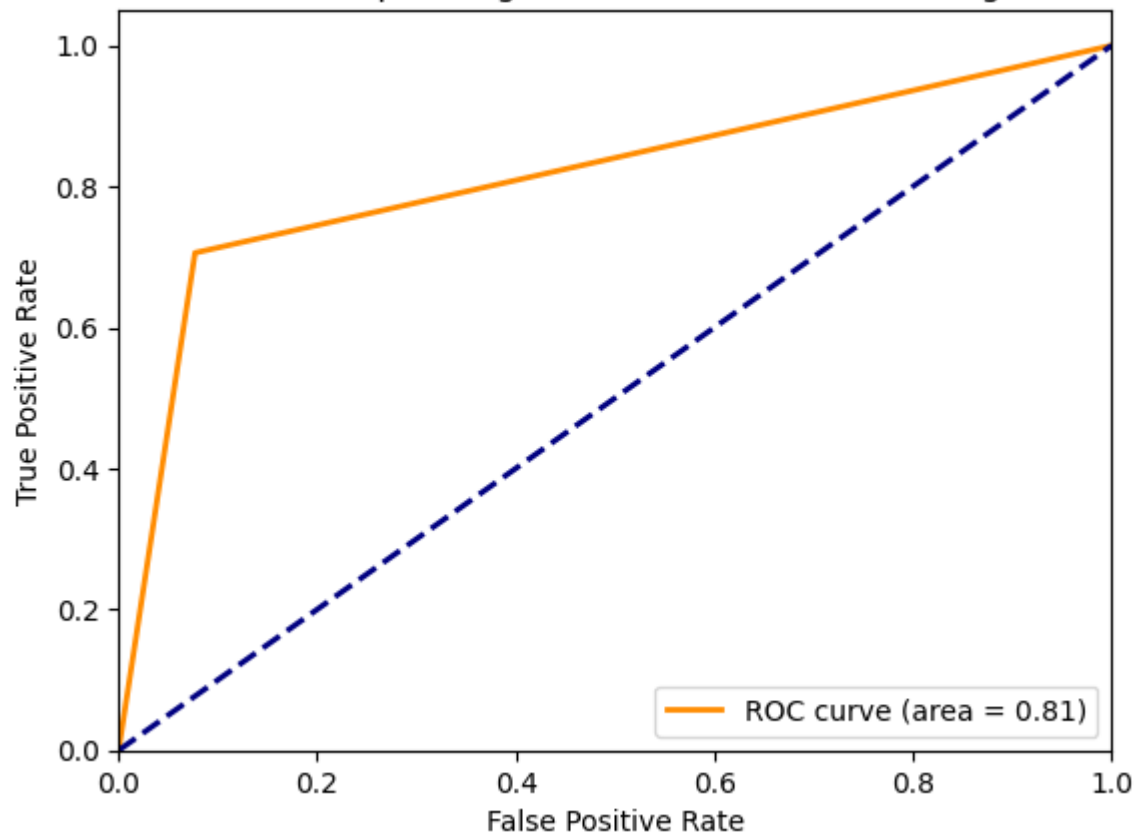
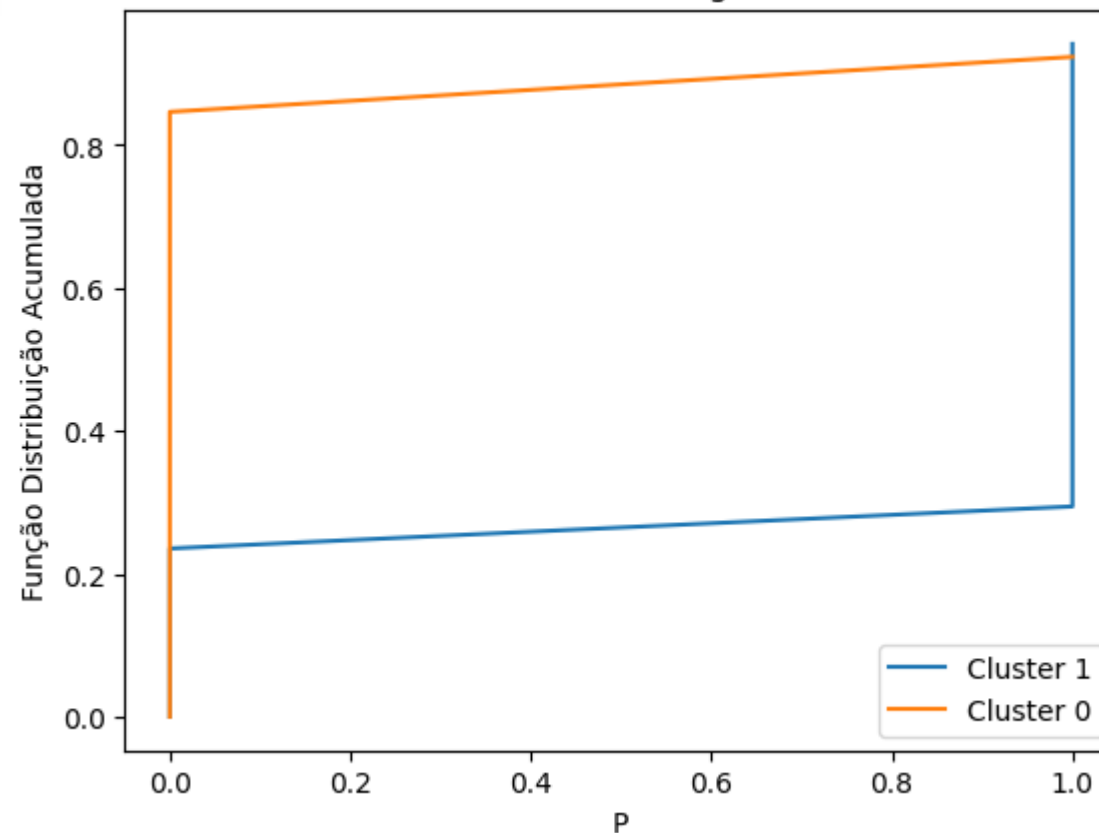
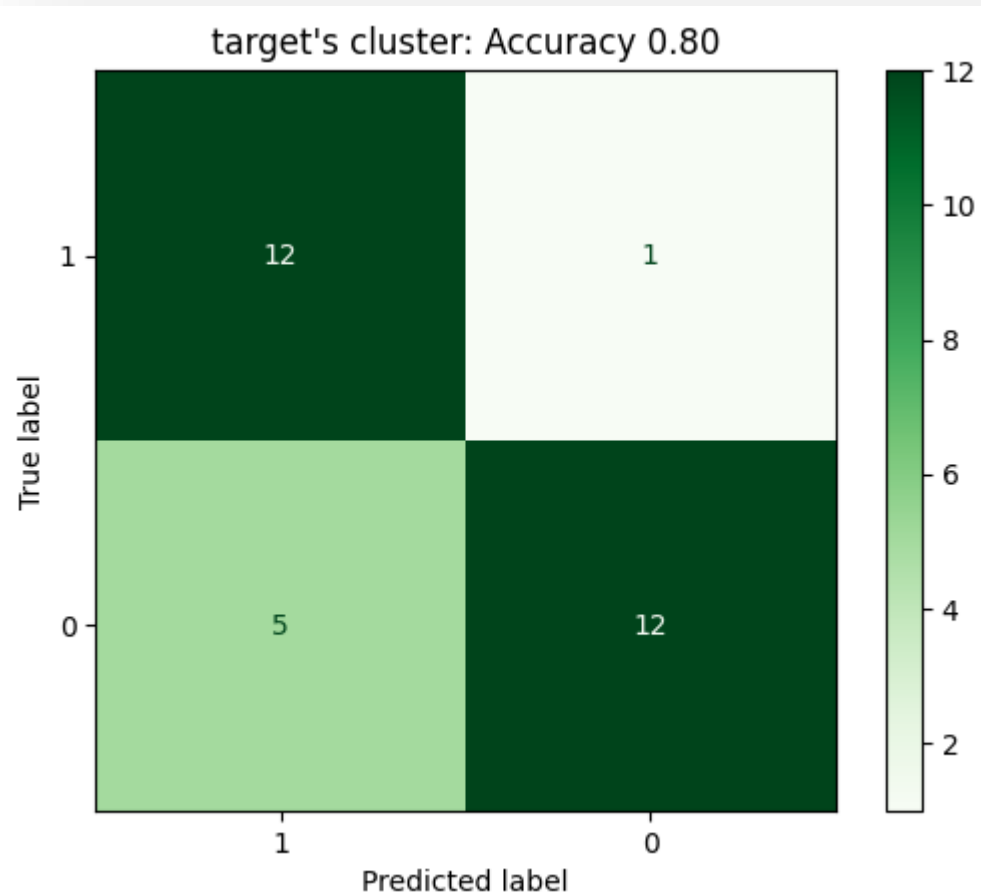


Gráfico KS target



Decision Trees – uni paramétrica



Classification Report:				
	precision	recall	f1-score	support
0	0.71	0.92	0.80	13
1	0.92	0.71	0.80	17
accuracy			0.80	30
macro avg	0.81	0.81	0.80	30
weighted avg	0.83	0.80	0.80	30

Decision Trees – uni paramétrica



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8	0.9231	0.7059	0.8	0.629	0.8145	0.629

Receiver Operating Characteristic (ROC) for target

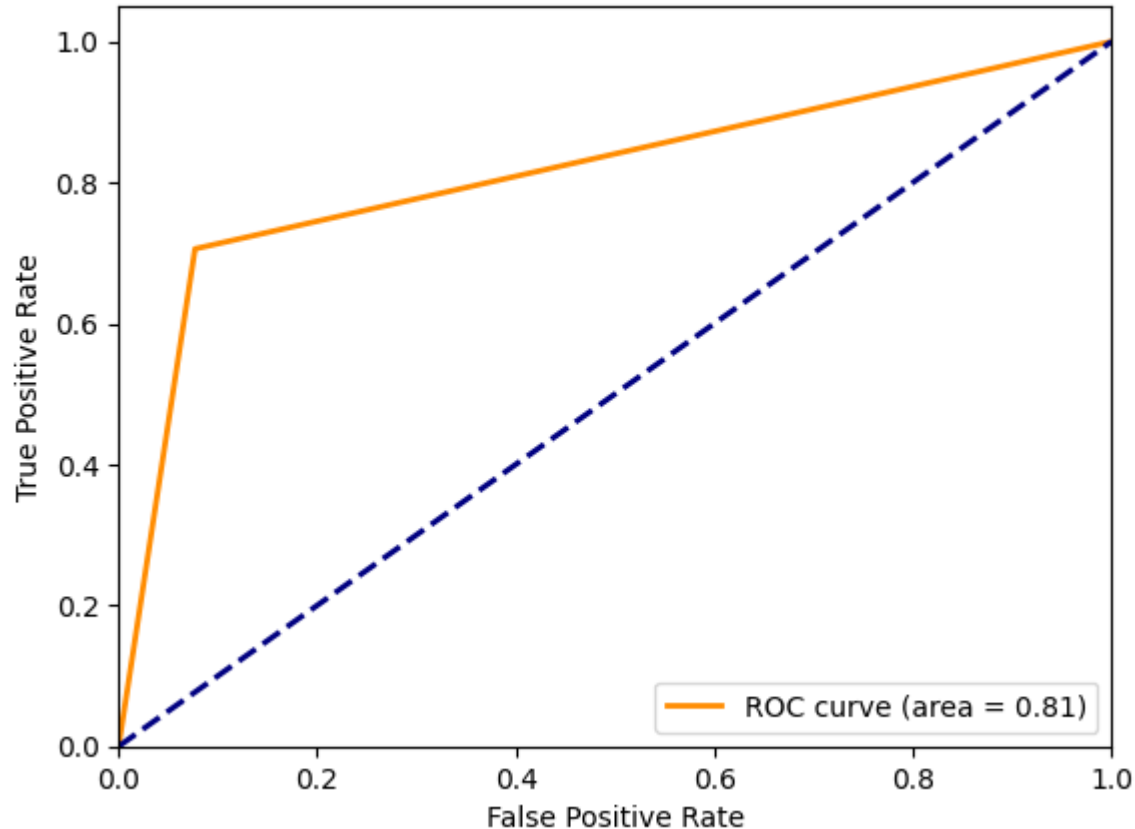
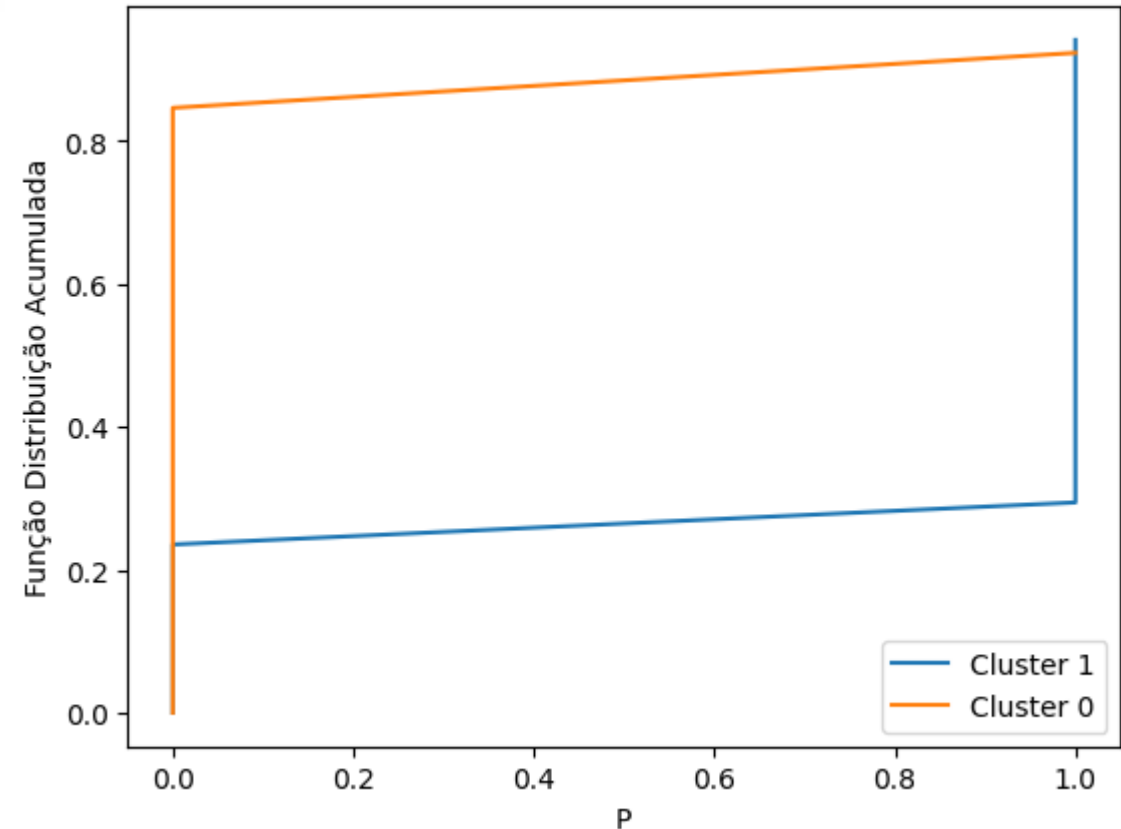


Gráfico KS target



Decision Trees



Uni paramétrica

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8	0.9231	0.7059	0.8	0.629	0.8145	0.629

Multiparamétrica

Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8	0.9231	0.7059	0.8	0.629	0.8145	0.629

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Decision Trees



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873
dec_tree	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290

Random Forests



Uma **Random Forest** (ou Floresta Aleatória, em português) é um algoritmo de aprendizado de máquina de conjunto (**ensemble**) que combina múltiplas árvores de decisão para produzir uma previsão única, mais precisa e estável. É um método versátil de aprendizado supervisionado, utilizado tanto para tarefas de **classificação** (prever uma categoria) quanto de **regressão** (prever um valor numérico).

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Random Forests



Agregação por Bootstrap (Bagging): Cada árvore na floresta é treinada em um subconjunto diferente e aleatório dos dados de treinamento originais, **amostrados com reposição** (o que significa que alguns pontos de dados podem aparecer múltiplas vezes, enquanto outros ficam de fora). Os pontos de dados não utilizados para cada árvore são chamados de amostras "**out-of-bag**" (OOB) e podem ser usados para avaliação interna do modelo.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

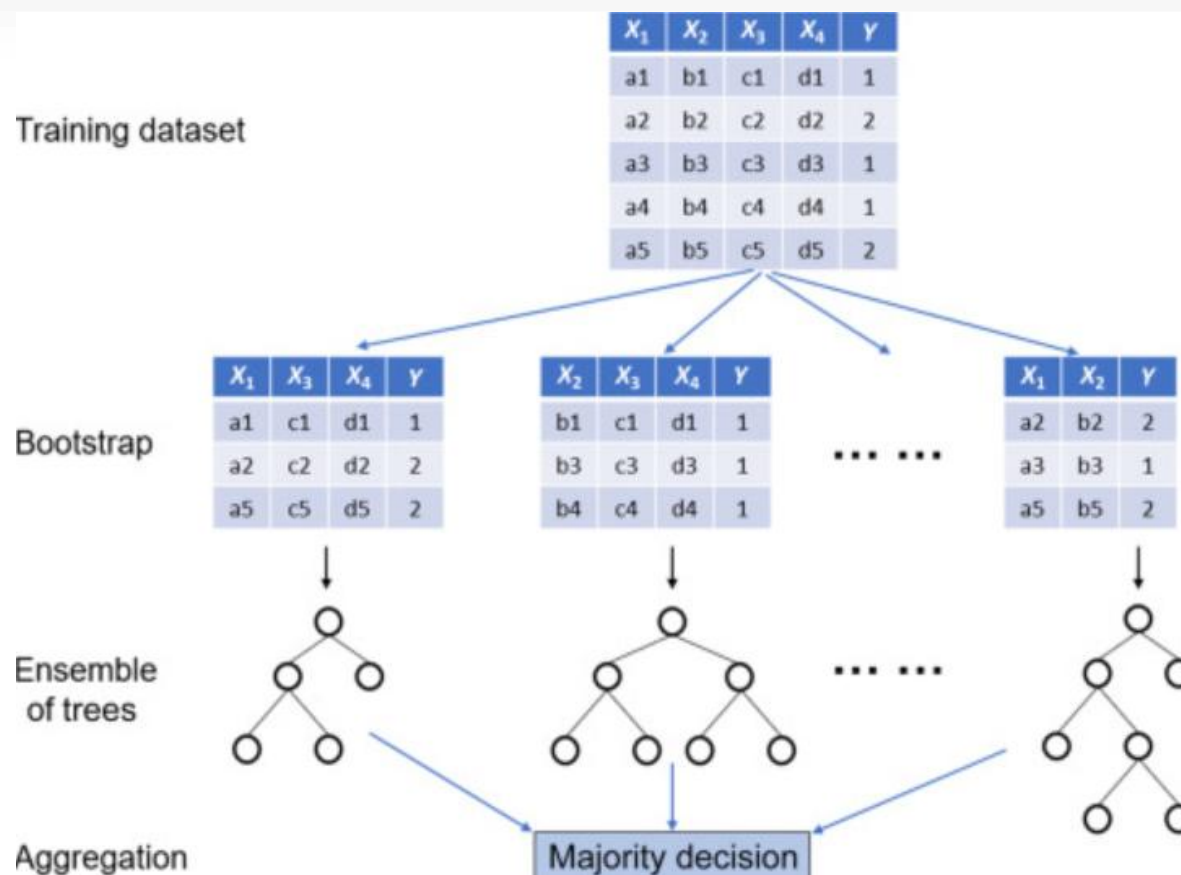
Random Forests



Seleção Aleatória de Atributos (Features): Em cada nó dentro de uma árvore de decisão, apenas um subconjunto aleatório de atributos (**variáveis de entrada**) é considerado ao decidir a melhor divisão, em vez de **todos os atributos**. Isso garante que as árvores sejam diversas e não altamente correlacionadas entre si.

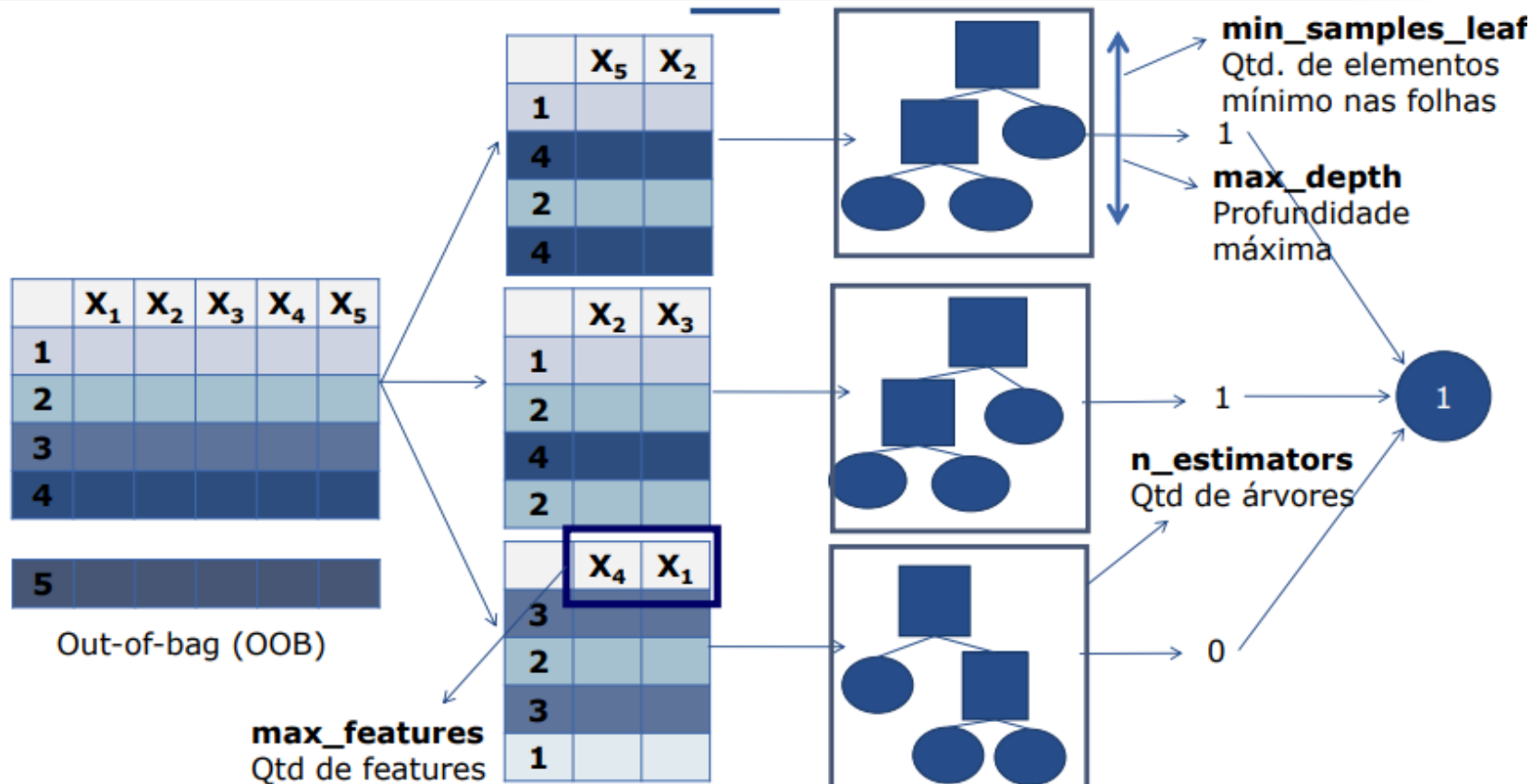
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Random Forests - bootstrap



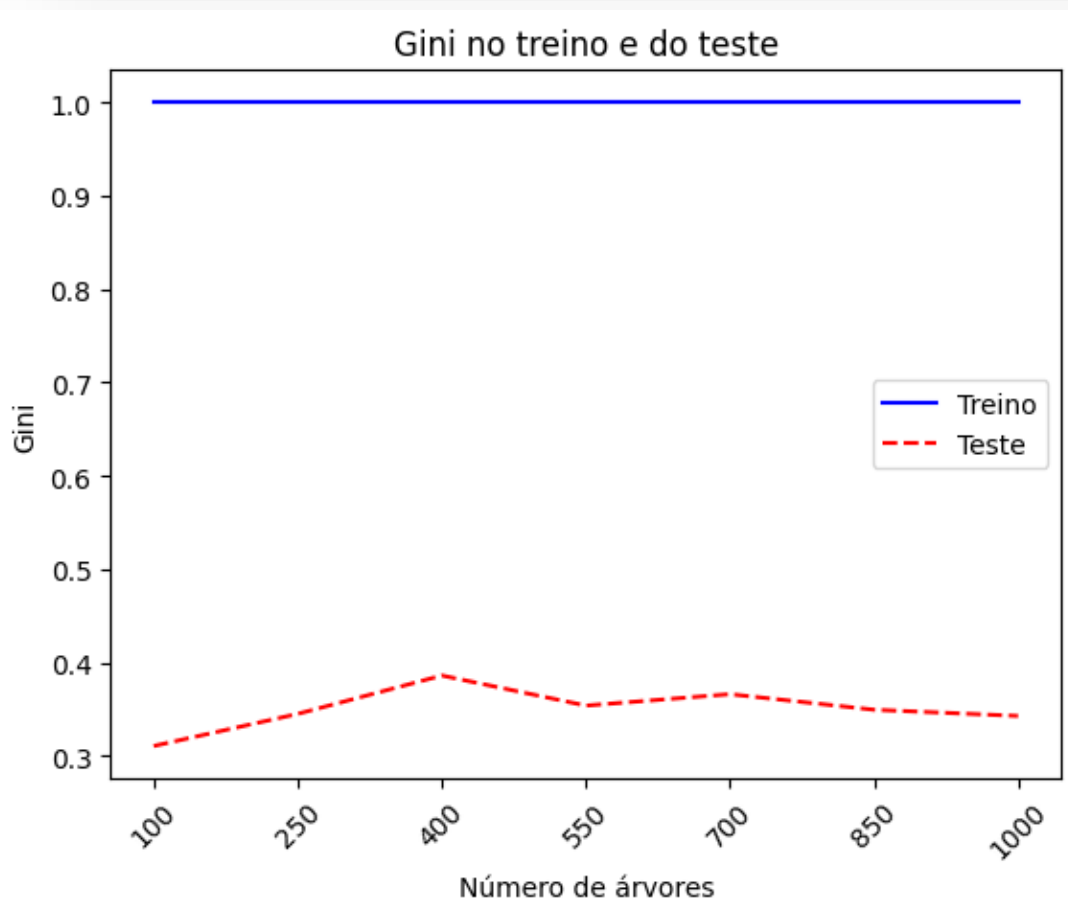
$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Random Forests - hiperparâmetros



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} (2x + h)$$

Random Forests – n_estimators



gini

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Grid Search

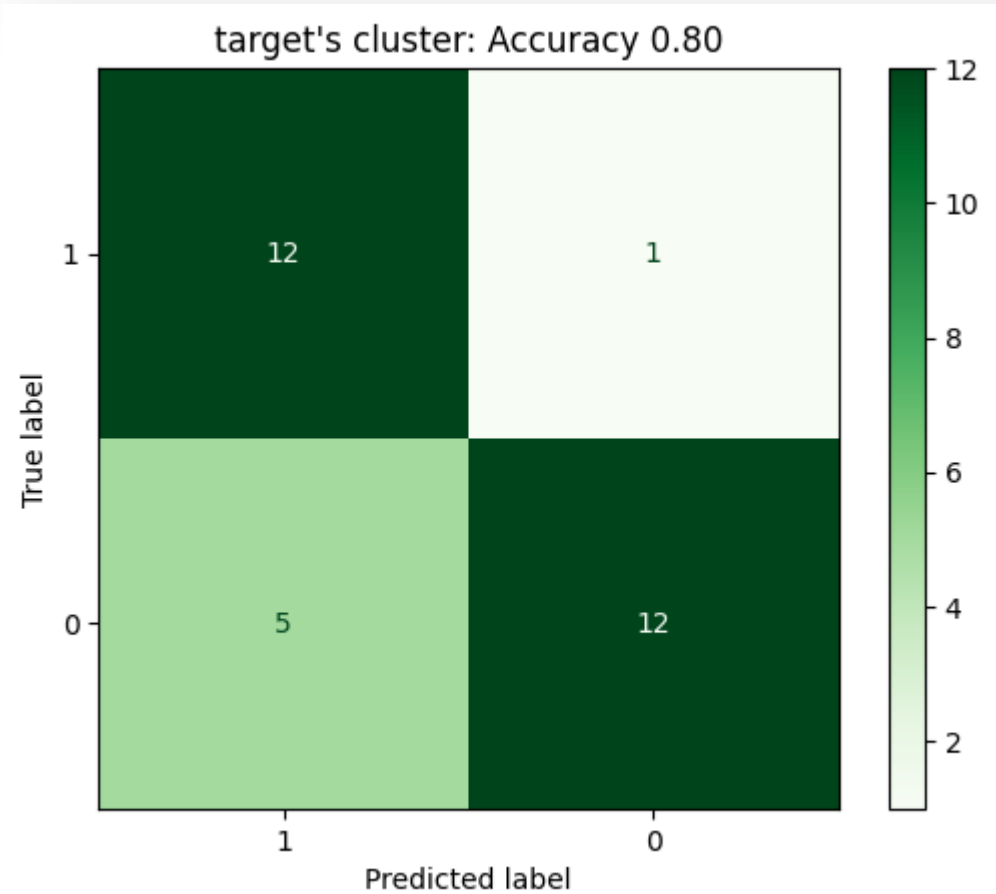


```
params = {  
    'max_depth' : [2, 10, 15],  
    'min_samples_leaf': [1, 2, 5],  
    'n_estimators': list(range(100, 1001, 150))  
}
```

```
grid_rf = GridSearchCV(  
    estimator=rf,           # Algoritmo a ser utilizado  
    param_grid=params,     # Parâmetros a serem testados  
    scoring='accuracy',    # Métrica de qualidade do modelo  
    cv=5,                  # Número de folds para validação cruzada  
    verbose=0,             # Verbosidade do processo  
    n_jobs=-1              # Use todos os CPUs disponíveis  
)
```


Random Forests

```
RandomForestClassifier(max_depth=1)  
0.8857142857142857  
0.8857142857142857
```



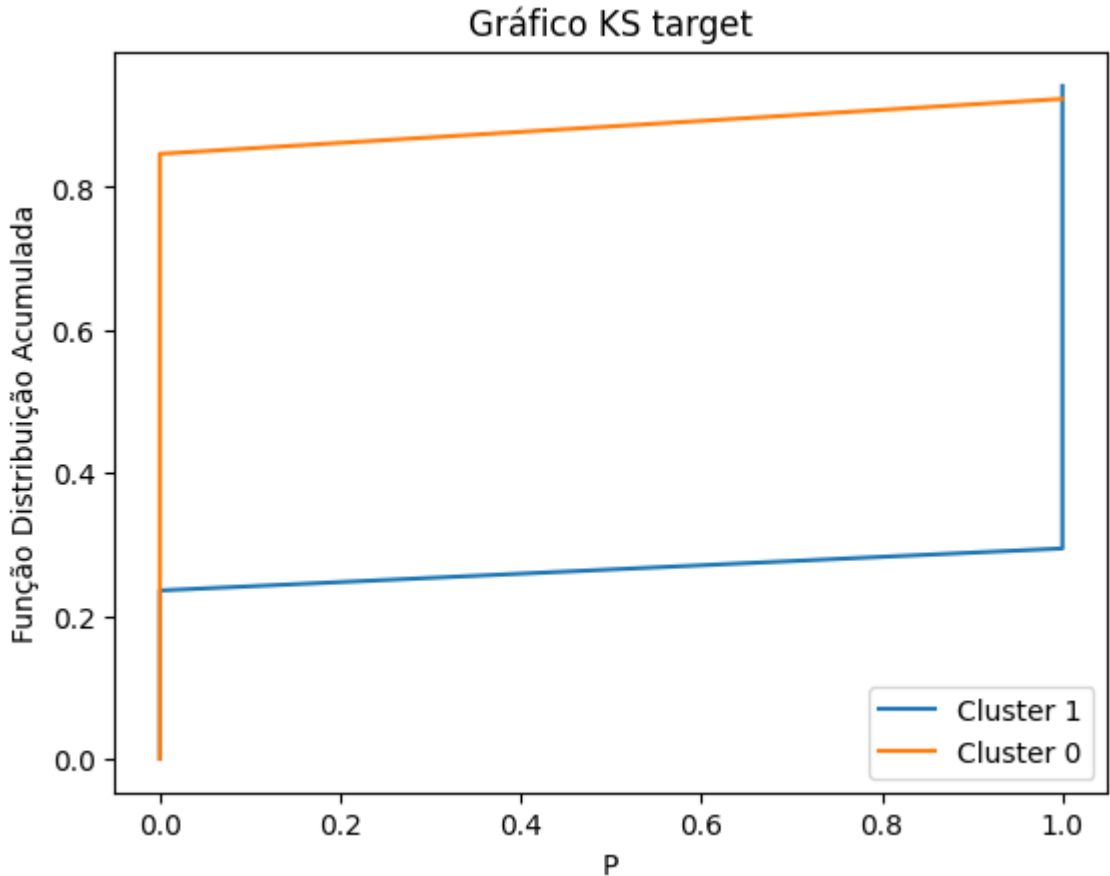
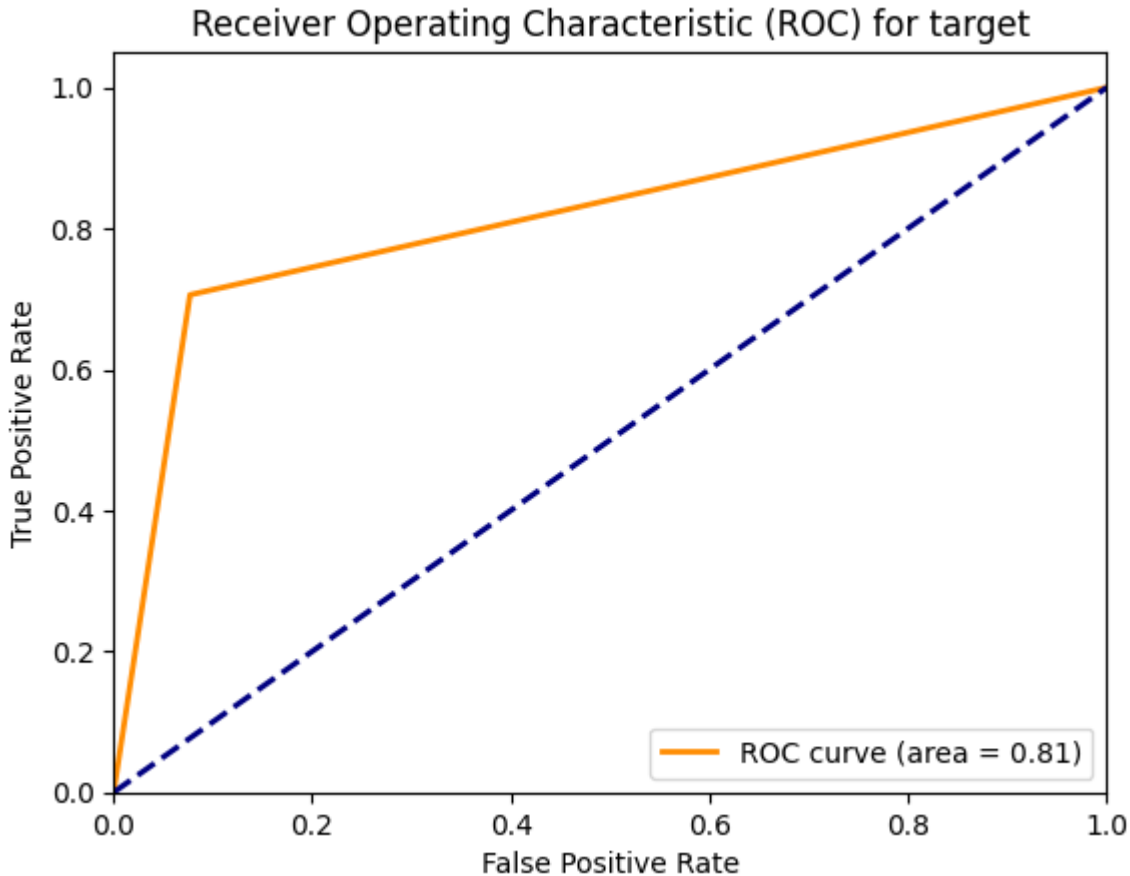
Classification Report:				
	precision	recall	f1-score	support
0	0.71	0.92	0.80	13
1	0.92	0.71	0.80	17
accuracy			0.80	30
macro avg	0.81	0.81	0.80	30
weighted avg	0.83	0.80	0.80	30

Random Forests

```
RandomForestClassifier(max_depth=1)  
0.8857142857142857  
0.8857142857142857
```



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8	0.9231	0.7059	0.8	0.629	0.8145	0.629



Decision Trees



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873
Dec Tree	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
Random Forest	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290

Gradient Boosting Machine – GBM



Boosting: é um método de *ensemble* (mistura de modelos) que tem a finalidade de combinar modelos **fracos** de modo a resultar em um **forte**. A forma mais popular do ***boosting*** hoje consiste em uma construção sequencial de modelos, cada um tentando reduzir o erro do anterior.

Gradient Boosting Machine: É o algoritmo de ***boosting*** mais popular. Consiste em construir árvores sequencialmente, em que uma árvore tenta ***melhorar*** a previsão sobre o erro da anterior.

GBM



XGBoosting: é uma implementação do ***gradient boosting*** bastante eficiente e de código aberto, que ficou bastante popular por ser utilizada em diversas implementações de soluções vencedoras em competições de ciência de dados pelo mundo todo.

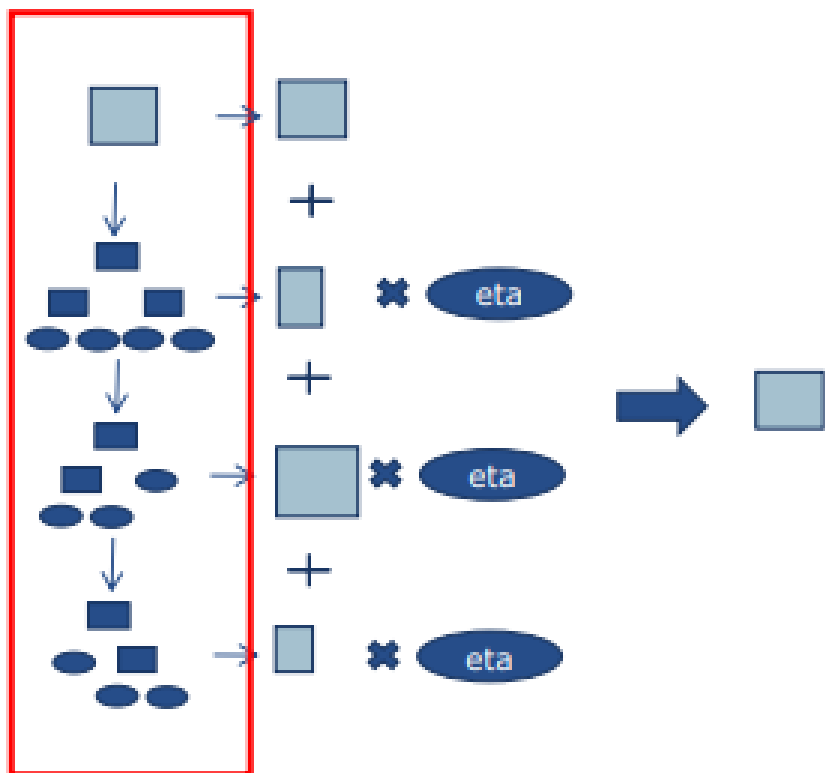
GBM



A lógica do algoritmo é a seguinte:

1. Partir da **média** do target
2. Calcular o **resíduo** entre a **média** e o valor do **target** de cada linha
3. Ajustar uma **árvore de decisão** na base de desenvolvimento usando o **resíduo como target**
4. Gravar os **desvios** de previsão em cada observação (**resíduos**)
5. Ajustar uma nova árvore para explicar os **resíduos obtidos em 2**
6. Repetir 2, 3, 4 e 5 até atingir um **critério de parada**

GBM - hiperparâmetros



n_estimators: número de árvores;

max_depth: profundidade das árvores;

min_samples_leaf: ligação por folha;

learning_rate: taxa de correção de erro;

Floresta de Árvores

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

GBM



GBM

Passo na
direção

eta = 0,1

	X ₁	X ₂	X ₃	Y (altura cm)
1				190
2				185
3				170
4				165
5				175
6				159

Pred ₁	Pred ₂	Pred ₃
174	16	12,15
174	11	12,15
174	-6,5	-3,35
174	-6,5	-8,35
174	1	0,9
174	-15	-13,5

Pred ₁ + (eta * Pred ₂) + (eta * Pred ₃)
176,82
176,32
173
172,5
174,2
171,2

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

GBM



GBM

Resíduo (Y - Ŷ)

	X ₁	X ₂	X ₃	Y (altura cm)
1				190
2				185
3				170
4				165
5				175
6				159

Resíduo ₁	Resíduo ₂	Resíduo ₃
16	14,4	13,2
11	9,9	8,7
-4	-3,35	-3
-9	-8,35	-7,5
1	0,9	0,8
-15	-13,5	-12,2



2 passos em direção a resposta

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

GBM



```
gb = GradientBoostingClassifier()

parametros = {
    'max_depth' : [1, 2, 10],
    'min_samples_leaf': [1, 2, 5],
    'n_estimators': list(range(100, 1001, 150)),
    'learning_rate': [0.01, 0.5, 0.1],
}
```

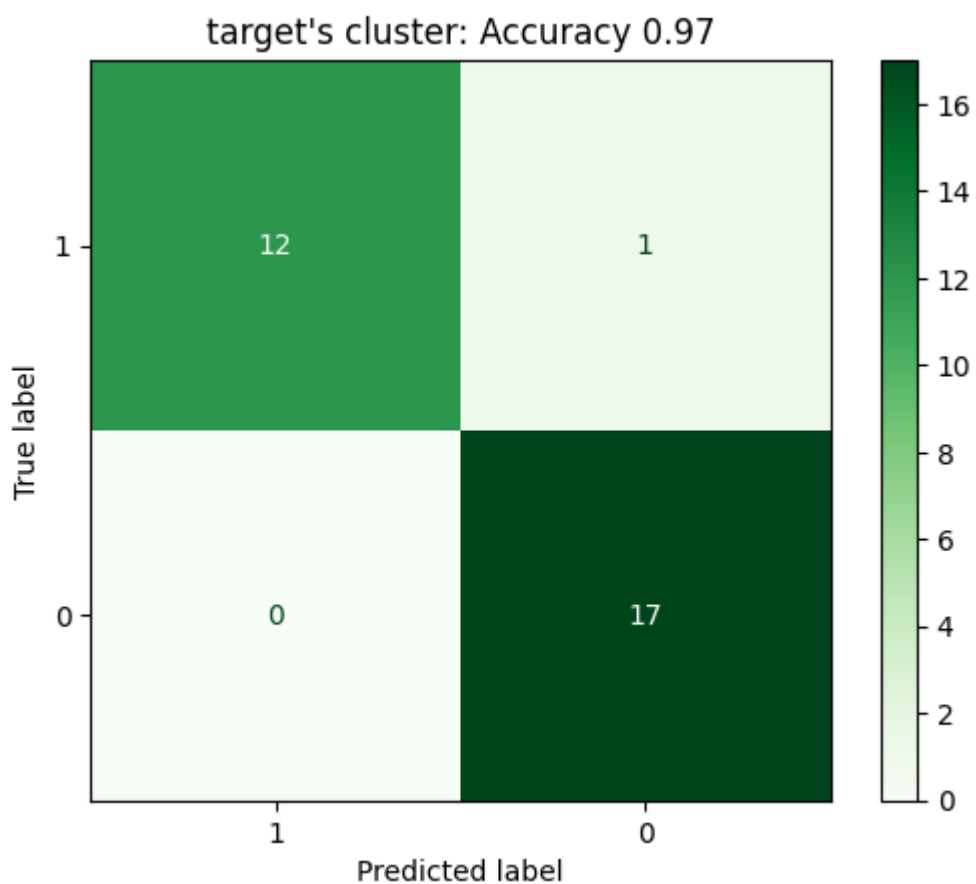
```
grid = GridSearchCV(
    estimator=gb,
    param_grid=parametros,
    scoring='roc_auc',
    verbose=False,
    cv=2
).fit(X_train, y_train)
```

```
grid.best_params_
```

```
{'learning_rate': 0.01,
 'max_depth': 10,
 'min_samples_leaf': 2,
 'n_estimators': 100}
```

GBM

```
{'learning_rate': 0.01,  
 'max_depth': 10,  
 'min_samples_leaf': 2,  
 'n_estimators': 100}
```



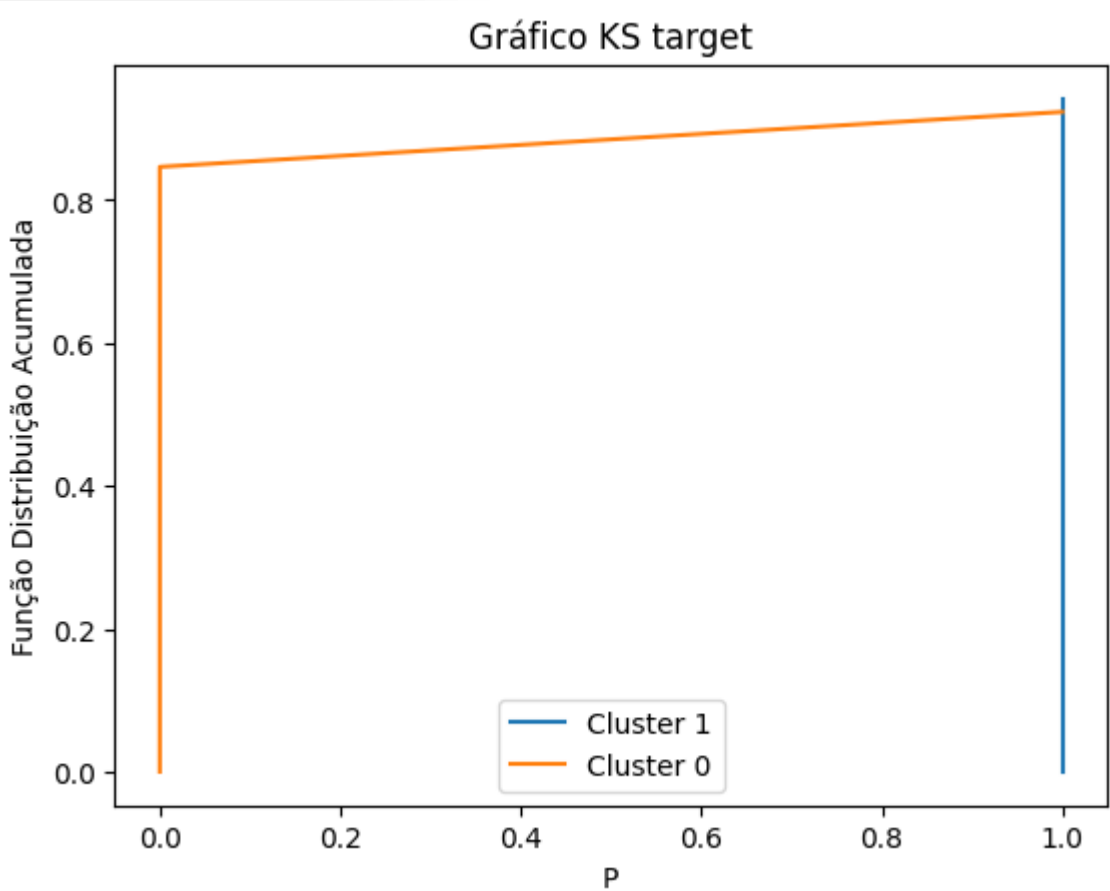
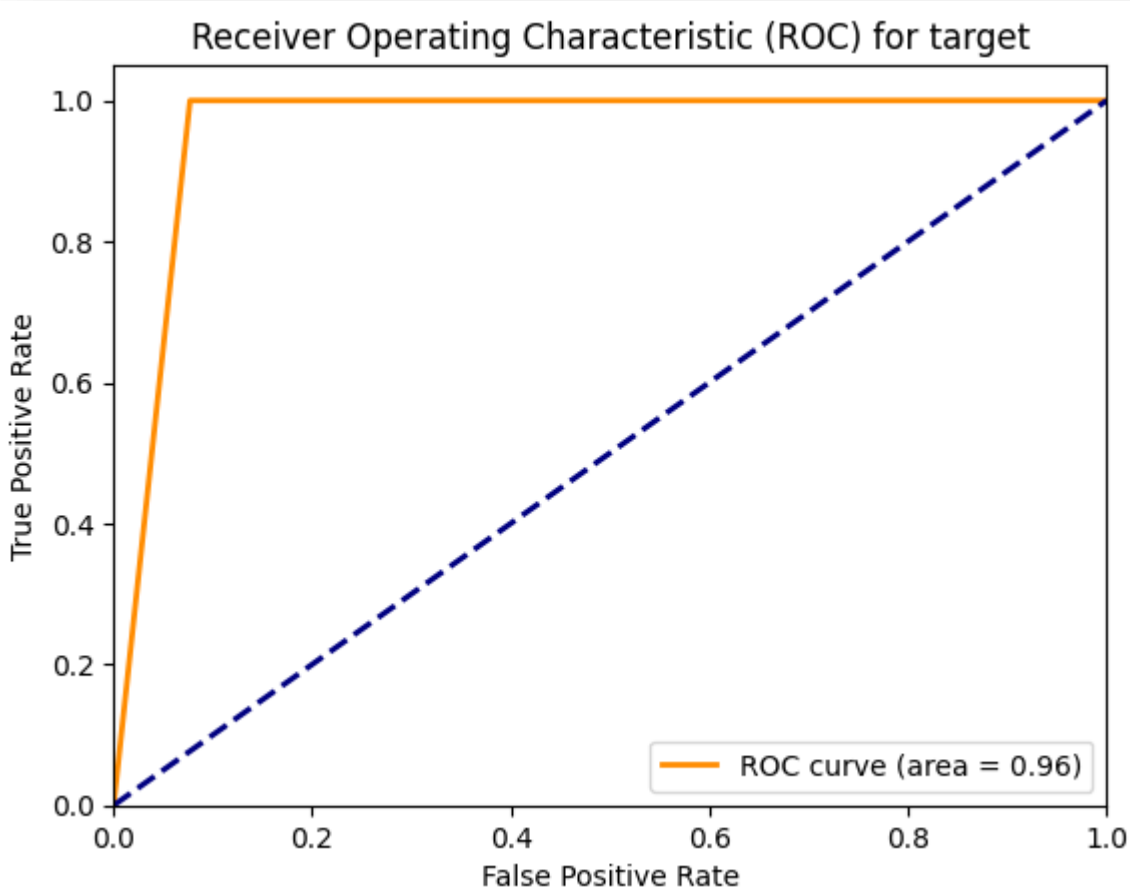
Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.92	0.96	13
1	0.94	1.00	0.97	17
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

GBM

```
RandomForestClassifier(max_depth=1)  
0.8857142857142857  
0.8857142857142857
```



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.9667	0.9444	1.0	0.9714	0.9231	0.9615	0.9231



GBM



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873
Dec Tree	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
Random Forests	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
GBM	0,9667	0,9444	1,0000	0,9714	0,9231	0,9615	0,9231

Naive Bayes



O **Naive Bayes** (em português, "Bayes Ingênuo") é um algoritmo de aprendizado de máquina supervisionado, simples, porém eficaz, utilizado para problemas de **classificação**. É um classificador probabilístico baseado no **Teorema de Bayes** e opera sob a premissa "**ingênua**" de que todas as características (ou features) são condicionalmente independentes umas das outras, dado o rótulo da classe.

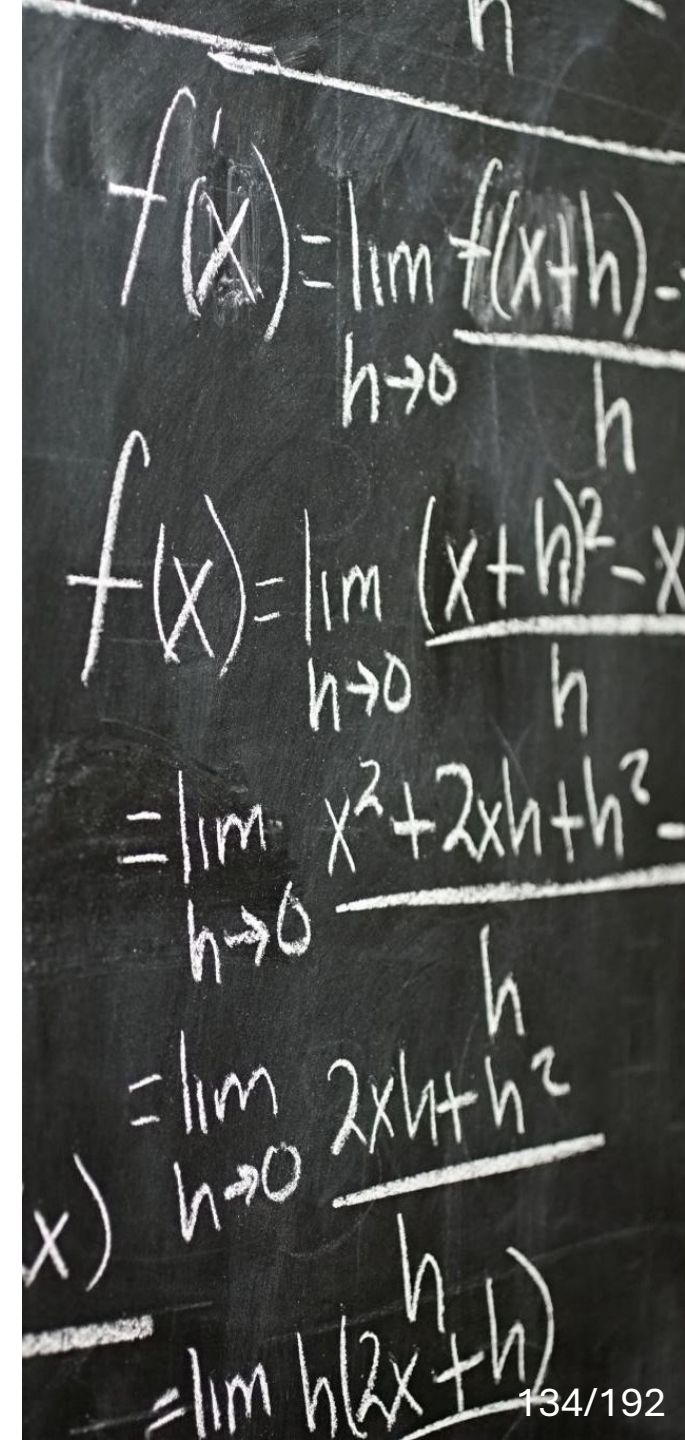
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Teorema de Bayes



O algoritmo funciona calculando a probabilidade de um determinado **evento** com base no conhecimento **prévio**, essencialmente encontrando a classe mais provável para um novo ponto de dados. A fórmula central, derivada do Teorema de Bayes, é:

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$



Teorema de Bayes

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$



$P(H|E)$: A probabilidade posterior (a probabilidade de uma classe dadas as características observadas) - isso é o que o modelo prevê.

$P(E|H)$: A verossimilhança (a probabilidade de observar as características dada a classe).

$P(H)$: A probabilidade a priori (a probabilidade inicial da classe antes que qualquer evidência seja considerada).

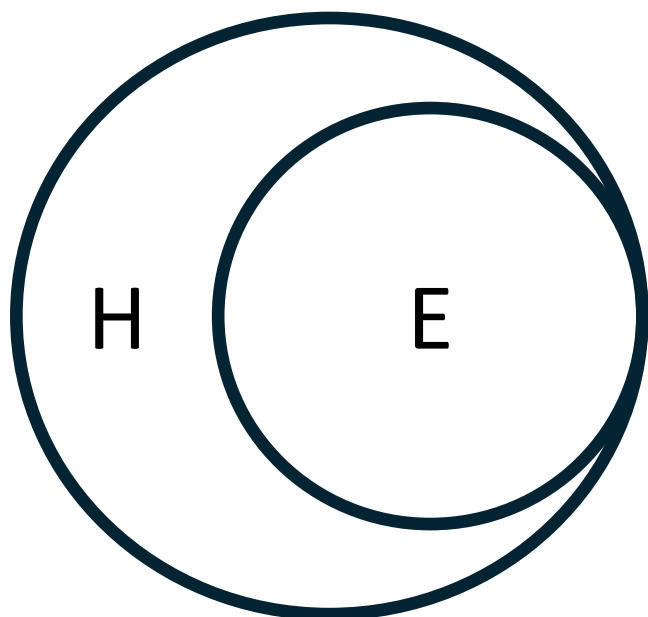
$P(E)$: A probabilidade marginal (a probabilidade da evidência em si) - este é um valor constante para todas as classes e frequentemente omitido na regra de decisão final.

Teorema de Bayes

Probabilidade de positivo no teste = 90%

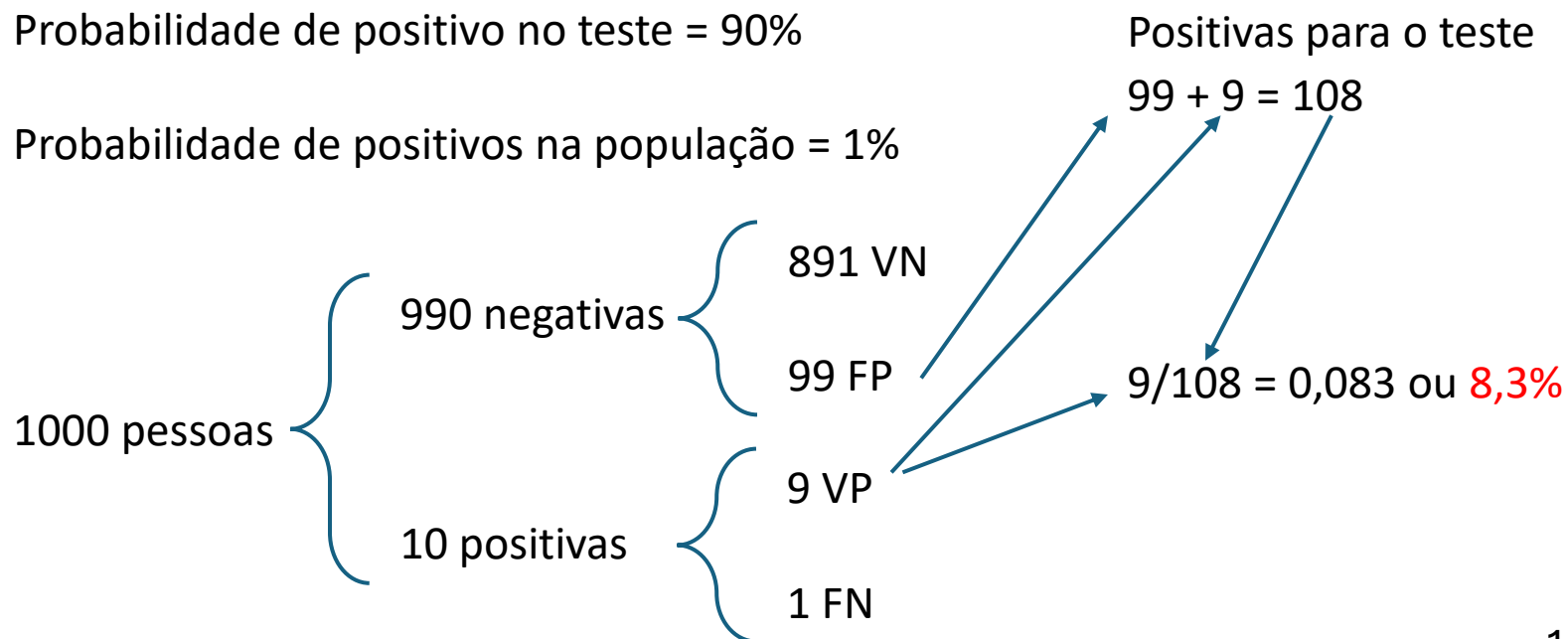
Probabilidade de positivos na população = 1%

Para um teste positivo, que tem uma acurácia de 90%, qual a probabilidade de estar certo?



Probabilidade de positivo no teste = 90%

Probabilidade de positivos na população = 1%



Teorema de Bayes

Probabilidade de positivo no teste = 90%

Probabilidade de positivos na população = 1%

$P(H|E)$:

Qual a probabilidade (P) de estar certo (H), dado (|) que o teste foi positivo (E)

$P(E|H)$ = acuraria = 90%:

Qual a probabilidade (P) do teste for positivo (E), dado (|) que o teste está certo (H)

$P(H) = 1\%$

Probabilidade de positivos na população

$P(E) = P(E|H) \times P(H) + P(E|\sim H) \times P(\sim H) = 90\% \times 1\% + 10\% \times 99\%$

Probabilidade de ser positivo

Probabilidade do evento, se a hipótese for verdadeira ($P(E|H)$), multiplicado pela probabilidade da hipótese for verdadeira $P(H)$ somado a

probabilidade do evento, se a hipótese for falsa ($P(E|\sim H)$), multiplicado pela probabilidade da hipótese for falsa $P(\sim H)$

Para um teste positivo, que tem uma acurácia de 90%, qual a probabilidade de estar certo?



$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$

Teorema de Bayes

Probabilidade de positivo no teste = 90%

Probabilidade de positivos na população = 1%

$$P(H|E) = \frac{P(E|H) \times P(H)}{P(E)}$$

$$P(H|E) = \frac{P(E|H) \times P(H)}{P(E|H) \times P(H) + P(E|\sim H) \times P(\sim H)}$$

$$P(H|E) = \frac{90\% \times 1\%}{90\% \times 1\% + 10\% \times 99\%}$$

$$P(H|E) = \frac{0,9 \times 0,01}{0,9 \times 0,01 + 0,1 \times 0,99}$$

$$P(H|E) = \frac{0,009}{0,009 + 0,099} = \frac{0,009}{0,108} = 0,083 \text{ ou } 8,3\%$$

Para um teste positivo, que tem uma acurácia de 90%, qual a probabilidade de estar certo?



Likelihood of the Evidence given that the Hypothesis is True

Prior Probability of the Hypothesis

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$

Posterior Probability of the Hypothesis given that the Evidence is True

Prior Probability that the evidence is True

Naive Bayes



GaussianNB: Para dados contínuos, assume uma distribuição gaussiana (normal) para as características.

MultinomialNB: Para dados de contagem (por exemplo, classificação de texto com contagem de palavras), adequado para características discretas que representam contagens.

BernoulliNB: Para características binárias ou booleanas, adequado para características que estão presentes ou ausentes.

A vertical strip on the right side of the slide showing a handwritten mathematical derivation on a chalkboard. The derivation calculates the derivative of f(x) = x^2 using the limit definition. The steps are: f'(x) = lim_{h to 0} (f(x+h) - f(x)) / h, then f(x) = lim_{h to 0} ((x+h)^2 - x^2) / h, which simplifies to lim_{h to 0} (x^2 + 2xh + h^2 - x^2) / h, then lim_{h to 0} (2xh + h^2) / h, and finally lim_{h to 0} h(2x + h) / h, resulting in 2x.
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Naive Bayes – hiperparâmetros



alpha:

É o parâmetro de suavização (*smoothing*) para os modelos **MultinomialNB** e **BernoulliNB**, e também pode ser utilizado no **ComplementNB**. Ele adiciona um valor a todas as contagens de frequência para evitar o problema de "**frequência zero**" (quando uma categoria não é vista durante o treinamento, o modelo atribui probabilidade zero, o que pode inviabilizar a previsão). Um valor comum é $\alpha=1.0$, que é a suavização de Laplace. Valores menores (e.g., entre 0.0 e 1.0) podem ser testados para encontrar o melhor ajuste para o seu conjunto de dados.

fit_prior:

Um hiperparâmetro booleano (True/False) que controla se o algoritmo deve aprender as **probabilidades de classe anteriores** (prior probabilities) a partir dos dados de treinamento ou se deve assumir **probabilidades de classe uniformes** (iguais para todas as classes). O padrão é True, o que geralmente funciona bem.

Naive Bayes



```
nb = BernoulliNB()
```

```
parametros = {  
    'alpha' : [1.0, 0.5, 0.1, 0.01],  
    'fit_prior': [True, False],  
}
```

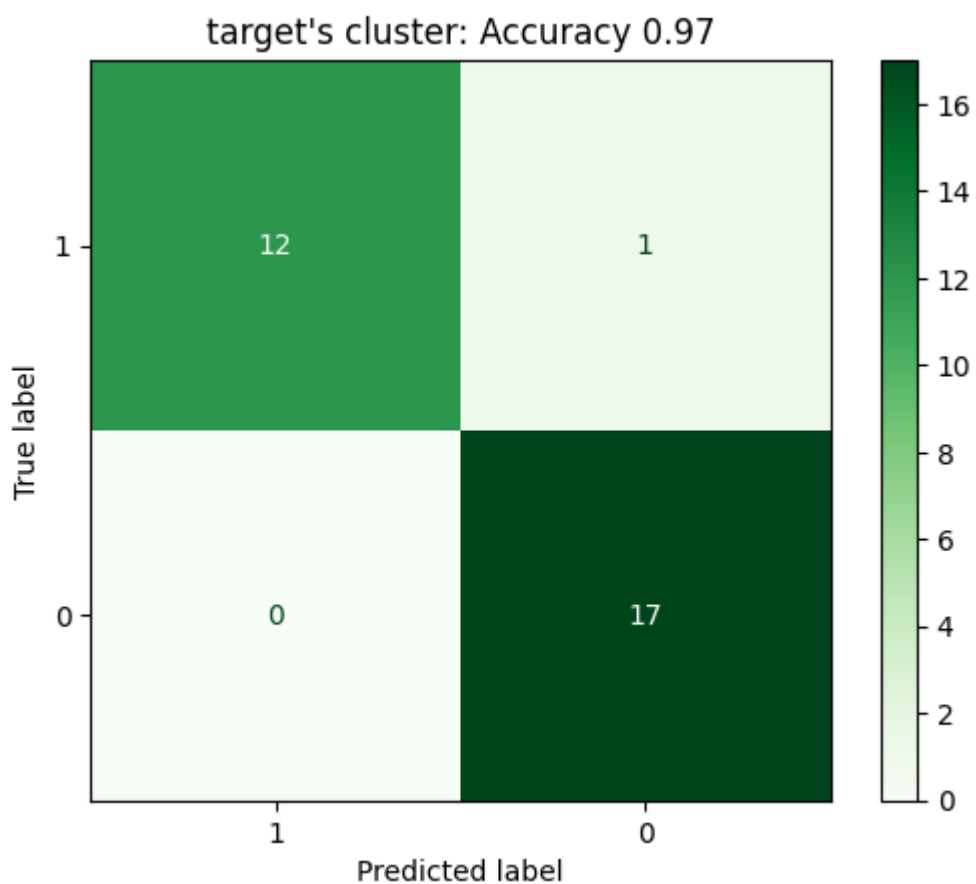
```
grid = GridSearchCV(  
    estimator=nb,  
    param_grid=parametros,  
    scoring='roc_auc',  
    verbose=False,  
    cv=2  
).fit(X_train, y_train)
```

```
grid.best_params_
```

```
{'alpha': 1.0, 'fit_prior': True}
```

Naive Bayes

```
{'alpha': 1.0, 'fit_prior': True}
```



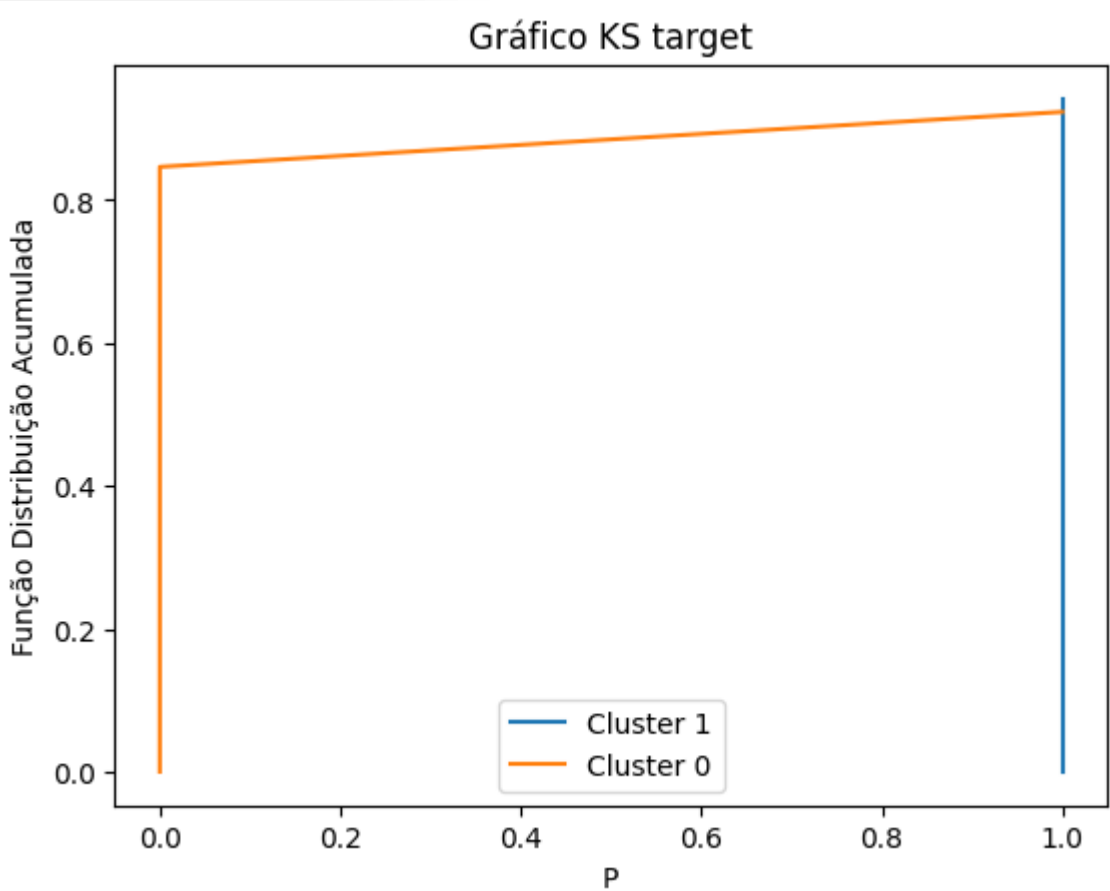
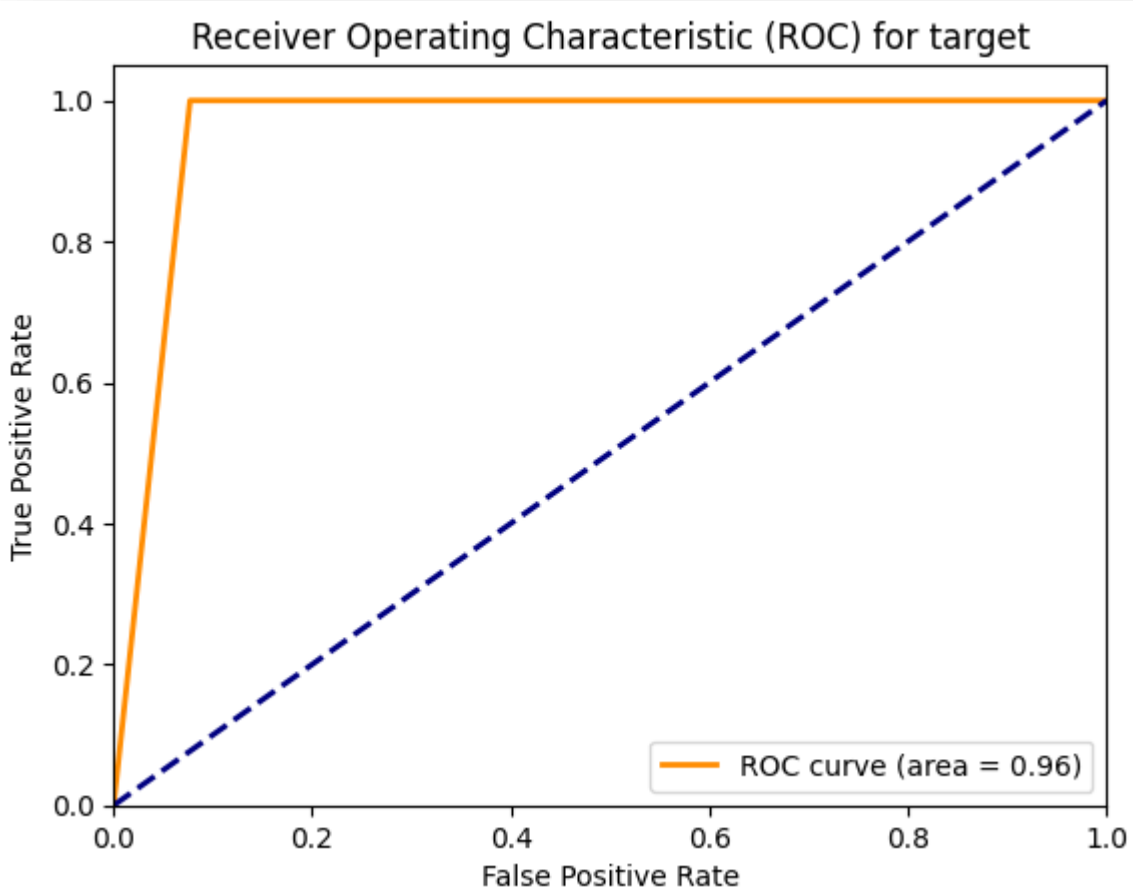
Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.92	0.96	13
1	0.94	1.00	0.97	17
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

Naive Bayes

```
RandomForestClassifier(max_depth=1)  
0.8857142857142857  
0.8857142857142857
```



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.9667	0.9444	1.0	0.9714	0.9231	0.9615	0.9231



Naive Bayes



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873
Dec Tree	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
Random Forests	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
GBM	0,9667	0,9444	1,0000	0,9714	0,9231	0,9615	0,9231
Naive Bayes	0,9667	0,9444	1,0000	0,9714	0,9231	0,9615	0,9231

Redes Neurais



Nessa variação de **Redes Neurais**, o que é modificado é apenas o output, que retorna diretamente as **classes** de interesse.

```
Model: "sequential"
```

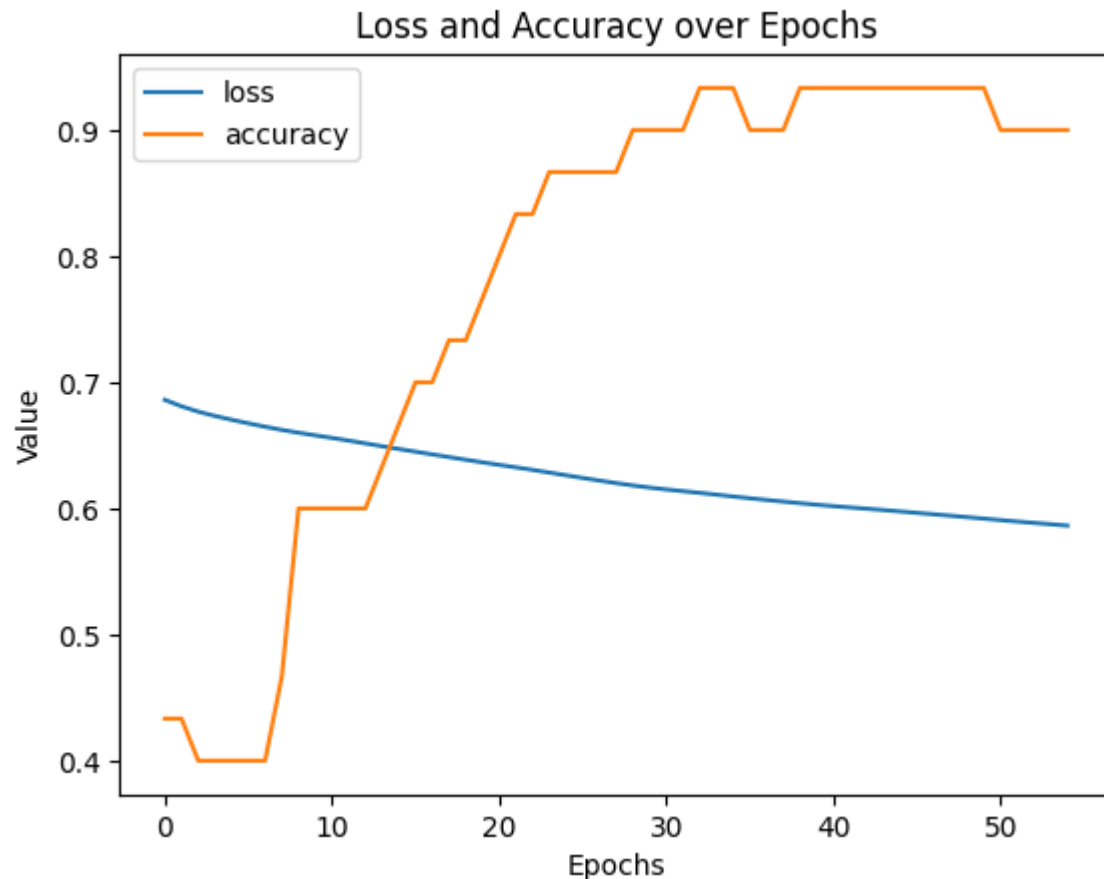
Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 7)	0
dense (Dense)	(None, 64)	512
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 1)	33

```
=====
Total params: 2,625
Trainable params: 2,625
Non-trainable params: 0
=====
```

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Redes Neurais

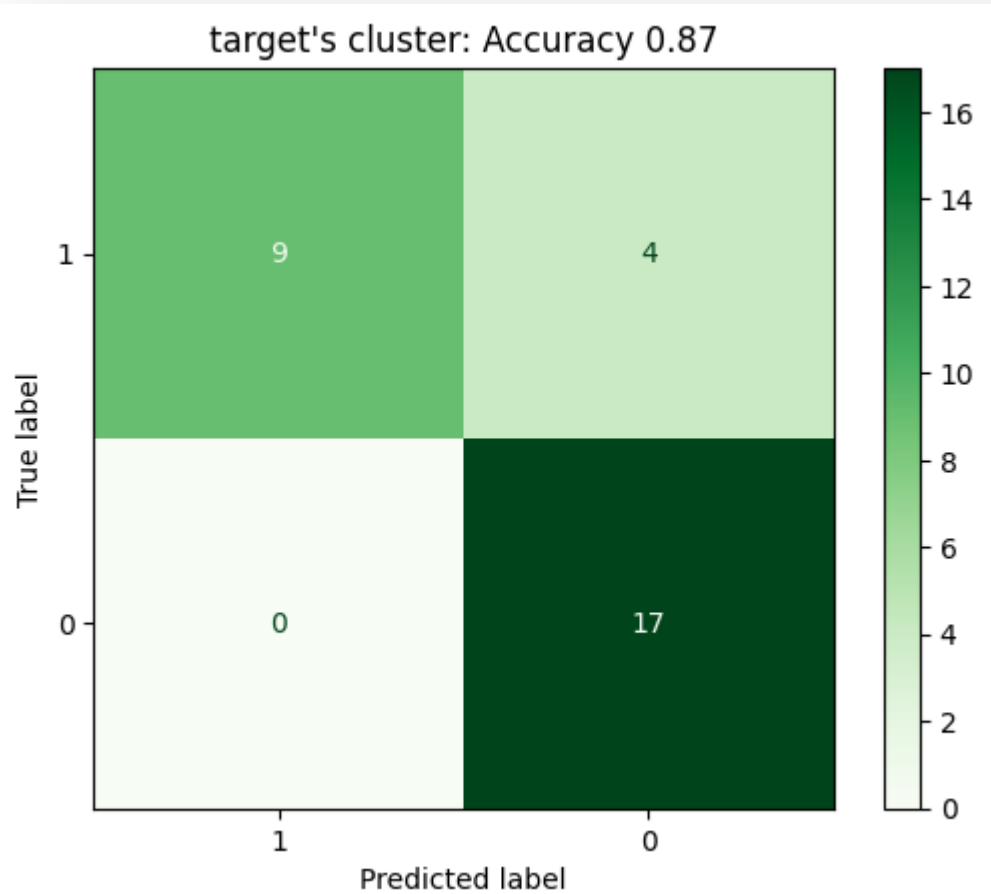
```
model.fit(  
    X_train,  
    y_train,  
    epochs=55,  
    batch_size=32,  
    verbose=0  
)
```



```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(X_train.shape[1],)),  
    keras.layers.Dense(64, activation='relu'),  
    keras.layers.Dense(32, activation='softmax'),  
    keras.layers.Dense(1, activation='sigmoid'),  
)  
  
opt = keras.optimizers.Adam(learning_rate=0.001)  
model.compile(  
    optimizer=opt,  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)  
  
model.summary()
```

Redes Neurais

```
{'alpha': 1.0, 'fit_prior': True}
```



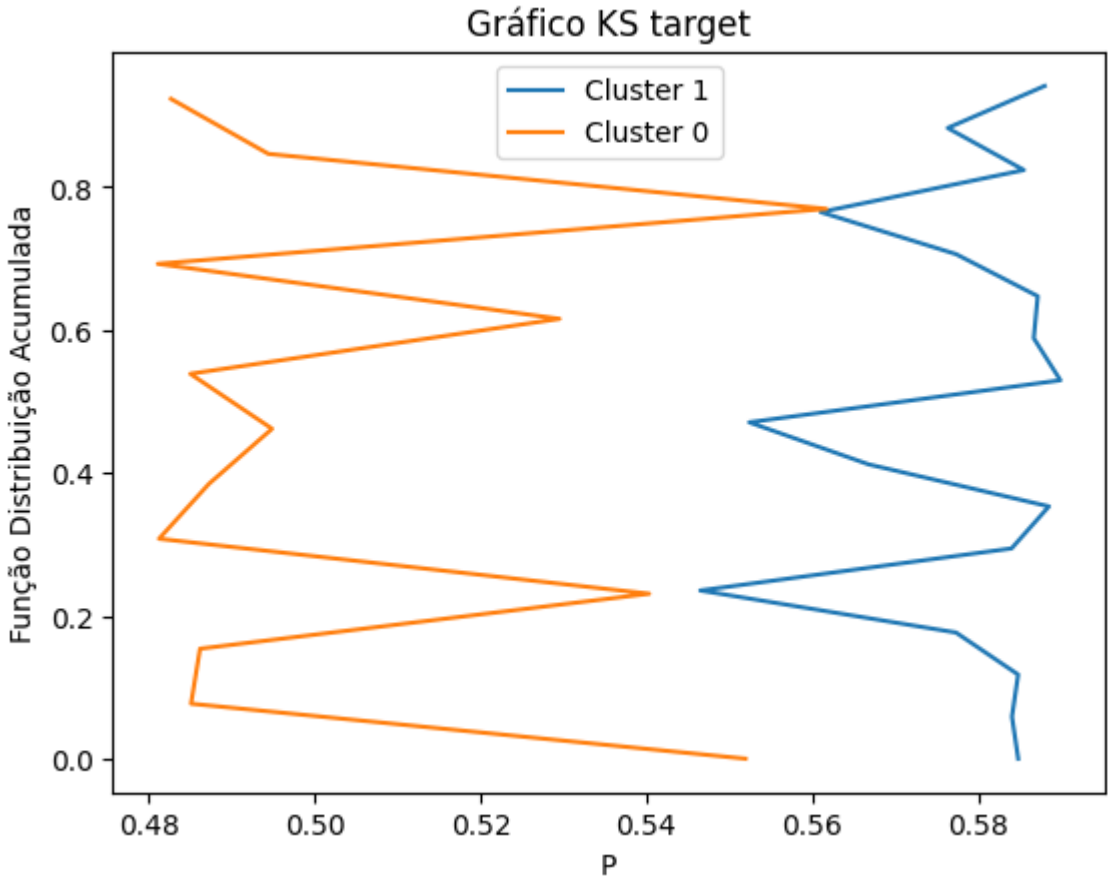
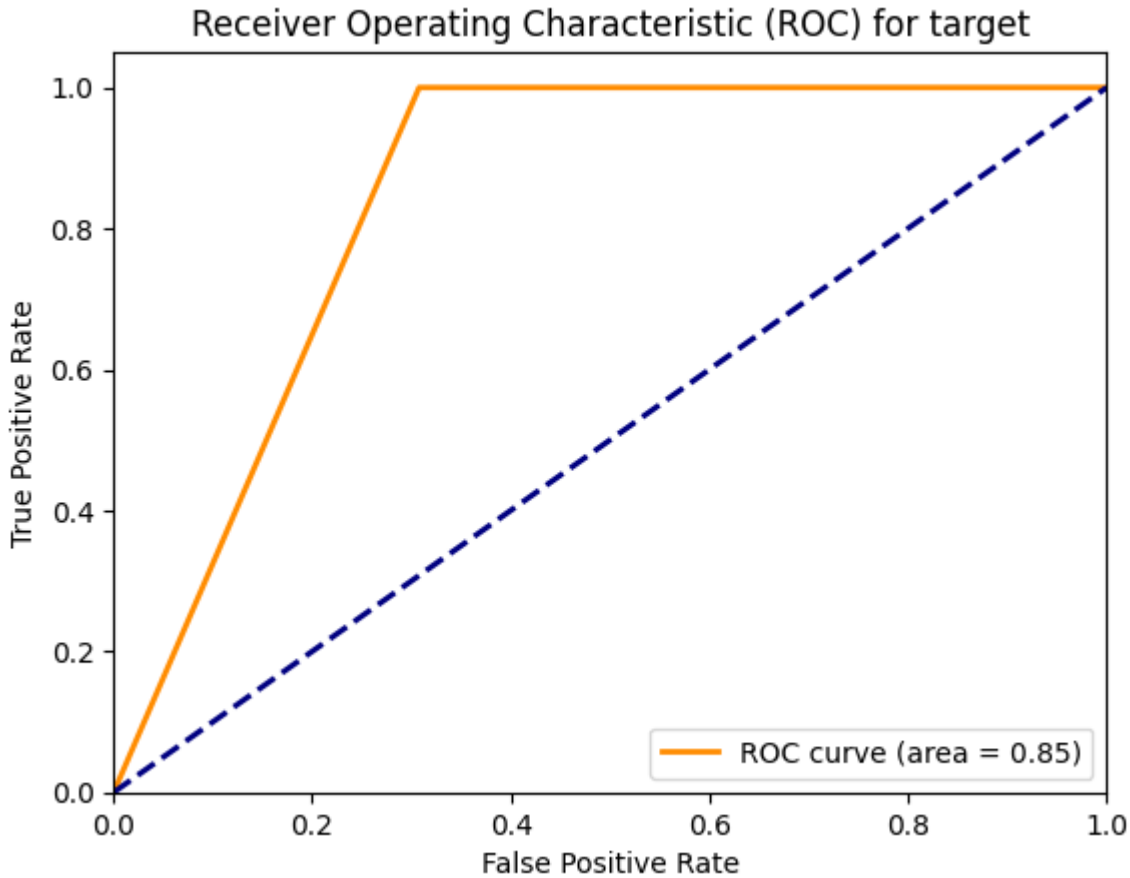
Classification Report:				
	precision	recall	f1-score	support
0	1.00	0.69	0.82	13
1	0.81	1.00	0.89	17
accuracy			0.87	30
macro avg	0.90	0.85	0.86	30
weighted avg	0.89	0.87	0.86	30

Redes Neurais

```
RandomForestClassifier(max_depth=1)
0.8857142857142857
0.8857142857142857
```



Acurácia	Precision	Recall	F1-Score	KS	AUC	GINI
0.8667	0.8095	1.0	0.8947	0.6923	0.8462	0.6923

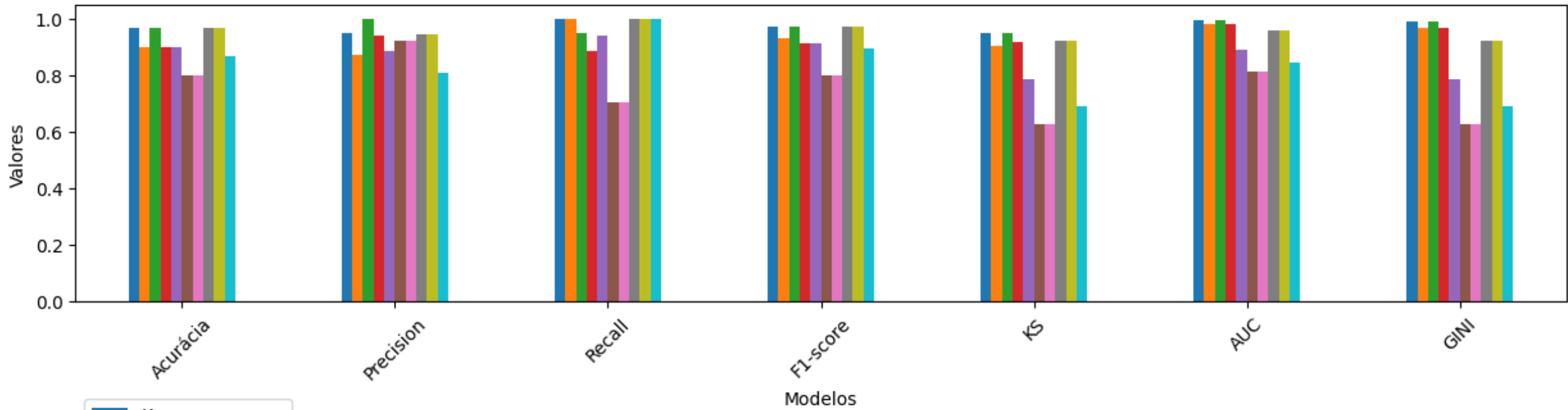


Redes Neurais



Modelo	Acurácia	Precision	Recall	F1-score	KS	AUC	GINI
K-means	0,9667	0,9524	1,0000	0,9756	0,9500	0,9950	0,9900
DBSCAN	0,9000	0,8750	1,0000	0,9333	0,9048	0,9841	0,9683
NN-k-means	0,9667	1,0000	0,9500	0,9744	0,9500	0,9950	0,9900
K-NN	0,9000	0,9412	0,8889	0,9143	0,9167	0,9838	0,9676
SVM	0,9000	0,8889	0,9412	0,9143	0,7873	0,8937	0,7873
Dec Tree	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
Random Forests	0,8000	0,9231	0,7059	0,8000	0,6290	0,8145	0,6290
GBM	0,9667	0,9444	1,0000	0,9714	0,9231	0,9615	0,9231
Naive Bayes	0,9667	0,9444	1,0000	0,9714	0,9231	0,9615	0,9231
Redes Neurais	0,8667	0,8095	1,0000	0,8947	0,6923	0,8462	0,6923

Métricas de Classificação

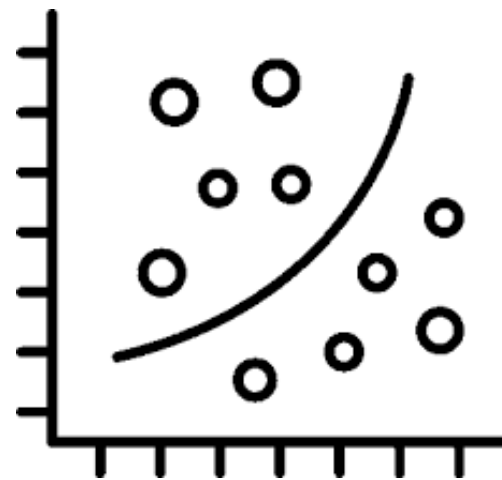




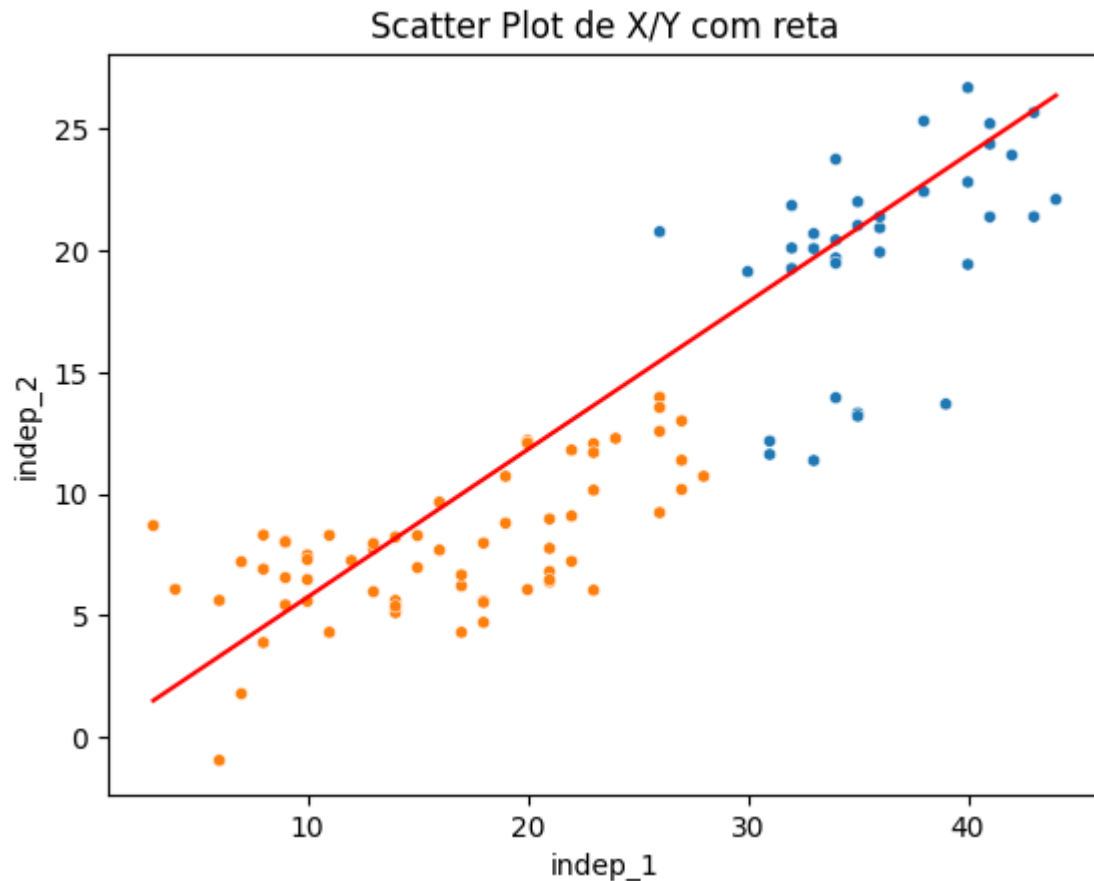
Vamos
praticar?

LIMC-IA

Regressão

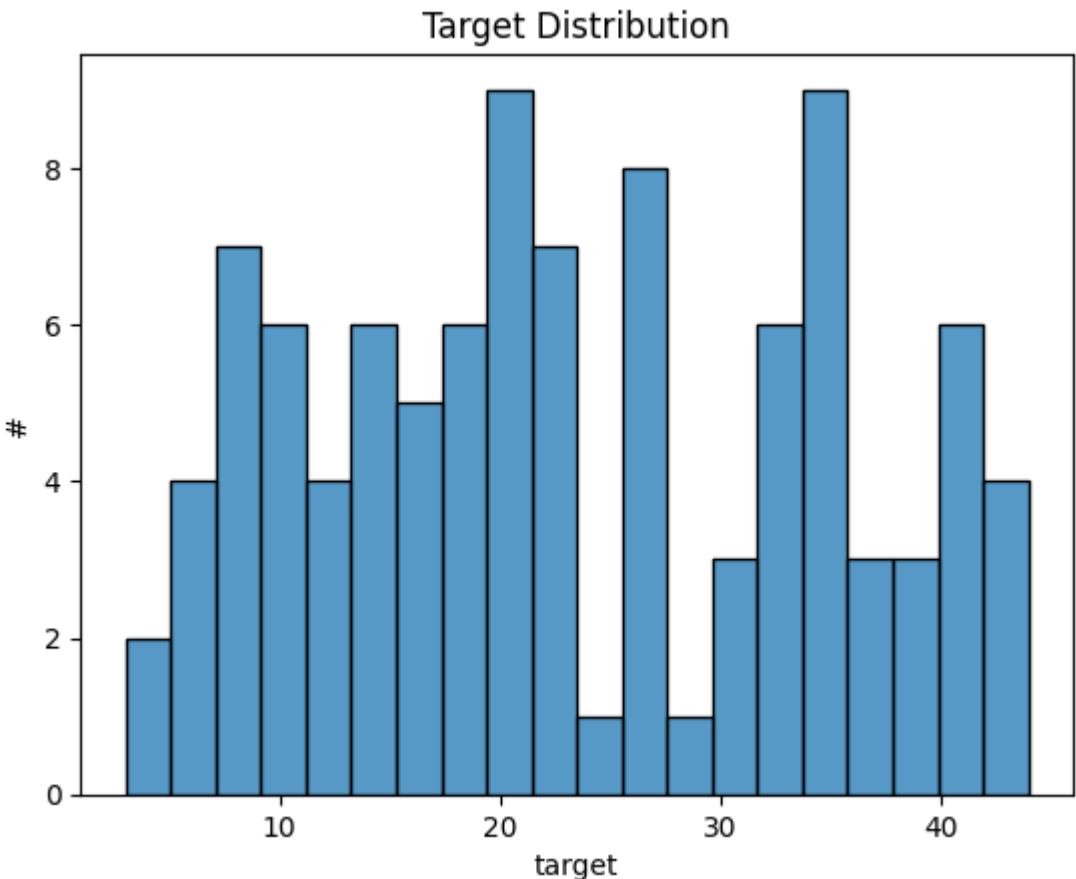


Dados – Categorização (50%)



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Dados – contínua



target
65.36
17.05
55.92
20.73
23.21

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão



Linear Simples;

Linear Multipla;

SVM;

Naive Bayes;

Neural Networks;

A photograph of a chalkboard showing the handwritten derivation of the derivative of $f(x) = x^2$ using the limit definition. The steps are as follows:
1. $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
2. $f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$
3. $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$
4. $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$
5. $= \lim_{h \rightarrow 0} h(2x + h)$
The final result is $f'(x) = 2x$.

$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
 $f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$
 $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$
 $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$
 $= \lim_{h \rightarrow 0} h(2x + h)$

Regressão Linear Simples



A **regressão linear simples** é um modelo estatístico que descreve a relação linear entre duas variáveis: uma variável **independente** (X) e uma variável **dependente** (Y). O objetivo é usar os dados para encontrar a linha reta que melhor se ajusta ao **conjunto de pontos**, permitindo prever valores de Y com base em X usando a fórmula da reta:

$$Y = mX + b.$$

$$Y = \beta_0 + \beta_1 X + e$$

Variável resposta Intercepto Coeficiente angular Variável explicativa Erro

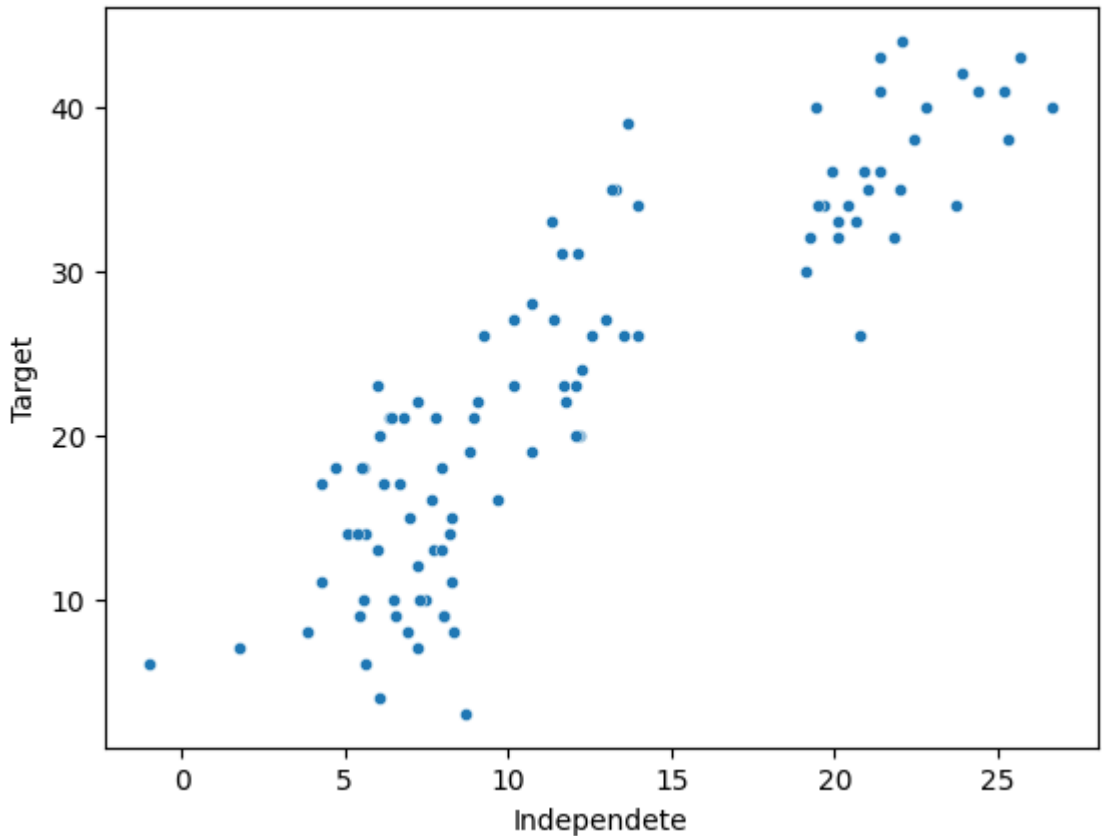
$$Y_i = \alpha + \beta X_i + \varepsilon_i$$

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Dados – contínua



Variável independete 1 vs Target



indep_2	target
24.36	41
8.05	9
19.92	36
7.73	13
6.21	17

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão Linear Simples

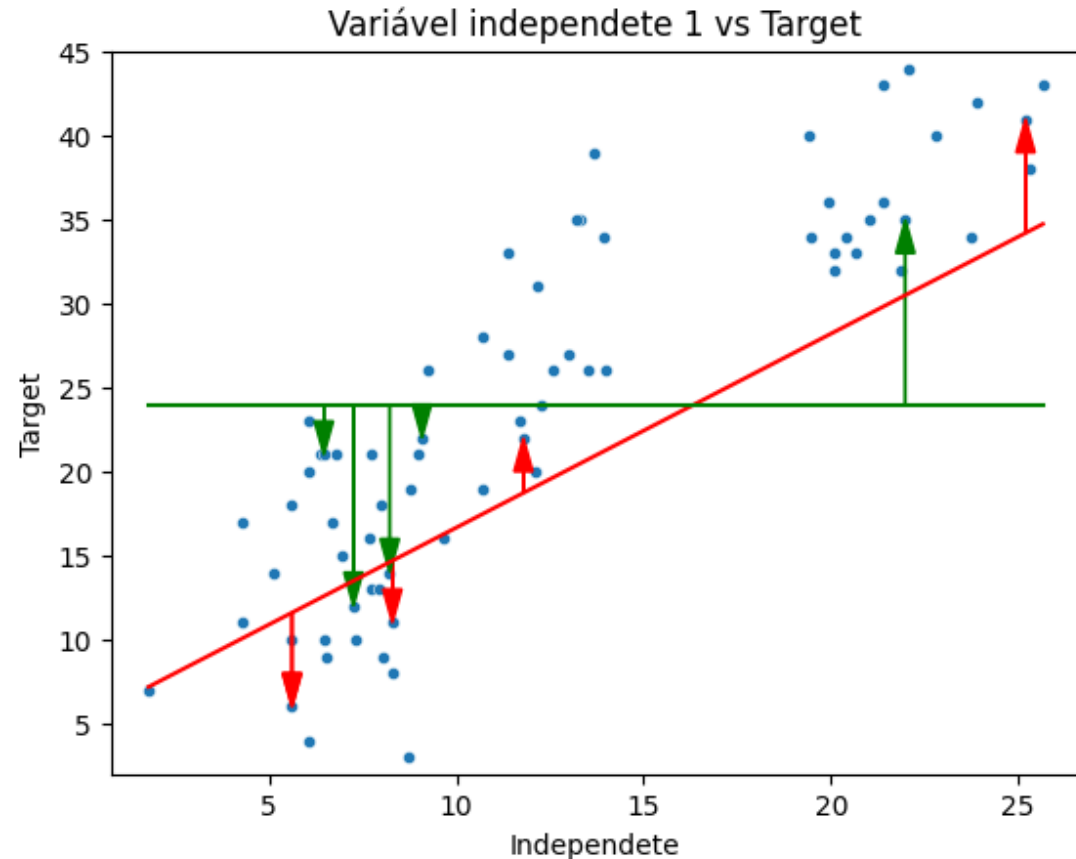


OLS Regression Results						
Dep. Variable:		target		R-squared:	0.753	
Model:		OLS		Adj. R-squared:	0.749	
Method:		Least Squares		F-statistic:	207.4	
Date:		Mon, 24 Nov 2025		Prob (F-statistic):	2.47e-22	
Time:		11:10:50		Log-Likelihood:	-218.34	
No. Observations:		70		AIC:	440.7	
Df Residuals:		68		BIC:	445.2	
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.5752	1.438	3.877	0.000	2.706	8.445
indep_2	1.4859	0.103	14.400	0.000	1.280	1.692
Omnibus:		0.089	Durbin-Watson:		1.707	
Prob(Omnibus):		0.956	Jarque-Bera (JB):		0.015	
Skew:		-0.027	Prob(JB):		0.993	
Kurtosis:		2.954	Cond. No.		30.3	

Handwritten mathematical derivation of the derivative of $f(x) = x^2$ using the limit definition:

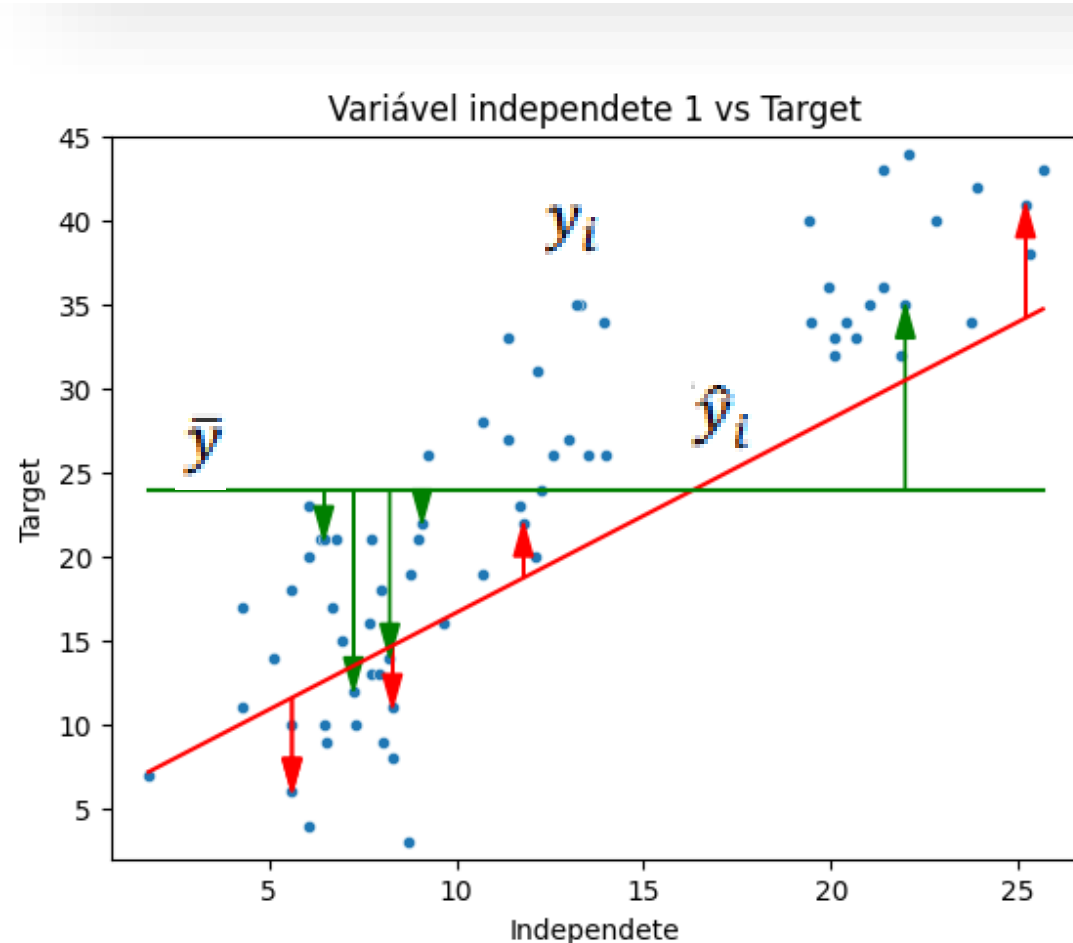
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão – Métricas



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Regressão – Métricas



$\bar{y}$ média ↑↓ resíduos

y_i observado $\hat{y}_i$ predito

$$SQT = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$SQM = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

$$SQE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$SQT = SQM + SQE$$

R^2 – coeficiente de determinação

$$R^2 = \frac{SQM}{SQT} = 1 - \frac{SQE}{SQT}$$

Métrica	Quando Utilizar	Limitações	Considerações Adicionais
Coeficiente de Determinação (R^2)	- Modelos lineares para avaliar o grau de ajuste. - Comparar diferentes modelos.	- Pode dar resultados enganadores em presença de outliers. - Não lida bem com relações não-lineares.	Útil como métrica inicial, mas não indica o erro absoluto ou a escala do erro.
Erro Médio Absoluto (MAE)	- Contextos onde é necessária uma medida intuitiva do erro médio. - Quando se quer evitar penalizar grandes erros.	- Não captura o impacto dos grandes desvios. - Insensível à magnitude do erro.	Simples de interpretar, ideal para conjuntos de dados com poucas flutuações bruscas.
Raiz do Erro Quadrático Médio (RMSE)	- Quando se deseja dar maior peso a grandes erros. - Modelos com distribuição normal dos erros.	- Penaliza severamente grandes desvios. - Não é robusto contra outliers.	Usado em contextos onde grandes desvios são indesejáveis, como em previsões financeiras.
Erro Médio Percentual Absoluto (MAPE)	- Modelos onde a interpretação do erro percentual é mais útil. - Comparação entre diferentes escalas de séries temporais.	- Se valores reais forem zero ou próximos de zero, a métrica se torna instável.	Ideal para comparações entre séries temporais com diferentes magnitudes.

Métricas - Regressão

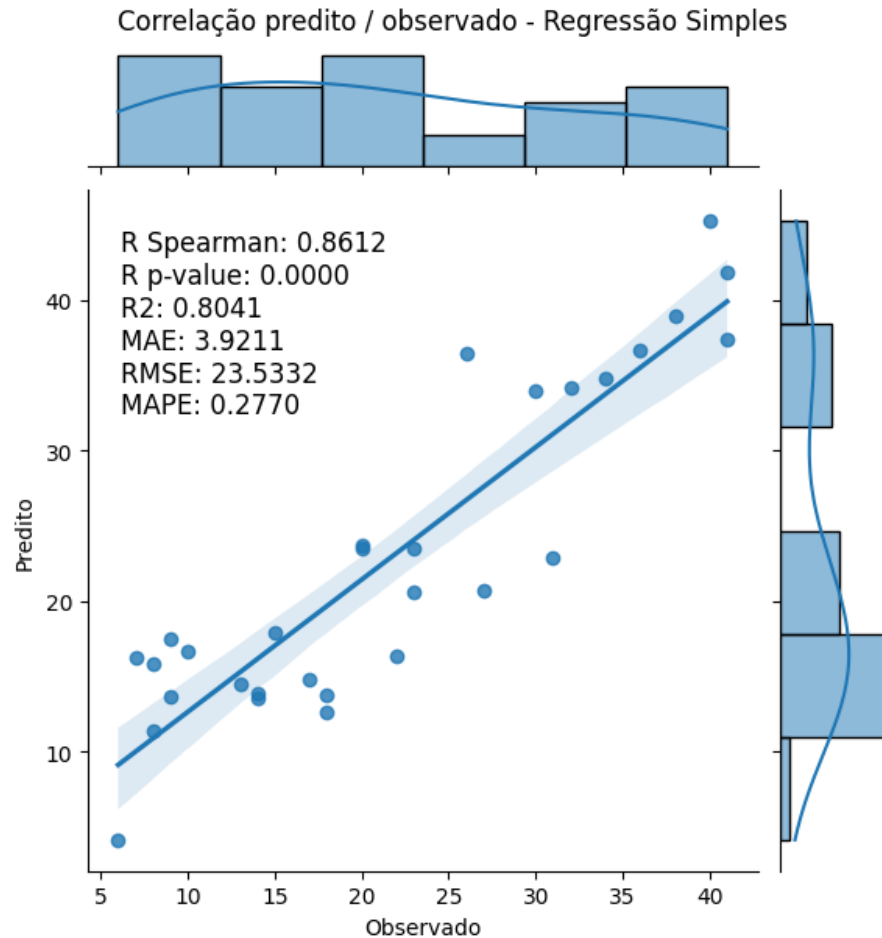
Coeficiente de correlação (r)

- **O que é:** Uma medida estatística da intensidade e direção de uma relação **linear** entre duas variáveis.
- **Intervalo:** Varia entre -1 e 1.
- **Interpretação:**
 - $r = 1$: Correlação positiva perfeita.
 - $r = -1$: Correlação negativa perfeita.
 - $r = 0$: Nenhuma correlação linear.
 - Valores próximos de 1 ou -1 indicam uma correlação forte, enquanto valores próximos de 0 indicam uma correlação fraca.
- **Sinal:** O sinal (positivo ou negativo) indica a direção da relação. Um sinal positivo significa que as variáveis se movem na mesma direção, e um negativo significa que se movem em direções opostas. 

Coeficiente de determinação (R^2)

- **O que é:** A proporção da variância da variável dependente que é explicada pelas variáveis independentes em um modelo de regressão.
- **Intervalo:** Varia entre 0 e 1 (ou 0% e 100%).
- **Cálculo:** É calculado elevando o coeficiente de correlação ao quadrado ($R^2 = r^2$).
- **Interpretação:**
 - $R^2 = 1$ (ou 100%): O modelo explica toda a variabilidade dos dados.
 - $R^2 = 0$ (ou 0%): O modelo não explica nenhuma da variabilidade dos dados.
 - Um valor alto de R^2 sugere que o modelo se ajusta bem aos dados.

Regressão – Métricas



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Regressão Linear Simples



Modelo	R Spearman Correlação	R p-value Correlação	R ² Determinação	MAE	RMSE	MAPE
Linear Simples	0,86	0,00	0,80	3,92	23,53	0,27

Regressão Linear Múltipla



A **regressão linear múltipla** é uma técnica estatística que usa **múltiplas variáveis independentes** para prever o resultado de **uma variável dependente**, modelando a relação linear entre elas. É uma extensão da **regressão linear simples**, pois considera mais de uma variável preditora para analisar seu impacto conjunto. O objetivo é encontrar a melhor combinação de variáveis independentes para prever o valor de uma variável de resposta e **entender a magnitude** e direção da influência de cada preditor.

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão Linear Múltipla



$$Y_i = \alpha + \beta X_i + \varepsilon_i$$

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Predição da Variável Dependente

Intercepto ou constante do modelo

Incremento na variável 01 (Ex: Idade)

Incremento na variável 02 (Ex: Horas)

variável 02 (Ex: Horas)

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão Linear Múltipla



OLS Regression Results						
Dep. Variable:	target		R-squared:	0.788		
Model:	OLS		Adj. R-squared:	0.779		
Method:	Least Squares		F-statistic:	88.08		
Date:	Mon, 24 Nov 2025		Prob (F-statistic):	4.19e-31		
Time:	11:10:51		Log-Likelihood:	-304.67		
No. Observations:	100		AIC:	619.3		
Df Residuals:	95		BIC:	632.4		
Df Model:	4					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.1281	1.444	3.551	0.001	2.261	7.995
indep_2	1.1540	0.212	5.433	0.000	0.732	1.576
indep_3_max	3.7028	0.909	4.072	0.000	1.888	5.528
indep_3_min	1.4253	0.881	1.617	0.109	-0.324	3.175
indep_4_min	-0.8185	1.158	-0.707	0.481	-3.118	1.481
indep_4_med	1.5137	0.755	2.004	0.048	0.014	3.013
indep_4_max	4.4329	2.371	1.870	0.065	-0.274	9.140
Omnibus:	0.436	Durbin-Watson:	1.771			
Prob(Omnibus):	0.804	Jarque-Bera (JB):	0.345			
Skew:	-0.143	Prob(JB):	0.842			
Kurtosis:	2.961	Cond. No.	1.94e+17			

Descrição dos índices obtidos no modelo

coef		Estimativa pontual do coeficiente beta para a variável
std err		Erro padrão dado o coeficiente
z		Estatística Z que é o coef/std error
P> z		Probabilidade de o coef ser zero
[0.025 0.975]		Intervalo de confiança para 95%

Regressão Linear Multipla



OLS Regression Results						
Dep. Variable:	target		R-squared:	0.788		
Model:	OLS		Adj. R-squared:	0.779		
Method:	Least Squares		F-statistic:	88.08		
Date:	Mon, 24 Nov 2025		Prob (F-statistic):	4.19e-31		
Time:	11:10:51		Log-Likelihood:	-304.67		
No. Observations:	100		AIC:	619.3		
Df Residuals:	95		BIC:	632.4		
Df Model:	4					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.1281	1.444	3.551	0.001	2.261	7.995
indep_2	1.1540	0.212	5.433	0.000	0.732	1.576
indep_3_max	3.7028	0.909	4.072	0.000	1.888	5.508
indep_3_min	1.4253	0.881	1.617	0.109	-0.324	3.175
indep_4_min	-0.8185	1.158	-0.707	0.481	-3.118	1.481
indep_4_med	1.5137	0.755	2.004	0.048	0.014	3.013
indep_4_max	4.4329	2.371	1.870	0.065	-0.274	9.140
Omnibus:	0.436	Durbin-Watson:	1.771			
Prob(Omnibus):	0.804	Jarque-Bera (JB):	0.345			
Skew:	-0.143	Prob(JB):	0.842			
Kurtosis:	2.961	Cond. No.	1.94e+17			

Descrição dos índices obtidos no modelo

R-squared		Coeficiente de determinação
AIC e BIC		Qualidade do ajuste com a complexidade do modelo
Omnibus		Testar se não segue uma distribuição normal
Jarque-Bera		Testar se não segue uma distribuição normal
Durbin-Watson		Teste para autocorrelação nos resíduos
Kurtosis		Presença de outliers

Regressão Linear Múltipla

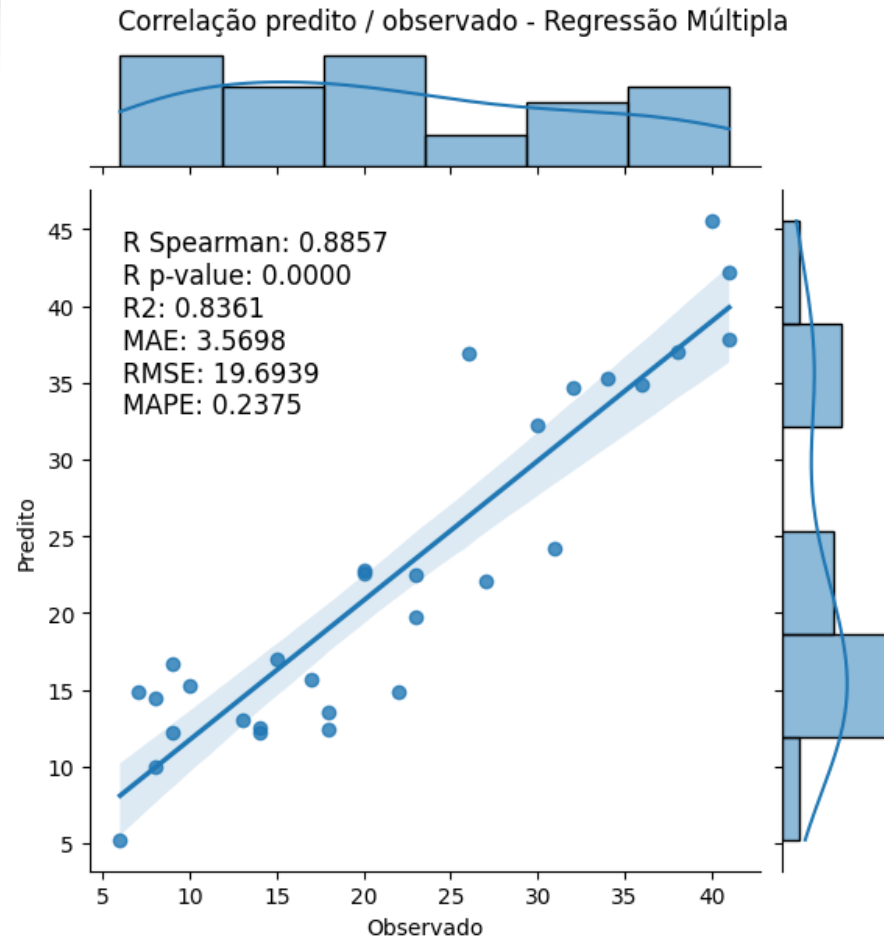


OLS Regression Results						
Dep. Variable:	target		R-squared:	0.788		
Model:	OLS		Adj. R-squared:	0.779		
Method:	Least Squares		F-statistic:	88.08		
Date:	Mon, 24 Nov 2025		Prob (F-statistic):	4.19e-31		
Time:	11:10:51		Log-Likelihood:	-304.67		
No. Observations:	100		AIC:	619.3		
Df Residuals:	95		BIC:	632.4		
Df Model:	4					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	5.1281	1.444	3.551	0.001	2.261	7.995
<u>indep_2</u>	1.1540	0.212	5.433	0.000	0.732	1.576
<u>indep_3_max</u>	3.7028	0.909	4.072	0.000	1.888	5.528
indep_3_min	1.4253	0.881	1.617	0.109	-0.324	3.175
indep_4_min	-0.8185	1.158	-0.707	0.481	-3.118	1.481
<u>indep_4_med</u>	1.5137	0.755	2.004	0.048	0.014	3.013
indep_4_max	4.4329	2.371	1.870	0.065	-0.274	9.140
Omnibus:	0.436	Durbin-Watson:		1.771		
Prob(Omnibus):	0.804	Jarque-Bera (JB):		0.345		
Skew:	-0.143	Prob(JB):		0.842		
Kurtosis:	2.961	Cond. No.		1.94e+17		

R-squared		0,788		0,782
AIC e BIC		619,3 e 632,4		619,8 e 630,3
Omnibus		0,804		0,936
Jarque-Bera		0,842		0,984
Durbin-Watson		1,771		1,785
Kurtosis		2,961		2,994

OLS Regression Results						
Dep. Variable:	target		R-squared:	0.782		
Model:	OLS		Adj. R-squared:	0.775		
Method:	Least Squares		F-statistic:	115.0		
Date:	Mon, 24 Nov 2025		Prob (F-statistic):	1.16e-31		
Time:	11:10:52		Log-Likelihood:	-305.92		
No. Observations:	100		AIC:	619.8		
Df Residuals:	96		BIC:	630.3		
Df Model:	3					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	4.3369	1.330	3.261	0.002	1.697	6.977
indep_2	1.4588	0.081	18.105	0.000	1.299	1.619
indep_3_max	2.2713	1.067	2.130	0.036	0.154	4.388
indep_4_med	0.6077	1.148	0.529	0.598	-1.672	2.887
Omnibus:	0.132	Durbin-Watson:		1.785		
Prob(Omnibus):	0.936	Jarque-Bera (JB):		0.033		
Skew:	-0.044	Prob(JB):		0.984		
Kurtosis:	2.994	Cond. No.		41.6		

Regressão – Métricas



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Regressão Linear Multipla

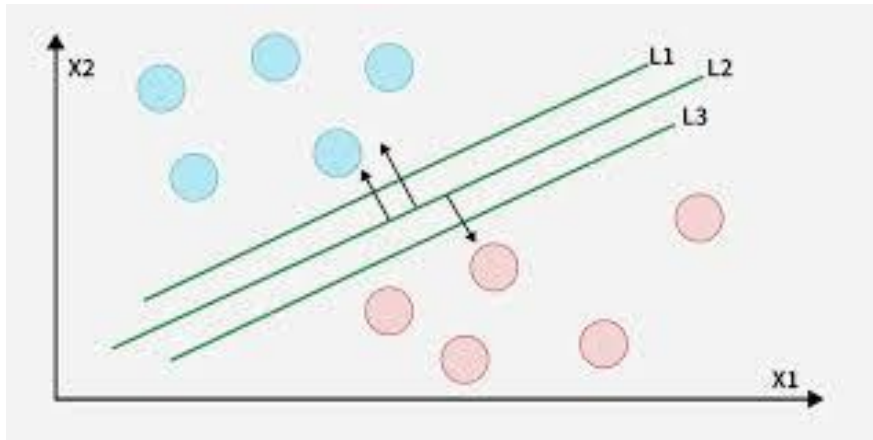


Modelo	R Spearman Correlação	R p-value Correlação	R ² Determinação	MAE	RMSE	MAPE
Linear Simples	0,86	0,00	0,80	3,92	23,53	0,27
Linear Multiplo	0,88	0,00	0,83	3,56	19,69	0,23

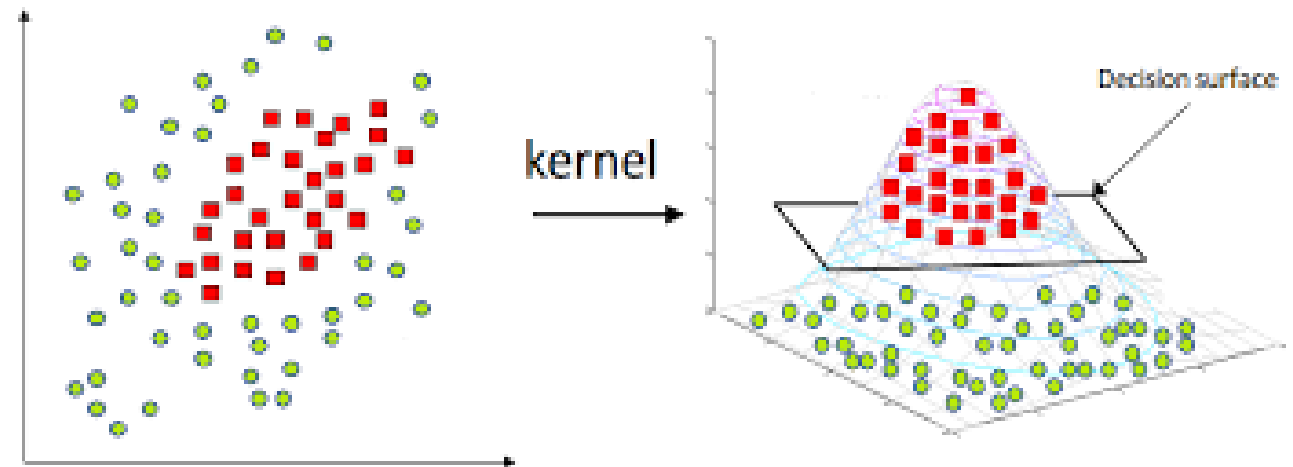
Support Vector Machine – SVM



Support Vector



Support Vector Machine



Support Vector Machine – SVM



```
kernel = ['linear', 'poly', 'rbf', 'sigmoid']
```

```
svm = SVC()
```

```
param_grid = {  
    'C': [0.1, 1, 10, 100],  
    'gamma': [0.001, 0.01, 0.1, 1],  
    'kernel': kernel,  
}
```

```
grid_svm = GridSearchCV(  
    estimator=svm,  
    param_grid=param_grid,  
    cv=2,  
    scoring='r2',  
    verbose=1,  
    n_jobs=-1  
)
```

```
grid_svm.fit(X_train, y_train)  
grid_svm.best_params_
```

```
{'C': 0.1, 'gamma': 0.001, 'kernel': 'sigmoid'}
```

C:

Parâmetro de regularização. A intensidade da regularização é inversamente proporcional a C. Deve ser estritamente positivo. A penalidade é uma penalidade l2 ao quadrado.

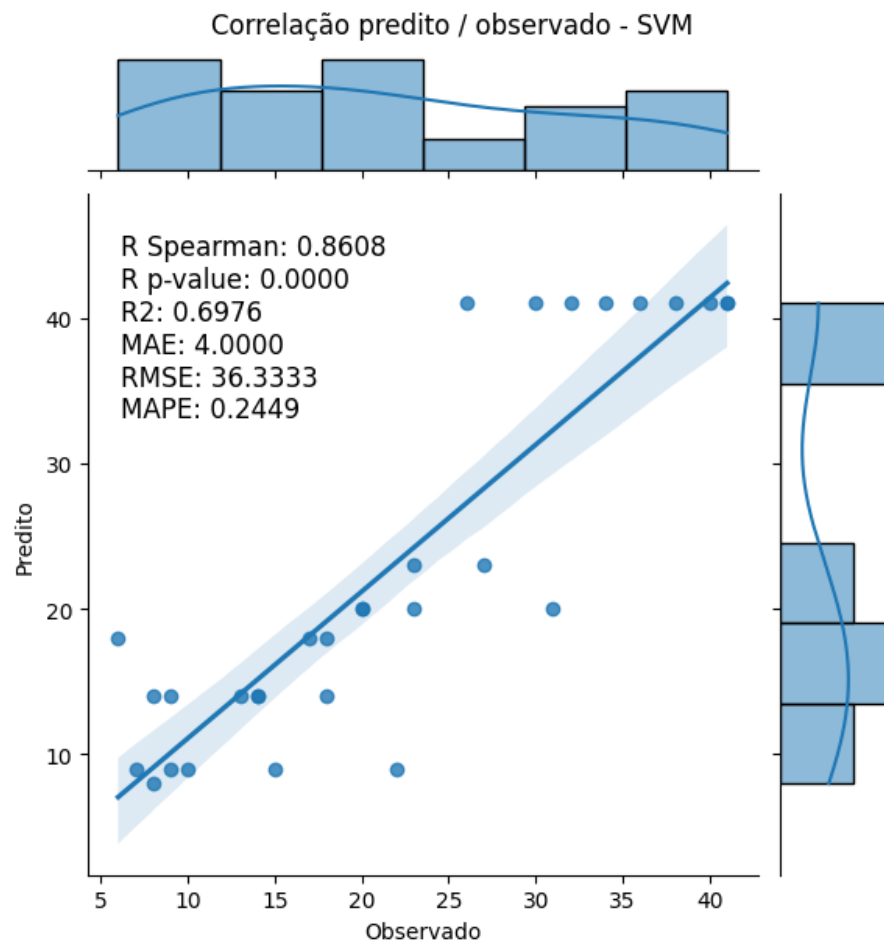
kernel:

Especifica o tipo de kernel a ser usado no algoritmo. Se nenhum for fornecido, será usado o tipo 'rbf'. Se uma função for fornecida, ela será usada para pré-calcular a matriz do kernel a partir das matrizes de dados; essa matriz deve ser um array de formato (n_samples, n_samples).

gamma:

Coeficiente kernel para 'rbf', 'poly' e 'sigmoid'.

Support Vector Machine – SVM



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Support Vector Machine – SVM



Modelo	R Spearman Correlação	R p-value Correlação	R ² Determinação	MAE	RMSE	MAPE
Linear Simples	0,86	0,00	0,80	3,92	23,53	0,27
Linear Multipla	0,88	0,00	0,83	3,56	19,69	0,23
SVM	0,86	0,00	0,69	4,00	36,33	0,24

Naive Bayes



MultinomialNB: Para dados de contagem (por exemplo, classificação de texto com contagem de palavras), adequado para características discretas que representam contagens.

$$P(H|E) = \frac{P(E|H) * P(H)}{P(E)}$$

A blackboard with handwritten mathematical derivations. The first line shows the definition of the derivative: $f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$. The second line shows the function $f(x) = x^2$ being substituted: $f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$. The third line shows the expansion of the numerator: $= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$. The fourth line shows the simplification: $= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$. The fifth line shows the final result: $= \lim_{h \rightarrow 0} h(2x + h)$.
$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Naive Bayes



```
nb = MultinomialNB()

parametros = {
    'alpha' : [1.0, 0.5, 0.1, 0.01],
    'fit_prior': [True, False],
}

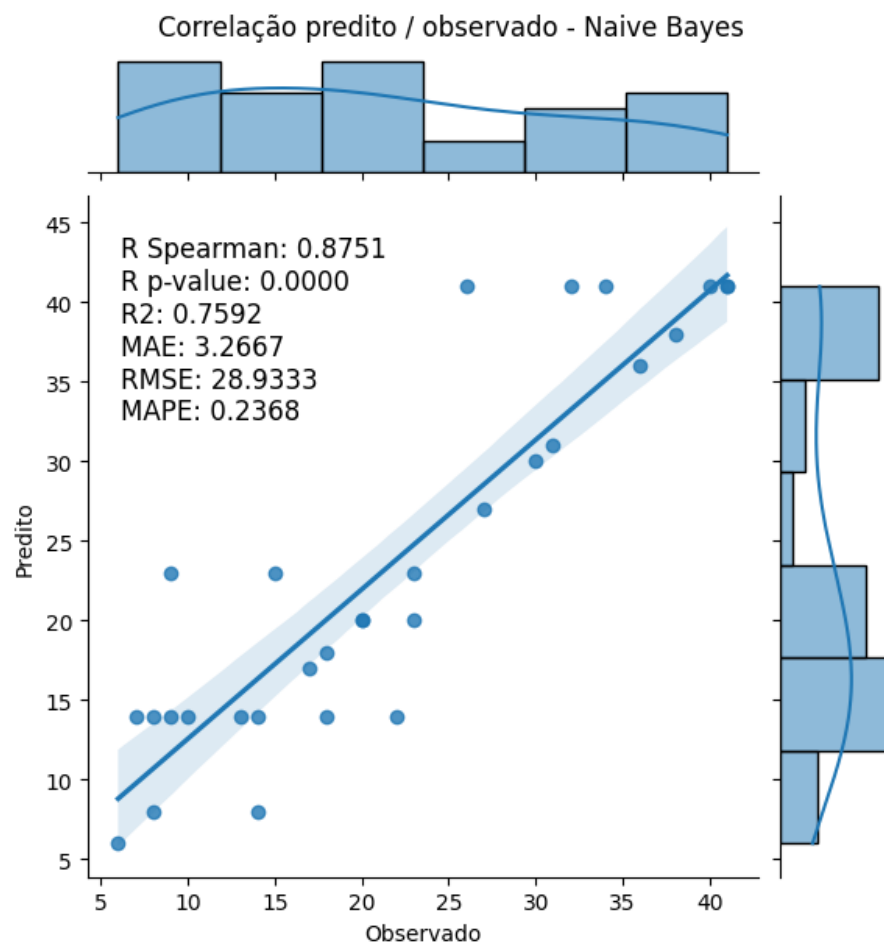
grid_nb = GridSearchCV(
    estimator=nb,
    param_grid=parametros,
    scoring='r2',
    verbose=False,
    cv=2
).fit(X_train, y_train)

grid_nb.best_params_
```

```
{'alpha': 0.01, 'fit_prior': True}
```

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Naive Bayes



$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \\ f(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} \\ &= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h} \\ &= \lim_{h \rightarrow 0} h(2x + h) \end{aligned}$$

Naive Bayes



Modelo	R Spearman Correlação	R p-value Correlação	R ² Determinação	MAE	RMSE	MAPE
Linear Simples	0,86	0,00	0,80	3,92	23,53	0,27
Linear Multipla	0,88	0,00	0,83	3,56	19,69	0,23
SVM	0,82	0,00	0,62	4,50	45,03	0,27
Naive Bayes	0,87	0,00	0,75	3,26	28,93	0,23

Redes Neurais



Nessa variação de **Redes Neurais**, o que é modificado é o output, e o compilador, que retorna diretamente as **classes** de interesse.

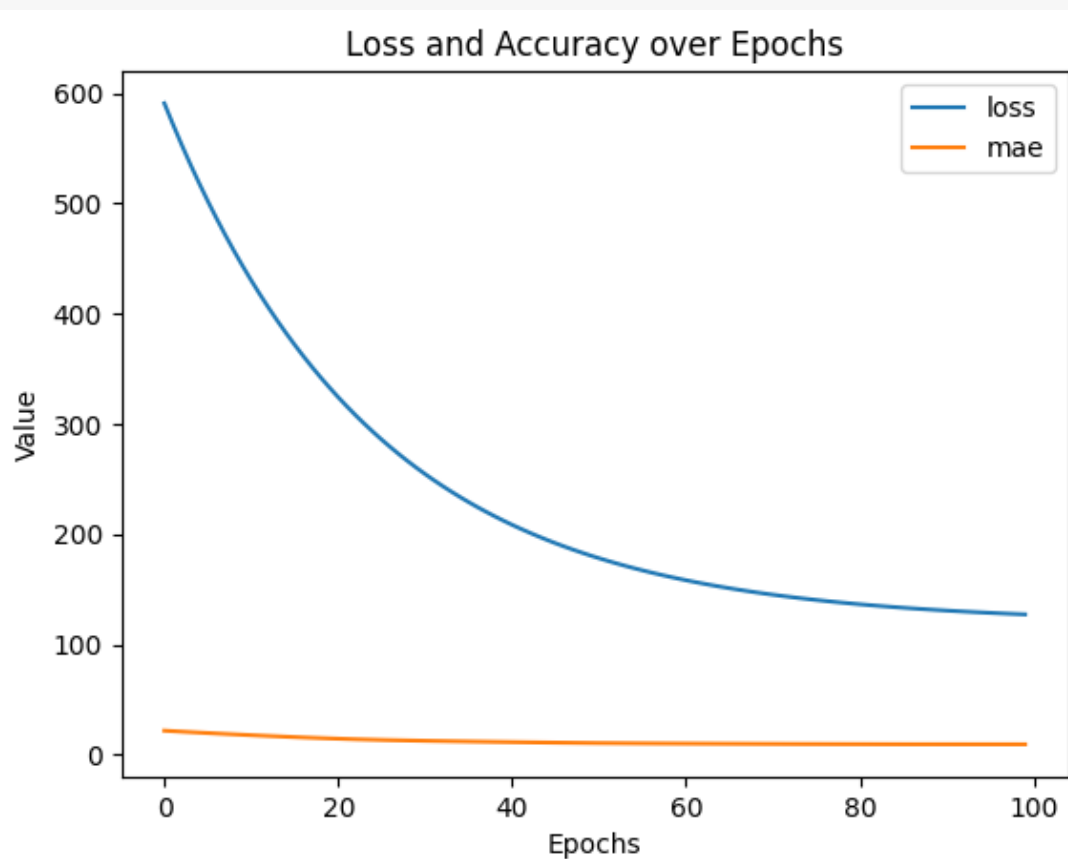
```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(X_train.shape[1],)),
3     keras.layers.Dense(64, activation='softmax'),
4     keras.layers.Dense(32, activation='softmax'),
5     keras.layers.Dense(1, activation='linear'),
6 ])
7
8 opt = keras.optimizers.Adam(learning_rate=0.001)
9 model.compile(
10     optimizer='sgd',
11     loss='mse',
12     metrics=['mae']
13 )
14
15 model.summary()
```

✓ 0.0s

Model: "sequential_2"

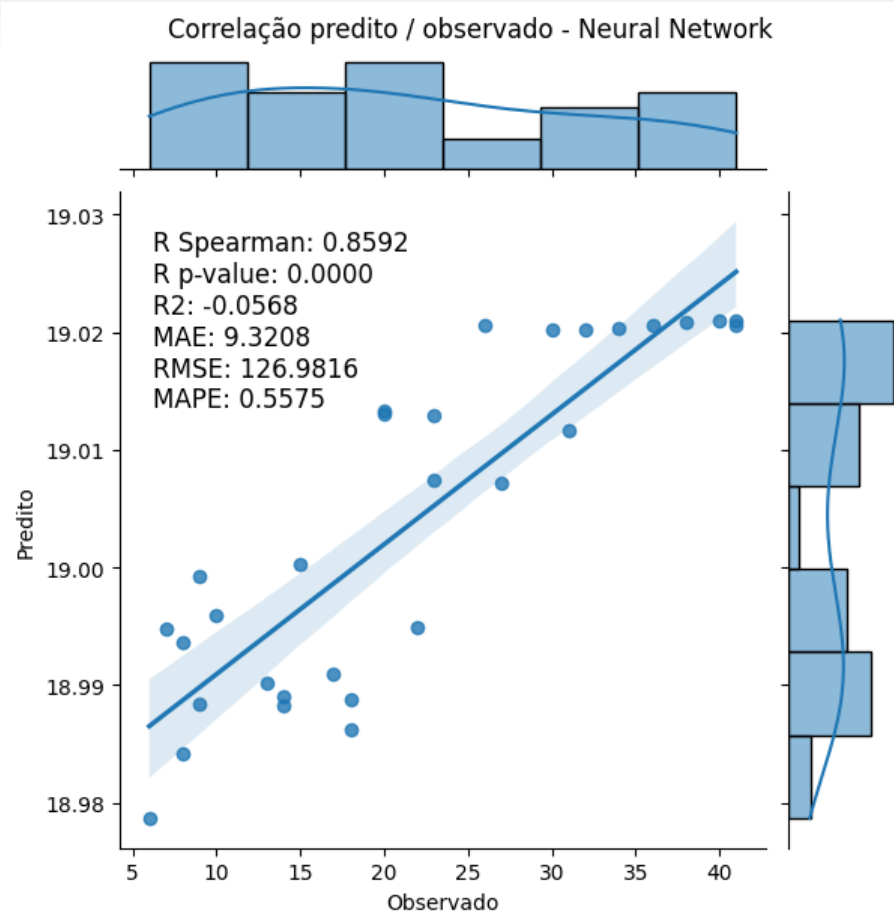
Layer (type)	Output Shape	Param #
=====		
flatten_2 (Flatten)	(None, 6)	0
dense_6 (Dense)	(None, 64)	448
dense_7 (Dense)	(None, 32)	2080
dense_8 (Dense)	(None, 1)	33
=====		
Total params: 2,561		
Trainable params: 2,561		
Non-trainable params: 0		

Redes Neurais



$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Redes Neurais

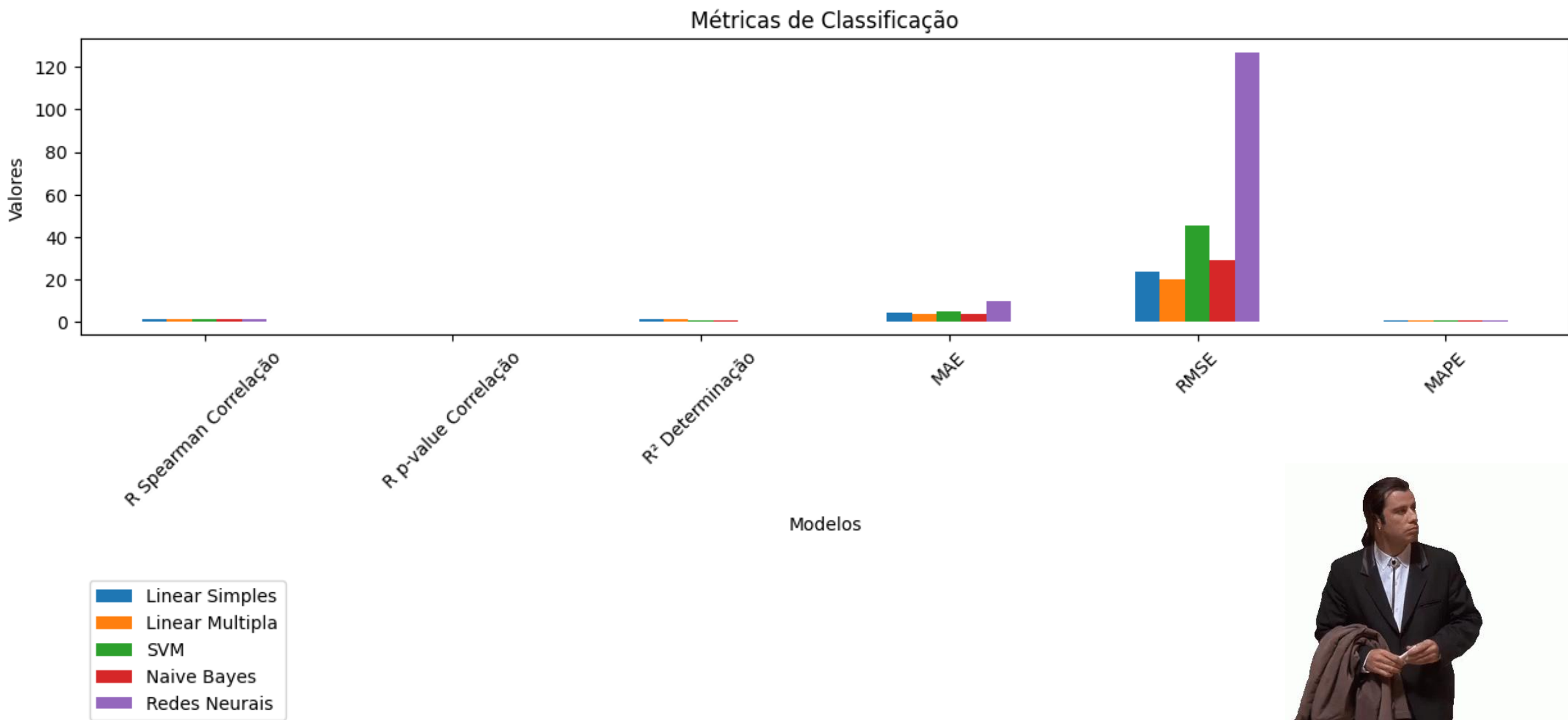


$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$
$$f(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h}$$
$$= \lim_{h \rightarrow 0} \frac{2xh + h^2}{h}$$
$$= \lim_{h \rightarrow 0} h(2x + h)$$

Naive Bayes



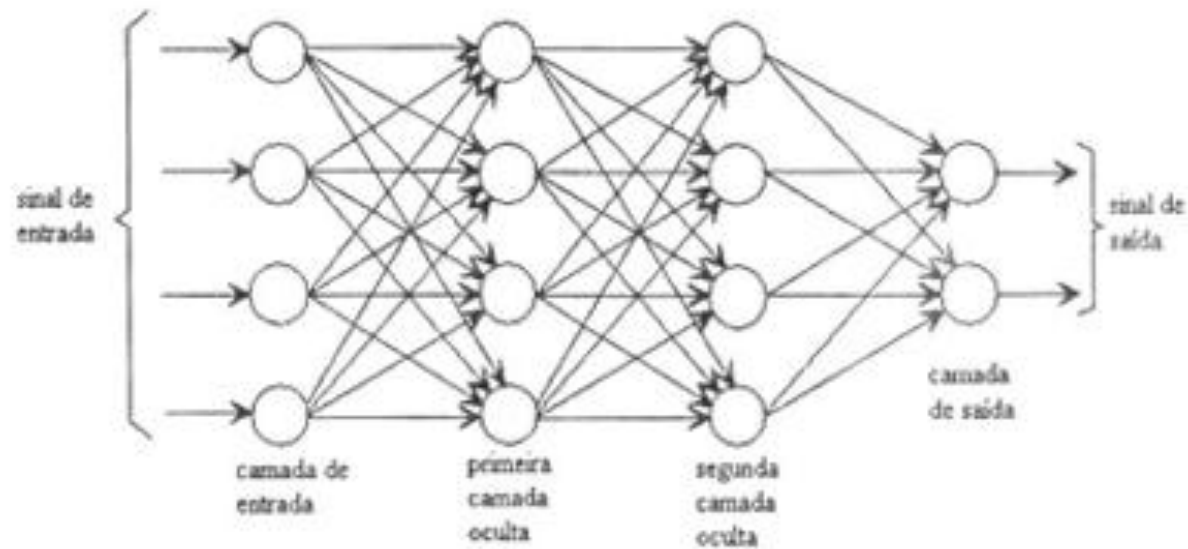
Modelo	R Spearman Correlação	R p-value Correlação	R ² Determinação	MAE	RMSE	MAPE
Linear Simples	0,86	0,00	0,80	3,92	23,53	0,27
Linear Multipla	0,88	0,00	0,83	3,56	19,69	0,23
SVM	0,82	0,00	0,62	4,50	45,03	0,27
Naive Bayes	0,87	0,00	0,75	3,26	28,93	0,23
Redes Neurais	0,85	0,00	-0,05	9,32	126,98	0,55



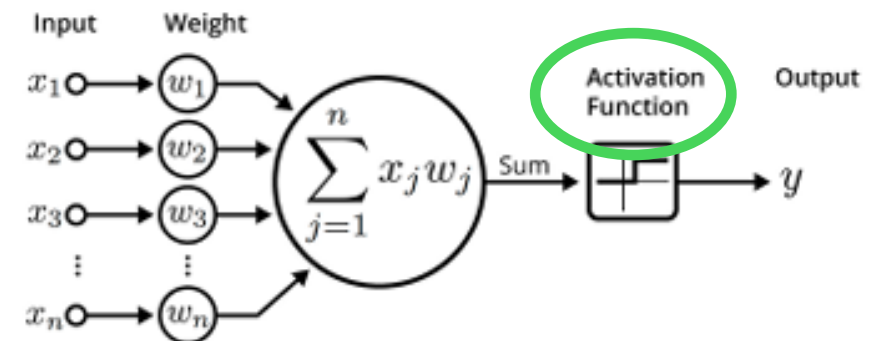
Redes Neurais – Sistema de Algoritmos



Figura 2: Exemplo de RNA defendida por Ludwig e Costa (2007)



Fonte: LUDWIG e COSTA (2007, p. 15)



Fonte: SHUBH SAXENA (2017).

Redes Neurais - hiperparâmetros



Função de ativação

Uma **função de ativação** é uma parte fundamental das redes neurais que determina se um neurônio deve ser ativado ou não, transformando a entrada ponderada em um sinal de saída.

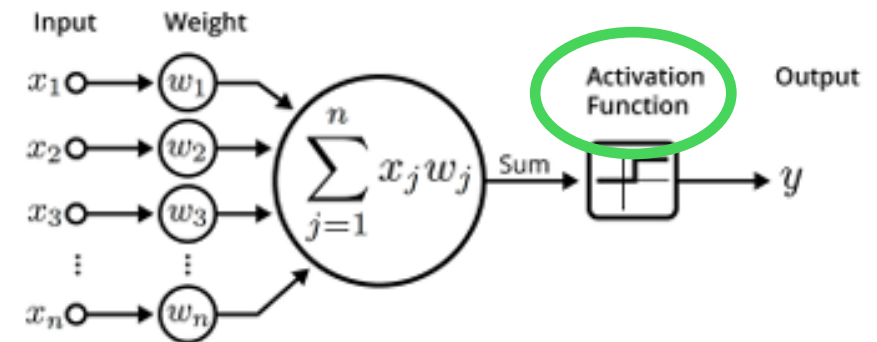
```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(X_train.shape[1],)),
3     keras.layers.Dense(64, activation='softmax'),
4     keras.layers.Dense(32, activation='softmax'),
5     keras.layers.Dense(1, activation='linear'),
6 ])
7
8 opt = keras.optimizers.Adam(learning_rate=0.001)
9 model.compile(
10     optimizer='sgd',
11     loss='mse',
12     metrics=['mae']
13 )
14
15 model.summary()
```

✓ 0.0s

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 6)	0
dense_6 (Dense)	(None, 64)	448
dense_7 (Dense)	(None, 32)	2080
dense_8 (Dense)	(None, 1)	33

Total params: 2,561
Trainable params: 2,561
Non-trainable params: 0



Fonte: SHUBH SAXENA (2017).

Redes Neurais - hiperparâmetros



Função de ativação

Uma **função de ativação** é uma parte fundamental das redes neurais que determina se um neurônio deve ser ativado ou não, transformando a entrada ponderada em um sinal de saída.

Exemplos de funções de ativação comuns

- **Sigmoide:** Uma função que mapeia valores de entrada para um intervalo entre 0 e 1, sendo útil para problemas de classificação binária. Sua expressão é $\sigma(x) = 1/(1 + e^{-x})$.
- **Tanh (Tangente Hiperbólica):** Similar à sigmoide, mas mapeia valores para o intervalo entre -1 e 1. Por ser centrada em zero, geralmente é preferível à sigmoide na prática.
- **ReLU (Unidade Linear Retificada):** Uma das mais populares. Para valores positivos, retorna o próprio valor; para valores negativos, retorna zero. Embora simples, pode causar o problema de "neurônios mortos" para valores negativos.
- **Leaky ReLU (Unidade Linear Retificada com Vazamento):** Uma variante da ReLU que aborda o problema dos "neurônios mortos" permitindo uma pequena inclinação para os valores negativos.
- **Softmax:** Comumente usada na camada de saída para problemas de classificação multiclasse, onde os valores de saída são interpretados como probabilidades. 

Redes Neurais - hiperparâmetros



Algoritmo de otimização

Um algoritmo de **otimização** em redes neurais é o método usado para ajustar os pesos e vieses da rede, minimizando a função de custo (ou perda). O objetivo final é fazer com que a rede neural produza resultados o mais próximo possível da saída desejada, tornando-a mais precisa na tarefa para a qual foi treinada.

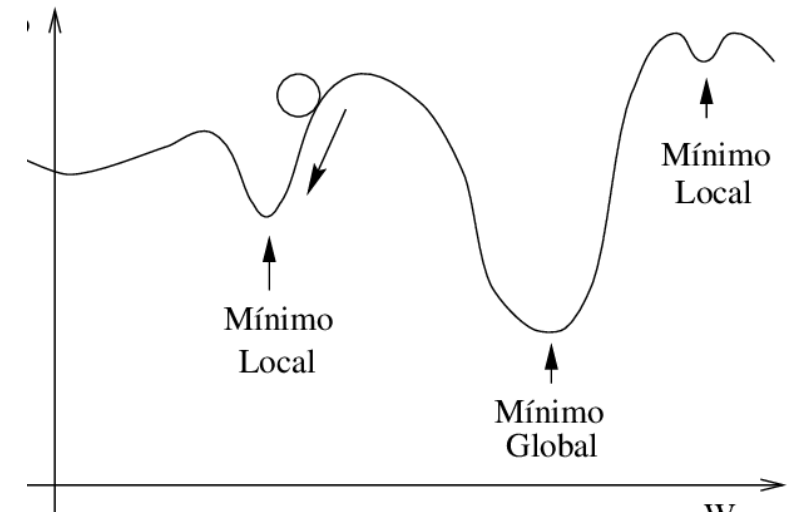
```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(X_train.shape[1],)),
3     keras.layers.Dense(64, activation='softmax'),
4     keras.layers.Dense(32, activation='softmax'),
5     keras.layers.Dense(1, activation='linear'),
6 ])
7
8 opt = keras.optimizers.Adam(learning_rate=0.001)
9 model.compile(
10     optimizer='sgd',
11     loss='mse',
12     metrics=['mae']
13 )
14
15 model.summary()
```

✓ 0.0s

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 6)	0
dense_6 (Dense)	(None, 64)	448
dense_7 (Dense)	(None, 32)	2080
dense_8 (Dense)	(None, 1)	33

Total params: 2,561
Trainable params: 2,561
Non-trainable params: 0



Redes Neurais - hiperparâmetros

Algoritmo de otimização

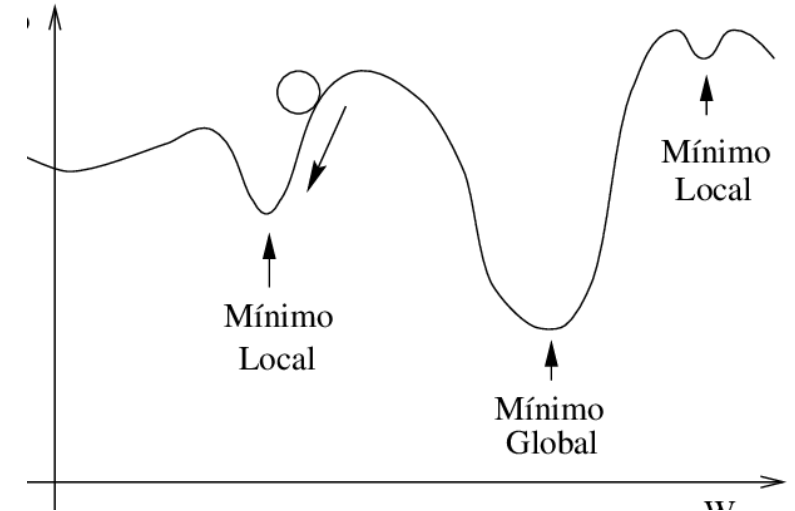


O processo geralmente envolve as seguintes etapas, realizadas durante o treinamento da rede:

1 - **Cálculo da Perda:** A rede faz uma previsão para uma entrada e a função de custo (ex: Erro Quadrático Médio, Entropia Cruzada) mede o "quão errado" o resultado está, gerando um valor de perda.

2 - **Cálculo do Gradiente:** O algoritmo usa o backpropagation para calcular o gradiente da função de custo em relação a cada peso e viés da rede. O gradiente é um vetor que aponta para a direção de maior aumento da função de perda.

3 - **Atualização dos Parâmetros:** O otimizador utiliza o gradiente para determinar a direção e a magnitude do ajuste que deve ser feito nos pesos e vieses. Como queremos minimizar a perda, a atualização é feita na direção oposta ao gradiente.

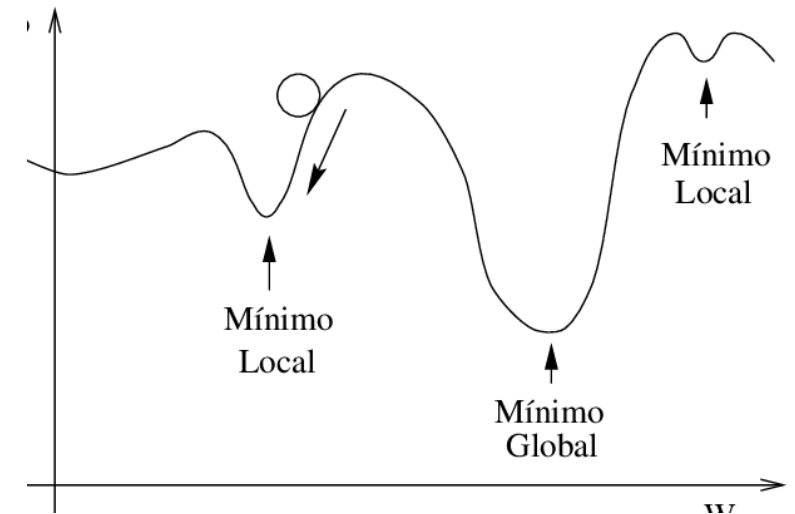


Redes Neurais - hiperparâmetros



Algoritmo de otimização

Algoritmo	Descrição Breve
Descida do Gradiente Estocástico (SGD)	O método base. Atualiza os parâmetros usando o gradiente de um único exemplo (ou um pequeno <i>batch</i>) por vez.
SGD com Momentum	Adiciona um termo de "momento" (inércia) que ajuda o otimizador a acelerar em direções consistentes e a atravessar mínimos locais superficiais.
Adagrad	Adapta a taxa de aprendizado, tornando-a inversamente proporcional à raiz quadrada da soma dos gradientes passados. É útil para dados esparsos.
RMSprop	Semelhante ao Adagrad, mas usa uma média móvel exponencial dos gradientes quadráticos para evitar que a taxa de aprendizado caia muito rapidamente.
Adam (Adaptive Moment Estimation)	Combina os conceitos de Momentum e RMSprop . É geralmente o otimizador mais popular e robusto, adaptando as taxas de aprendizado para cada parâmetro.



Redes Neurais - hiperparâmetros



Função de Perda

A **função de perda** (ou **função de custo**, ou **loss function**) em redes neurais é uma fórmula matemática que quantifica a margem de erro de um modelo. Ela mede a discrepância entre o valor previsto pela rede e o valor real para um dado exemplo de treinamento.

```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(X_train.shape[1],)),
3     keras.layers.Dense(64, activation='softmax'),
4     keras.layers.Dense(32, activation='softmax'),
5     keras.layers.Dense(1, activation='linear'),
6 ])
7
8 opt = keras.optimizers.Adam(learning_rate=0.001)
9 model.compile(
10     optimizer='sgd',
11     loss='mse',
12     metrics=['mae']
13 )
14
15 model.summary()
```

✓ 0.0s

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 6)	0
dense_6 (Dense)	(None, 64)	448
dense_7 (Dense)	(None, 32)	2080
dense_8 (Dense)	(None, 1)	33

Total params: 2,561
Trainable params: 2,561
Non-trainable params: 0

É crucial entender que a métrica **NÃO** é o mesmo que a **função de perda (ou custo)**, embora ambas sejam calculadas sobre as previsões da rede:

•Função de Perda (loss):

- É a função **central** que o algoritmo de otimização **tenta minimizar**.
- É usada para calcular o **gradiente** e atualizar os pesos da rede (é *diferenciável*).

•Métrica (metrics):

- É usada para **avaliar e reportar** o desempenho do modelo em termos **compreensíveis para humanos**.
- O algoritmo de otimização **não** usa a métrica diretamente para ajustar os pesos.

Em resumo: A perda guia o treinamento; a métrica mede o sucesso do treinamento.

Redes Neurais - hiperparâmetros



Função de Perda

1. Para Problemas de Regressão (Previsão de Valores Contínuos)

- **Erro Quadrático Médio (Mean Squared Error - MSE):** Calcula a média dos quadrados das diferenças entre os valores previstos e reais.

- $$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Característica:** Penaliza erros grandes de forma **muito severa** devido ao termo quadrático, tornando-a sensível a *outliers*.
- **Erro Absoluto Médio (Mean Absolute Error - MAE):** Calcula a média dos valores absolutos das diferenças.

- $$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Característica:** É mais **robusta a outliers** que o MSE, pois a penalidade aumenta linearmente com o erro.

2. Para Problemas de Classificação (Previsão de Classes/Categorias)

- **Entropia Cruzada Binária (Binary Cross-Entropy):** Usada para problemas de **classificação binária** (duas classes, ex: sim/não, 0/1).
 - **Característica:** Pensa no erro como uma divergência entre as distribuições de probabilidade previstas e reais. Penaliza fortemente previsões incorretas de alta confiança.
- **Entropia Cruzada Categórica (Categorical Cross-Entropy):** Usada para problemas de **classificação multi-classe** (mais de duas classes, ex: gato, cachorro, pássaro).
 - **Característica:** Semelhante à binária, mas generalizada para medir o erro quando há múltiplas classes.

Redes Neurais - hiperparâmetros



Métrica

1. Para Problemas de Classificação

Essas métricas medem a precisão com que a rede atribui exemplos às suas classes corretas:

- **Acurácia (`accuracy`)**: A métrica mais básica. Mede a proporção de previsões corretas em relação ao total de previsões.
- **Precisão (`Precision`)**: De todas as vezes que a rede previu uma classe positiva, quantas estavam realmente corretas.
- **Recall (`Recall` / **Sensibilidade**)**: De todos os exemplos que eram realmente positivos, quantos a rede identificou corretamente.
- **F1-Score**: A média harmônica da Precisão e do Recall. É útil quando há um desequilíbrio entre as classes.
- **AUC (Area Under the Curve)**: A área sob a Curva ROC, que mede a capacidade do modelo de distinguir entre classes.

2. Para Problemas de Classificação (Previsão de Classes/Categorias)

- **Entropia Cruzada Binária (Binary Cross-Entropy)**: Usada para problemas de **classificação binária** (duas classes, ex: sim/não, 0/1).
 - **Característica**: Pensa no erro como uma divergência entre as distribuições de probabilidade previstas e reais. Penaliza fortemente previsões incorretas de alta confiança.
- **Entropia Cruzada Categórica (Categorical Cross-Entropy)**: Usada para problemas de **classificação multi-classe** (mais de duas classes, ex: gato, cachorro, pássaro).
 - **Característica**: Semelhante à binária, mas generalizada para medir o erro quando há múltiplas classes.

Redes Neurais - hiperparâmetros



compile

fit

```
compile(  
    optimizer='rmsprop',  
    loss=None,  
    loss_weights=None,  
    metrics=None,  
    weighted_metrics=None,  
    run_eagerly=False,  
    steps_per_execution=1,  
    jit_compile='auto',  
    auto_scale_loss=True
```

```
model.fit(  
    X_train,  
    y_train,  
    epochs=100,  
    batch_size=32,  
    verbose=0  
)
```

epochs:

Quantas serão as rodadas de forward e backpropagation?

batch_size:

Número de exemplos de treinamento usados em uma iteração.

verbose:

Envia (1) ou não (0) mensagens durante o treinamento

Redes Neurais - hiperparâmetros



```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(X_train.shape[1],)),
    keras.layers.Dense(64, activation='softmax'),
    keras.layers.Dense(32, activation='softmax'),
    keras.layers.Dense(1, activation='linear'),
])

model.compile(
    optimizer='sgd', # Stochastic Gradient Descent (SGD).
    loss='mse',
    metrics=['mae']
)
```

```
model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=32,
    verbose=0
)
```

Redes Neurais - hiperparâmetros



```
# Define the EarlyStopping callback
early_stopping = EarlyStopping(
    monitor='val_loss', # Monitor validation loss
    mode='min',         # Minimize validation loss
    patience=10,        # Stop after 10 epochs with no improvement
    min_delta=0.001,    # Minimum change to be considered an improvement
    restore_best_weights=True # Restore best weights
)
```

```
# Train the model with early stopping
history = model.fit(
    X_train, y_train,
    epochs=100,
    validation_data=(X_val, y_val),
    callbacks=[early_stopping],
    verbose=1
)
```


Redes Neurais + GridSearchCV



```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.wrappers.scikit_learn import KerasClassifier
from sklearn.model_selection import GridSearchCV
# ... (Importação de dados e pré-processamento)
```

Redes Neurais + GridSearchCV



1. Função de Criação do Modelo

```
def create_model(optimizer, activation1, activation2):  
    model = keras.Sequential([  
        keras.layers.Flatten(input_shape=(X_train.shape[1],)),  
        keras.layers.Dense(64, activation=activation1),  
        keras.layers.Dense(32, activation=activation2),  
        keras.layers.Dense(1, activation='linear'),  
    ])  
  
    model.compile(  
        optimizer=optimizer,  
        loss='mse',  
        metrics=['mae']  
    )  
    return model
```

Redes Neurais + GridSearchCV



```
# 2. Envolvimento do Modelo Keras
# O build_fn aponta para a função que cria o modelo.
# Você também pode fixar hiperparâmetros de treinamento como epochs e batch_size
# dentro de KerasClassifier.

keras_model = KerasClassifier(build_fn=create_model, verbose=0)
```

Redes Neurais + GridSearchCV



```
# 3. Definição da Grade de Hiperparâmetros
# 3. Definição da Grade de Hiperparâmetros
param_grid = {
    'optimizer': ['SGD', 'RMSprop', 'Adam'],
    'epochs': [10, 50], # Hiperparâmetro passado para .fit() do Keras
    'batch_size': [5, 10, 30], # Hiperparâmetro passado para .fit() do Keras
    'activation1': ['relu', 'tanh', 'sigmoid', 'softmax'], # Hiperparâmetro passado para a camada Dense
    'activation2': ['relu', 'tanh', 'sigmoid', 'softmax'], # Hiperparâmetro passado para a camada Dense
}

# 4. Criação e Execução do GridSearchCV
grid = GridSearchCV(estimator=keras_model, param_grid=param_grid, n_jobs=-1, cv=3)

# O gridsearch fará o fit e o cross-validation
grid_result = grid.fit(X_train, y_train, callbacks=[early_stopping])
```




Vamos
praticar?

LIMC-IA

Python Machine Learning

Algoritmos Machine Learning

Guilherme Silveira
gfsilveira@gmail.com

08/08/2026